

The ToLO IES5 Report

The ToLO IES5 Report

Exported on 04/25/2024

Table of Contents

1	ToLO IES5 Report.....	8
1.1	Broad roadmap for future evolution	8
1.2	Other report sections	9
1.3	ToLO conceptual model	10
1.4	Example-based empirical development two pronged approach	11
1.5	Top Level Ontology (ToLO) architecture overview	12
1.5.1	Core Constructional Ontology (CoCO) architecture	13
1.5.1.1	Current Status	14
1.5.1.2	Work-in-Progress	14
1.5.1.3	Future Work	14
1.5.1.4	Constructionalism of the CoCO - overview	15
1.5.1.5	Work-in-Progress - plurals - formalising collections.....	17
1.5.1.6	Future Work - tupleology - radical adaption-evolution	18
1.5.2	Foundation Level Ontology (FoLO) architecture	22
1.5.2.1	Current Work	22
1.5.2.2	Future Work	22
1.5.2.3	Future Work - building a dimensional worldframewology.....	23
1.5.3	Common Level Ontology (CoLO) architecture	24
1.5.3.1	Our approach to Counterpart Theory.....	25
1.5.3.2	Future Work - evolving onomatology	30
1.6	Additional Future Work.....	31
1.6.1	Future Work - increasing expressiveness.....	32
1.6.1.1	Historic Work.....	32
1.6.1.2	Current Work	32
1.6.1.3	Future work.....	33
1.6.2	Future Work - implementing an Ontology Multi-Language Support Framework	34

1.6.2.1	Future Work	35
1.6.3	Future Work - ensuring the consistency of the ToLO	36
1.6.4	Future Work - formalising testimony, including belief management.....	37
1.6.4.1	Empirical - examples.....	37
1.6.4.2	Background literature review	37
1.6.5	Future Work - exploiting opportunities for formalising and automating a controlled doxastology	38
1.6.6	Future Work - formalising relative possibilities.....	39
1.7	Appendices.....	40
1.7.1	Appendix - Intelligence Examples	41
1.7.1.1	Use Cases.....	42
1.7.1.2	Examples	44
1.7.2	Appendix - CLIF Formalisation Ecosystem	52
1.7.3	Appendix - Ontology Multi-Language Support Framework	53
1.7.3.1	Introduction	53
1.7.3.2	OMSF - Architecture (Proposed)	54
1.7.3.3	OMSF - Language/Storage Format.....	66
1.7.3.4	OMSF - Tools.....	101
1.7.4	Appendix - Work-in-Progress.....	144
1.7.5	Appendix - Future Work	145
1.8	Bibliography.....	146
1.8.1	(Florio 2021 The Many and the One)	148
1.8.2	(Florio 2022 Core Constructional Ontology)	149
1.8.3	(Graham 2015 NORSOK Z-014: A 21st Century Update).....	150
1.8.4	(Hansson 2022 Logic of Belief Revision)	151
1.8.5	(ISO/IEC 21838-1:2021).....	152
1.8.6	(ISO/IEC 24707:2018).....	153
1.8.7	(Leonard 2023 Epistemological Problems of Testimony)	154
1.8.8	(Lewis 1968 Counterpart Theory and Quantified Modal Logic).....	155

1.8.9	(Lewis 1986 On the plurality of worlds).....	156
1.8.10	(Linnebo 2022 Plural Quantification).....	157
1.8.11	(Pagin 2021 Assertion).....	158
1.8.12	(Partridge 1996 Business Objects).....	159
1.8.12.1	(pp. 214-215) here and now - de se and nunc.....	160
1.8.12.2	(pp. 220, 227) object patterns - individual and class levels	164
1.8.12.3	patterns - derived	168
1.8.13	(Partridge 2011 An Information Model for Geospatial and Temporal References)	170
1.8.14	(Partridge 2013 Geospatial and Temporal Reference).....	171
1.8.15	(Partridge 2016 Formalization of the classification pattern).....	172
1.8.16	(Partridge 2018 Ontology then Agentology)	173
1.8.17	(Partridge 2018 Thoroughly Modern Accounting).....	174
1.8.18	(Partridge 2019 Coordinate Systems).....	175
1.8.19	(Partridge 2019 Grounding for an Enterprise Computing Nomenclature Ontology)	176
1.8.20	(Partridge 2020 Implicit Requirements for Ontological Multi-Level Types in the UNICLASS Classification)	177
1.8.21	(Partridge 2022 Top-Level Categories)	178
1.8.22	(Pigozzi 2021 Belief Merging and Judgment Aggregation).....	179
1.8.23	(Schaffer 2015 What Not to Multiply Without Necessity)	180
1.9	Acknowledgements	181
1.9.1	Lead Authors:.....	181
1.9.2	Authors:	181
1.9.3	Contributors:.....	181
2	Resources.....	182
2.1	ToLO Models.....	183
2.1.1	ToLO EA (qea file) Current Model	184
2.1.2	ToLO EA (qea file) Model including WIP and FW.....	185
2.2	Graphics	186

2.2.1	Graphics - Documents	187
2.2.1.1	Graphics - (Partridge 1996 Business Objects)	188
2.2.2	Graphics - ToLO Architecture Framework	194
2.2.2.1	CoCO breakdown to components with dependencies (circle)	195
2.2.2.2	CoCO breakdown to components with dependencies (wall)	196
2.2.2.3	CoCO breakdown to modules with dependencies	197
2.2.2.4	CoCO detailed breakdown to mortar with dependencies	198
2.2.2.5	CoLO breakdown to modules with dependencies	199
2.2.2.6	FoLO breakdown to components with dependencies	200
2.2.2.7	ToLO breakdown to data sections with dependencies	201
2.3	Resources - Ontology Multi-Language Support Framework (OMLSF)	202
2.3.1	Graphics - OMLSF - Architecture (Proposed)	203
2.3.1.1	simple hub and spoke structure	204
2.3.1.2	two-level hub and spoke structure	205
2.3.1.3	Option - UML Hub (CL to TPTP)	206
2.3.1.4	Option - UML Hub (TPTP to CL)	207
2.3.1.5	Option - CL Hub	208
2.3.1.6	Option - UML and CL Hub	209
2.3.1.7	UML as hub	210
2.3.1.8	ODM Overview	211
2.3.2	Languages List	212
2.3.2.1	Candidate Industry Standards Languages	212
2.3.2.2	Other Candidate Languages	213
2.4	bOSON 1.1 (bCLEARer Ordnance Survey Open Names) Surface Onomatology	215
2.5	bOSON 1.2 (bCLEARer Ordnance Survey Open Names) Deep Onomatology - Coordinates	216
2.5.1	Repository	216
2.5.2	Resources	216
2.6	IES 4.0	217

2.7 Issued Document Identifier 218



Issued Document Identifier

TL/IN371/P04231771/02

1 ToLO IES5 Report

The report outlines the work done by BORO on the IES5 project. This was in two main (interrelated) areas:

- The production of a conceptual model for the base IES5 top level ontology - ToLO
- The investigation of including counterpart theory in ToLO

The conceptual model in a couple of formats is linked to in this section: [ToLO conceptual model \(see page 10\)](#).

The analysis of the counterpart theory built upon the base ontology, taking a two prong empirical approach based around examples - described here: [Example-based empirical development two pronged approach \(see page 11\)](#)

A key part of this exercise was providing an overarching architecture for the ToLO, including both areas. This is done in the section [Top Level Ontology \(ToLO\) architecture overview \(see page 12\)](#).

This architecture has these three levels:

- [Core Constructional Ontology \(CoCO\) architecture \(see page 13\)](#)
- [Foundation Level Ontology \(FoLO\) architecture \(see page 22\)](#)
- [Common Level Ontology \(CoLO\) architecture \(see page 24\)](#)

As it turns out the boundaries between these levels also mark out the boundary between the two work areas. The base IES5 top level ontology currently resides in the first two levels - and counterpart theory in the third level.

1.1 Broad roadmap for future evolution

This architecture provides a broad framework for a roadmap for the future evolution of the ToLO. The elements of the architecture are at different stages of evolution. One can roughly breakdown these stages as:

- current work
- work-in-progress
- future work

The currently identified areas for work in progress and future work are summarised in lists of links in their respective appendices - excerpted below. Linked details are in the body of the report:

Appendix - Work-in-Progress

-  [Work-in-Progress - evolving counterpartology \(see page 26\)](#)
-  [Work-in-Progress - evolving doxastology \(see page 28\)](#)
-  [Work-in-Progress - evolving egonology \(see page 27\)](#)
-  [Work-in-Progress - evolving modalology \(see page 29\)](#)

 [Work-in-Progress - plurals - formalising collections \(see page 17\)](#)

Appendix - Future Work

-  [Future Work - building a dimensional worldframewology \(see page 23\)](#)
-  [Future Work - ensuring the consistency of the ToLO \(see page 36\)](#)
-  [Future Work - evolving onomatology \(see page 30\)](#)
-  [Future Work - exploiting opportunities for formalising and automating a controlled doxastology \(see page 38\)](#)
-  [Future Work - formalising testimony, including belief management \(see page 37\)](#)
-  [Future Work - implementing an Ontology Multi-Language Support Framework \(see page 34\)](#)
-  [Future Work - increasing expressiveness \(see page 32\)](#)
-  [Future Work - tupleology - radical adaption-evolution \(see page 18\)](#)

1.2 Other report sections

The development approach is described in this section: [Example-based empirical development two pronged approach \(see page 11\)](#).

Finally, the report contains these appendices:

- [Appendix - Intelligence Examples \(see page 41\)](#)
- [Appendix - CLIF Formalisation Ecosystem \(see page 52\)](#)
- [Appendix - Ontology Multi-Language Support Framework \(see page 53\)](#)
- [Appendix - Work-in-Progress \(see page 144\)](#)
- [Appendix - Future Work \(see page 145\)](#)

1.3 ToLO conceptual model

A 'conceptual' model of the current ToLO has been constructed in UML.

The model is structured to reflect the underlying ToLO architecture - described in a later section ([Top Level Ontology \(ToLO\) architecture overview \(see page 12\)](#)).

This architecture provides a broad framework for a roadmap for the future evolution of the ToLO. The elements of the architecture are at different stages of evolution. One can roughly breakdown these stages as:

- current work
- work-in-progress
- future work

The full model reflects these stages, in other words it contains architectural elements that are at work-in-progress and future work stages - these are clearly marked. To assist with distribution, we have created two versions of the current EA ToLO Model. The first contains just current modules - so a pure current model ([ToLO EA \(qea file\) Current Model \(see page 184\)](#)). The second contains the full set of modules ([ToLO EA \(qea file\) Model including WIP and FW \(see page 185\)](#)) Both are provided as zipped EA Sparx *.qea files.

To enable people without access to an EA file reader to review the model, we have also provided an HTML version of the model (including work-in-progress and future modules) in this report. It can be opened in a separate tab using this link.

1.4 Example-based empirical development two pronged approach

The ToLO's counterparty theory is based upon David Lewis's work as described in this section: [Our approach to Counterpart Theory \(see page 25\)](#). This also describes the architecture based work breakdown into modules. Across these modules we adopted a common approach shaped by an empirical backbone based upon examples.

The approach had two aspects:

1. (rational) - background literature review
2. empirical - through the use of specific examples

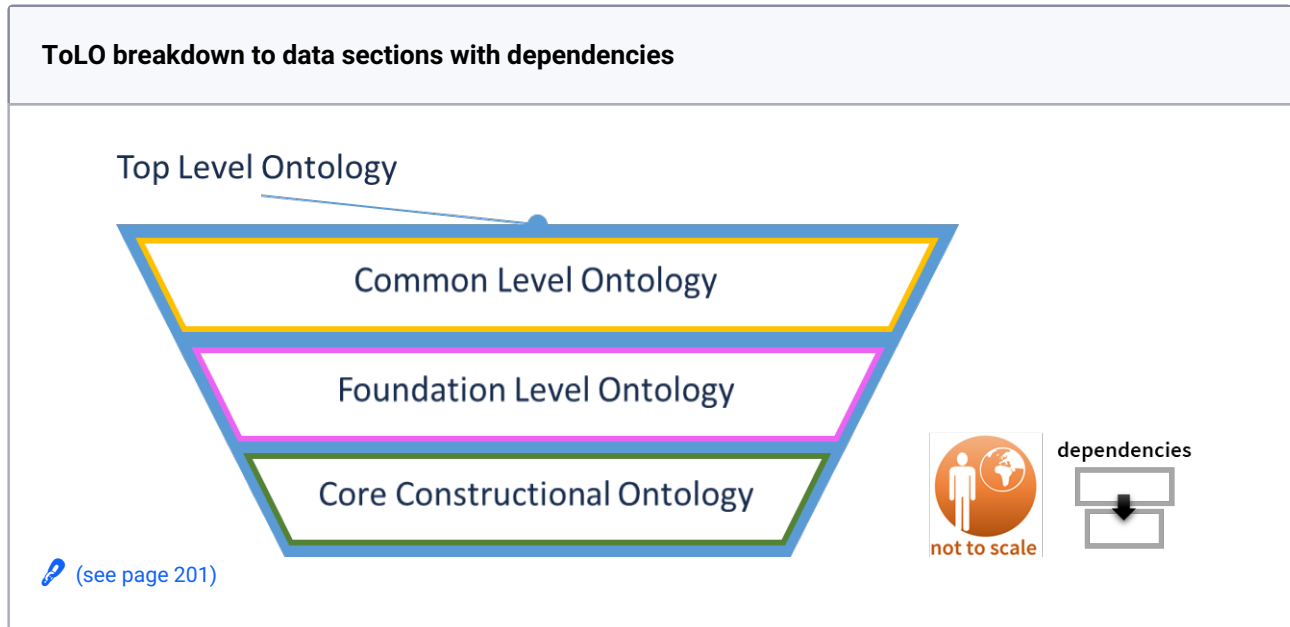
Obviously there is an interplay between these, where the review suggests examples and the examples suggest topics for investigation.

Lewis's counterparty theory evolved over time, going through a number of shifts, developing into a mature sophisticated system. It made sense for us to follow a similar path. To start by looking at how to accommodate the simpler examples, with a simpler theory. And so demonstrate what can be practically done with this, developing confidence in its worth. And then looking at how to accommodate the more sophisticated cases with a more sophisticated theory, further developing confidence in the theory. The background literature review helps us to identify the path we take - and the kinds of issues the examples need to address along the path.

For more details on the examples see: [Appendix - Intelligence Examples \(see page 41\)](#).

1.5 Top Level Ontology (ToLO) architecture overview

The top level ontology (ToLO) architecture breaks down into three sections, as shown below.

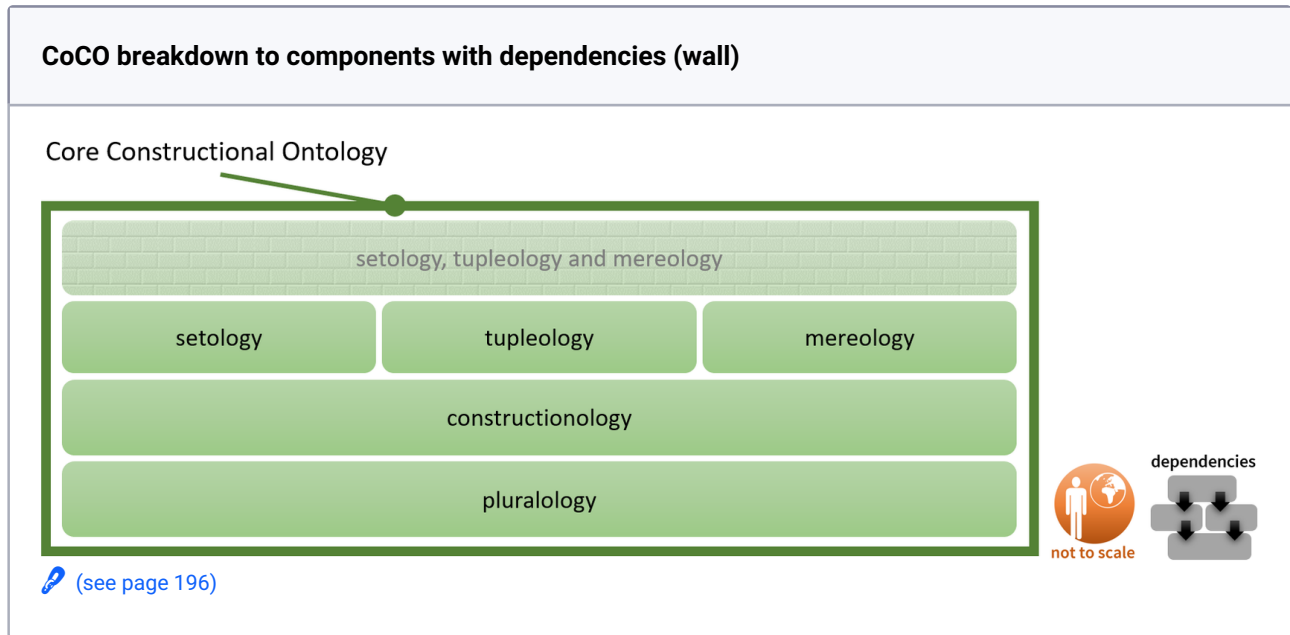


- [Core Constructional Ontology \(CoCO\) architecture \(see page 13\)](#)
- [Foundation Level Ontology \(FoLO\) architecture \(see page 22\)](#)
- [Common Level Ontology \(CoLO\) architecture \(see page 24\)](#)

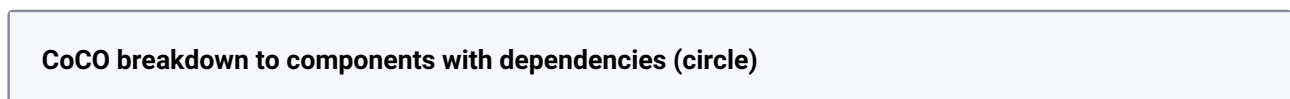
1.5.1 Core Constructional Ontology (CoCO) architecture

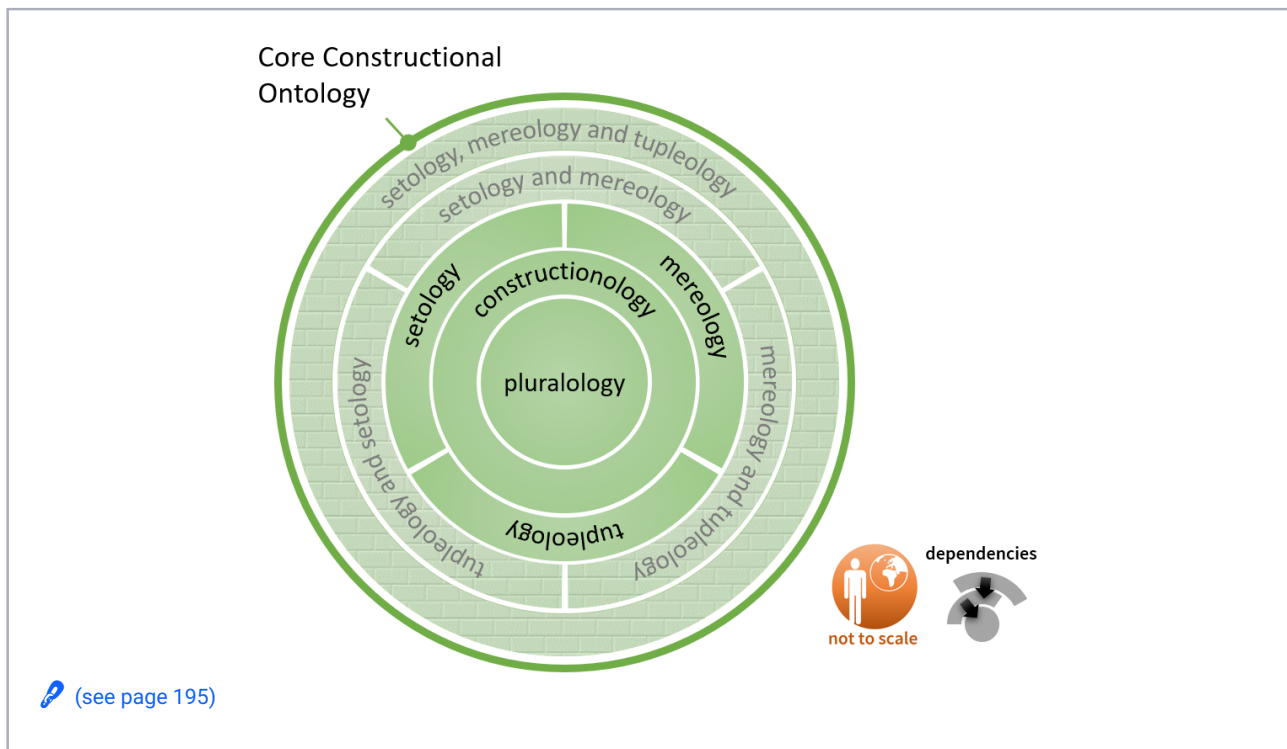
The Core Constructional Ontology has two foundation modules, pluralology and constructionology. On top of these are three modules, one for each ontological category: mereology, setology and tupleology. On top of these are the modules that integrate across the ontological categories.

This architecture is visualised below as a wall: we use the wall metaphor more generally across the ToLO architecture.



When we want to show the details of the integration modules clearly, we then visualise it as a circle - as shown below.





1.5.1.1 Current Status

We have developed an initial model for the CoCO based upon the NDT work.

The constructional approach central to the model is described here: [Constructionalism of the CoCO - overview \(see page 15\)](#)

1.5.1.2 Work-in-Progress

The pluralology at the foundation of the model is described in this sub-section - [Work-in-Progress - plurals - formalising collections \(see page 17\)](#) - as there is some outstanding work to formalise collections.

1.5.1.3 Future Work

Given the opportunities that ToLO's constructional nature provides for showing consistency, it is worth noting the possibility of taking advantage of them: see [Appendix - CLIF Formalisation Ecosystem \(see page 52\)](#).

There is also an opportunity for radical innovation on the conceptual model for relations, described here: [Future Work - tupleology - radical adaption-evolution \(see page 18\)](#). As the sub-section notes, we have started work, but there is still a significant amount of work to do.

1.5.1.4 Constructionalism of the CoCO - overview

This section give an overview of the CoCO's constructional approach, looking first at the basic constructional structure and then the main benefits. For more details see: ([Florio 2022 Core Constructional Ontology](#)) (see [page 149](#)).

1.5.1.4.1 Basic constructional structure

The basic constructional structure of the CoCO can be explained in terms of two features: it is foundational and it has a high degree of unification.

1.5.1.4.1.1 Foundational

Three characteristics enable the CoCO to play its foundational role. The first is its *categorical completeness*. That is, the CoCO provides the three basic categories of objects for the top ontology: sets, individuals, and tuples, together with their associated hierarchical relations (element-set, sub-super-set, part-whole, and tuple-place). The second characteristic is *object completeness*. That is, the ontology generates all the objects needed by the top ontology. One might think of it as an 'object factory' which supplies all the objects that might be needed in any domain. The third characteristic is that the basic categories provide objects in the ontology with appropriate *identity criteria*. These are broadly extensional, based on the type of constructor and its input. For example, two sets are identical if and only if they are constructed from the same objects. Similarly, two individuals are identical if and only if they have the same parts.

1.5.1.4.1.2 High degree of unification

The CoCO is unified in the following ways. First, it gives a *common development of three key domains*: sets, individuals, and tuples. Ontologies that involve sets and sum usually adopt set theory and mereology as separate theories, without an integrated development. We provide a unified treatment of sets, individuals and pairs (hence tuples) as *sui generis* objects. So, the three target domains—sets, individuals, and tuples—arise in similar ways through construction. Second, there is a *common basis for identity criteria*, which are crucial for the foundational role discussed above. Identity criteria for the objects of the basic types are extensional, with differences arising from the way the objects are constructed. Third, the approach offers *uniform ways of capturing key commonalities and differences among objects of the basic types*. Such commonalities and differences are captured by features of the underlying constructors.

1.5.1.4.2 Main benefits

This basic constructional structure has a number of benefits, we now highlight four main ones.

Categorical differences are constructional differences: in constructional approaches, different kinds of objects can be distinguished from one another by the manner in which they are constructed. So categorical differences are explained by constructional differences.

Dependency: some objects are built from others and hence “depend on” them. This provides an explanation of ontological dependence and the associated notion of grounding.

Reduction: the ontology is built out of a relatively small set of initial objects and thus achieves fundamental ontological parsimony in the sense of ([Schaffer 2015 What Not to Multiply Without Necessity](#)) (see [page 180](#)).

Consistency: construction can be a basis for consistency, avoiding paradoxes such as those of Russell and Burali-Forti, although we need to take care with the construction process. In CoCO's case, this is achieved by requiring that the construction be 'bottom-up' in the sense that the properties of the constructed objects are determined by the properties of the inputs to the construction.

1.5.1.5 Work-in-Progress - plurals - formalising collections

Plurals can be thought of as a way of talking about collections of things - for more details see ([Florio 2021 The Many and the One](#)) (see page 148) and also ([Linnebo 2022 Plural Quantification](#)) (see page 157).

They are useful in two ways for the ToLO.

Firstly, they provide a way about talking about the collections of objects that are applied to constructors without committing to sets. Plurals are introduced into the ToLO through the pluralology module. They provide the inputs for the constructional apparatus in the constructionology module.

Secondly, they provide a way of talking about collections of things that are too large to be constructed. There are well-known paradoxes in set theory (the most well-known is Russell's paradox) that can arise when sets (such as the universal set) are introduced as sets. With a plural approach, one can talk about the plurality of sets - so talk about all sets - without needing to introduce a universal set. Similarly for other collections that are too large to be sets.

One can broadly think of plurals as a way to have set-like collections of things, including sets, while differing from sets so as to avoid paradoxes. However, the same size issue emerges for plurals, when one considers the plurality of all plurals, this is too big to be a plural. It is useful in top level ontologies to have a top 'object' which subsumes all the other objects. We do this by introducing collections, a way of having plural-like collections. ([Florio 2022 Core Constructional Ontology](#)) (see page 149) provides a formalisation of plurals. There is a requirement to provide a formalisation of collections in the future.

1.5.1.6 Future Work - tupleology - radical adaption-evolution

1.5.1.6.1 Historic work

BORO has undertaken over the decades significant internally funded research into the nature of relations. There is a reasonably well-established literature on the shortcomings of the current formalisations of relations. We excerpt from the results of a literature survey in the next section, [tupleology - brief literature survey excerpt \(see page 19\)](#), to provide some insight into the state-of-the-art.

While there is a general acceptance that the current formalisation of relations is inadequate, there appears to be (as yet) no clear way forward. The approaches to date (including until recently, BORO's) have been based upon armchair speculation. BORO is now of the opinion that an empirical, implementation and testing strategy is the most likely way to identify a new scalable approach. BORO has piloted a constructional sandpit to empirically test the various existing approaches to constructing relations, to provide a foundation for devising new ways of formalising them.

1.5.1.6.2 Future work

A new, better, formalisation of relations that resolved the known shortcoming is likely to have an energising effect on modelling, enabling new, better ways of modelling.

BORO believes that there is a good chance of finding this new formalisation through the use of tools such as its constructional relation sandpit. The aim would be to firstly empirically test the various existing approaches to constructing relations to build a solid foundation. Then this could be used to search for better formalisations.

1.5.1.6.3 tupleology - brief literature survey excerpt

Listed below is an excerpt from the results of a brief literature survey on relations, with a focus on constructional approaches.

1.5.1.6.3.1 Early

Early Philosophy

Aristotle. (n.d.). The categories.

Russell, B. (1903). Principles of mathematics. Cambridge University Press.

Hochberg, H. (n.d.-a). "Russell's early analysis of relational predication and the asymmetry of the predication relation."

Early Set Theory

Wiener, N. (1912). A simplification of the logic of relations.

Kuratowski, C. (1921). "Sur la notion de l'ordre dans la théorie des ensembles. Fundamenta Mathematicae, 2(1), 161–171.

Enderton, H. B. (1977). Elements of set theory. Academic press.

Hochberg, H. (n.d.-b). "The Wiener-Kuratowski Procedure and the Analysis of Order."

Early IT

Codd, E. F. (1970). "A relational model of data for large shared data banks." Communications of the ACM, 13(6), 377–387.

Chen, P. P. S. (1976). "The entity-relationship model—toward a unified view of data." ACM Transactions on Database Systems (TODS), 1(1), 9–36.

Codd, E. F. (1990). The relational model for database management: version 2. Addison-Wesley Longman Publishing Co., Inc.

1.5.1.6.3.2 Critiques of the standard position

Williamson, T. (1985). "Converse relations." The Philosophical Review, 94(2), 249–262.

Fine, K. (2000). "Neutral relations." The Philosophical Review, 109(1), 1–33.

Ontology Sandpit - Constructionalism

Fine, K. (1991). "The Study of Ontology." Noûs, 25(3), 263–294.

Harel, D., Kozen, D. & Tiuryn, J. (2000). Dynamic logic. Dynamic logic. The MIT Press.

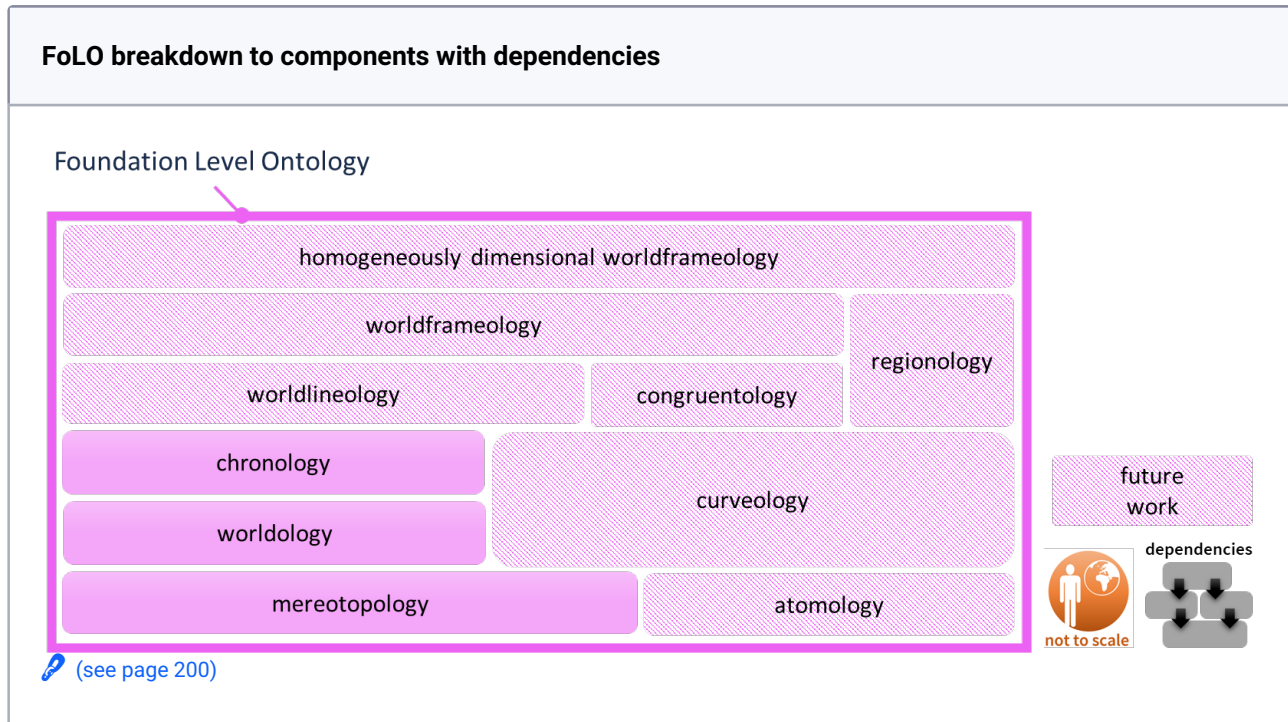
1.5.1.6.3.3 Other

- Bacon, A. (2020). "A Theory of Structured Propositions."
- Dixon, T. S. (2018). "Plural slot theory." *Oxford Studies in Metaphysics*.
- Dixon, T. S. (2019). "Relative Positionalism and Variable Arity Relations." *Metaphysics*.
- Donnelly, M. (2016). "Positionalism revisited." *The Metaphysics of Relations*. Oxford: Oxford University Press. Doi, 10.
- Dorr, C. (2004). "Non-symmetric relations." *Oxford Studies in Metaphysics*, 1(155-92), 73Notice.
- Dummit, D. S. & Foote, R. M. (2004). *Abstract Algebra* (3rd ed.). Wiley Hoboken.
- Effingham, N. (2020). "Slot Theory and Slotite Theory." *Philosophia*, 1–19.
- Fine, K. (2007). "Response to Fraser MacBride." *Dialectica*, 61(1), 57–62.
- Gaskin, R. & Hill, D. J. (2012). "On neutral relations." *Dialectica*, 66(1), 167–186.
- Gilmore, C. (2013). "Slots in universals." *Oxford Studies in Metaphysics*, Volume 8, 187.
- Gilmore, C. (2014). "Parts of propositions."
- Hungerford, T. W. (2014). *Abstract algebra: an introduction*. Cengage Learning.
- Leo, J. (2008). "Modeling relations." *Journal of Philosophical Logic*, 37(4), 353–385.
- Leo, J. (2013). "Coordinate-free Thinking." In *Proceedings of the Annual Meeting of the Cognitive Science Society* (Vol. 35).
- Leo, J. (2014). "Thinking in a Coordinate-Free Way about Relations." *Dialectica*, 68(2), 263–282.
- Levine, J. (2014). "Russell, Particularized Relations and Bradley's Dilemma." *Dialéctica*, 68(2), 231–261.
- Linnebo, Ø. (2017). "Plural quantification." *Stanford Encyclopedia of Philosophy*. Retrieved from <https://plato.stanford.edu/entries/plural-quant/>
- MacBride, F. (2007). "Neutral relations revisited." *Dialectica*, 61(1), 25–56.
- MacBride, F. (2011). "Relations and truthmaking." In *Proceedings of the Aristotelian Society* (Hardback) (Vol. 111, pp. 161–179).
- Orilia, F. (2011). "Relational order and onto-thematic roles." *Metaphysica*, 12(1), 1–18.
- Orilia, F. (2014). "Positions, Ordering Relations and O-Roles." *Dialéctica*, 68(2), 283–303.
- Van Inwagen, P. (2006). "Names for relations." *Philosophical Perspectives*, 20, 453–477.
- Wieland, J. W. (2010). "Anti-positionalism's regress." *Axiomathes*, 20(4), 479–493.
- MacBride, Fraser, "Relations", *The Stanford Encyclopedia of Philosophy* (Winter 2020 Edition), Edward N. Zalta (ed.), forthcoming URL = <<https://plato.stanford.edu/archives/win2020/entries/relations/>>
- Partridge, C., de Cesare, S., Mitchell, A., Gailly, F. & Khan, M. (2017). "Developing an Ontological Sandbox: Investigating Multi-Level Modelling's Possible Metaphysical Structures." In *MULTI-4th International Workshop on Multi-Level Modelling* (Vol. 2019, pp. 226–234).
- Partridge, C., de Cesare, S., Mitchell, A. & Odell, J. (2018). "Formalization of the classification pattern: survey of classification modeling in information systems engineering." *Software & Systems Modeling*, 17(1), 167–203.

Partridge at al., “The Fantastic Combinations and Permutations of Coordinate Systems’ Characterising Options: The Game of Constructional Ontology”, manuscript

1.5.2 Foundation Level Ontology (FoLO) architecture

We have a reasonably complete target architecture for the foundation level, which is visualised below. Of the ten modules, there are three modules currently available: Mereotopology, Worldology and Chronology.



1.5.2.1 Current Work

As the graphic shows, the current work involved these three modules:

1. Mereotopology
2. Worldology, and
3. Chronology

Mereotopology provides the machinery for connection. Worldology uses the mereotopological machinery to define worlds as maximally connected individuals. Chronology then provides the machinery for (world-bound) time relations.

1.5.2.2 Future Work

In the next section, [Future Work - building a dimensional worldframewology \(see page 23\)](#), we briefly describe the opportunities for future work.

1.5.2.3 Future Work - building a dimensional worldframewology

1.5.2.3.1 Historic work

The CoCO level ([Core Constructional Ontology \(CoCO\) architecture](#) (see page 13)) is reasonably well established, though there is some future work required. The base modules of FoLO ([Foundation Level Ontology \(FoLO\) architecture](#) (see page 22)) are also well-established.

1.5.2.3.2 Future work

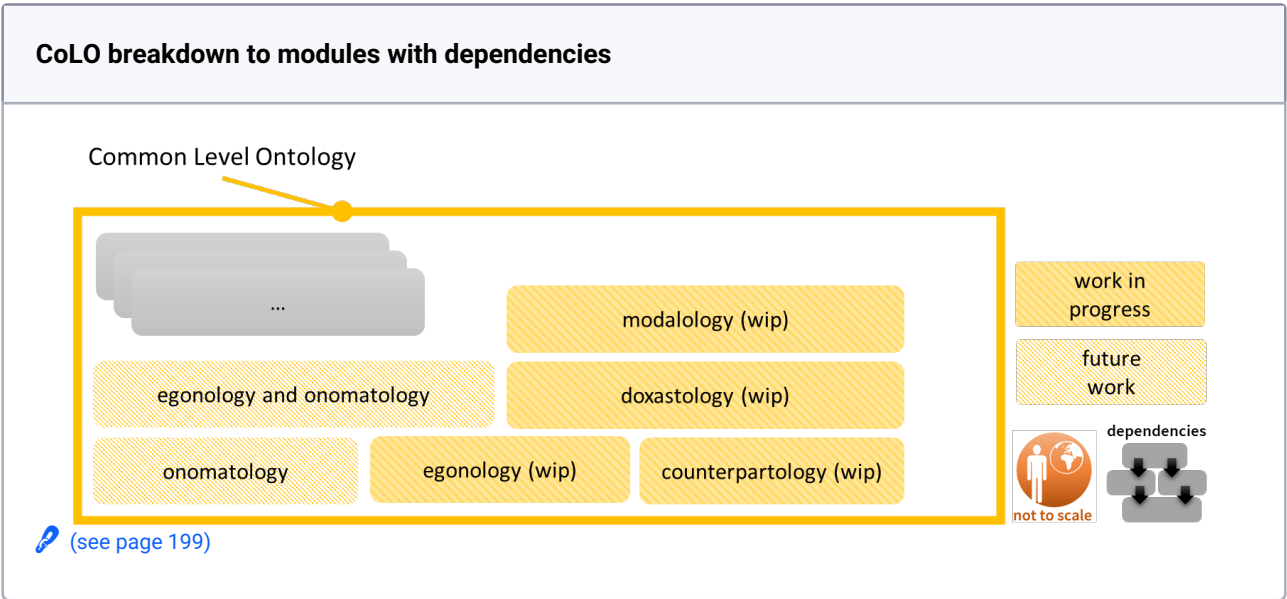
But there are also a significant number of modules that need to be developed to arrive at a full dimensional worldframeology (in the homogeneously dimensional worldframeology module). A central component of this will be 'worldlines' (introduced in the worldlineology module). For background on this approach see:

- ([Partridge 2011 An Information Model for Geospatial and Temporal References](#)) (see page 170)
- ([Partridge 2013 Geospatial and Temporal Reference](#)) (see page 171)
- ([Partridge 2019 Coordinate Systems](#)) (see page 175)

1.5.3 Common Level Ontology (CoLO) architecture

The Common Level Ontology (CoLO) architecture is emerging. This is to be expected as the range of modules that fall under the common level is likely to grow as the range of domains in which the ToLO is deployed increases.

A number of modules are currently being worked on as part of the work on counterpart theory (see [Our approach to Counterpart Theory \(see page 25\)](#)). These are counterpartology, egonology, doxastology and modalology. Modules identified for future work are onomatology including its integration with egonology (see [Future Work - evolving onomatology \(see page 30\)](#)). This, along with space for further emerging modules, is visualised as a wall below.



1.5.3.1 Our approach to Counterpart Theory

The four work-in-progress FoLO modules broadly cover what is, in homage to David Lewis, called counterpart theory:

- counterpartology
- egonology
- doxastology
- modalology

1.5.3.1.1 Our approach

The broad approach involved the two prongs (described here: [Example-based empirical development two pronged approach](#) (see page 11)). In this case, the background literature review played an important role in framing the approach.

The work reviewed Lewis's counterpart relation originally described in ([Lewis 1968 Counterpart Theory and Quantified Modal Logic](#)) (see page 155) and subsequent work including ([Lewis 1986 On the plurality of worlds](#)) (see page 156), where Lewis revised his original position. The significant literature surrounding counterpart theory was also reviewed. The review revealed a complex picture that had evolved over time - with a significant number of issues raised. So to simplify the management going forward two broad choices were made, both involving a kind of separation of concerns.

Firstly, a decision was to divide the architecture into four modules, each dealing with an aspect of counterpart theory, where modules built upon each other - as shown in the wall diagram.

Secondly, there was a decision to start with simple versions of the theory, leaving the more sticky issues until the simple ones had been resolved. More specifically, the decision was made to start with a simplified counterpart relation and work with examples that illustrated how the simple cases work.

We will follow this through all four modules, as they are closely interrelated. Once we have a consolidated overall picture, we will turn to more complicated cases.

1.5.3.1.2 Work-in-Progress - evolving counterpartology

1.5.3.1.2.1 Historic work

Counterpartology deals with counterpart relations in general, without any preferred world or speaker. A general, simple counterpart relation has been included in the BORO-based models from the early days. This will need refining for the more sophisticated counterpart theory requirements.

1.5.3.1.2.2 Current work

As a result of the architectural division into four modules, this module focuses on the counterpart relation, leaving closely related issues including actuality (see [Work-in-Progress - evolving egonology \(see page 27\)](#)) and doxastology (see [Work-in-Progress - evolving doxastology \(see page 28\)](#)) to their respective modules.

As an initial simplifying assumption, we start by treating the counterpart relation as an equivalence relation. We have been starting to consider examples for which this is sufficiently expressive to capture what is required.

1.5.3.1.2.3 Future work

Following the chosen approach, we plan to be example driven. We will create examples that will work with the simplified counterpart relation and explore how to model them. We will follow this through across all four modules, as they are closely interrelated.

Once this is firmly established, we will then look, step by step, at more complex counterpart relations. We will consider cases where the relation is one-to-many (and many-to-one). We will look at where there are counterparts in the same world.

1.5.3.1.3 Work-in-Progress - evolving egonology

1.5.3.1.3.1 Historic work

There is a general recognition that four-dimensional ontologies need to be supplemented with indexicals to, for example, mark out the present. In the initial BORO model ([Partridge 1996 Business Objects](#)) ([see page 159](#)) this was done with separate 'here' and 'now' event classes ([\(pp. 214-215\) here and now - de se and nunc](#) ([see page 160](#))). In early models the system itself was introduced into the model as 'This NF Agent' (TNFA) and often associated with the root naming spaces. Subsequently, BORO investigated further the relation between ontology and *de se*, both at the enterprise level ([Partridge 2018 Ontology then Agentology](#)) ([see page 173](#)) and specifically for accounting ([Partridge 2018 Thoroughly Modern Accounting](#)) ([see page 174](#)).

1.5.3.1.3.2 Current Work

In a number of places, including ([Lewis 1986 On the plurality of worlds](#)) ([see page 156](#)), Lewis describes the notion of an *actuality* as the basis for actuality and so belief in actuality. Where the actuality is the 'speaker' at a point in time of the utterance. He further claims that the *de se* subsumes the *de re*, that all *de re* claims are inherently *de se*.

For models we can think of each of these representing at every point in time they exist. Then, the actual world is the world in which the speaker exists.

Generally, we have a notion of 'MyActuality', which represents the timeslice of the 'Me' that is present, where both 'MyActuality' and 'Me' are indexicals - though they index in linked but different ways - what we might call *de se and nunc* and *de se*. The name 'MyActuality' is *de se and nunc* indexed (refers to) time slices in what might be called a 'presentist' way - it is indexed (refers to) to the present actuality - and this shifts as the present shifts. The name 'Me' is *de se* indexed (refers to) the 'speaker' - and this shifts as the speaker shifts.

From the perspective of an information system (such as a model), we have made these two objects explicit (and give them the appropriate names). The 'Me' object represents the model itself. The 'MyActuality' represents the present time slice of the system. We mark the different levels of indexing (*de se and nunc* and *de se*) with different stereotypes.

1.5.3.1.3.3 Future work

The system mechanisms for dealing with *de se* and *nunc* indexing need further investigation, especially in relation to belief revision. This is crucial for the operation of the system.

1.5.3.1.4 Work-in-Progress - evolving doxastology

1.5.3.1.4.1 Current Work

Doxastology is central to Lewis's mature counterpart theory. In egonology, we introduced the notion of 'MyActuality', which represents the timeslice of the 'Me' that is present when I am representing something- in the case of an IES Model-system, a time slice of the system when it is representing something. Lewis introduces (in [\(Lewis 1986 On the plurality of worlds\) \(see page 156\)](#)) the notion of doxastic alternative actualities - where such an alternative is the possible individual "who might, for all he believes, be himself. Individual X is one of them iff nothing that the believer believes, either explicitly or implicitly, rules out the hypothesis that he himself is X." This position firmly roots the model in the doxastically alternative worlds.

1.5.3.1.4.2 Future work

This doxastic approach appears to be closely aligned with the intelligence approach to information, where who asserts/believes the information is relevant. So it looks as if provides a way to formalise the intuitions that the community have about intelligence information (see [Future Work - exploiting opportunities for formalising and automating a controlled doxastology \(see page 38\)](#)). However, the details of this need to be worked out. Given the limits on the granularity of beliefs, there are likely to be a range of doxastic alternatives. This range has to be accomodated. One benefit of the range is that it neatly accommodates doxastic possibilities (see [Work-in-Progress - evolving modalology \(see page 29\)](#)). These also have to be worked out.

1.5.3.1.5 Work-in-Progress - evolving modalology

As the architecture ([Common Level Ontology \(CoLO\) architecture](#) (see page 24)) shows graphically, modalology is built on top of counterpartology, egonology and doxastology. A common way of thinking of modality is dealing with necessity and possibility. Modal statements are typically assigned truth values relative to a possible world. In Lewis's mature counterpart theory ([Lewis 1986 On the plurality of worlds](#)) (see page 156), one makes the assessment relative to a speaker at a time - often referred to as *de se* and *nunc*. In IES's case, the speakers include the various IES systems and their messages. One can regard the statements as being made by a time slice of the speaker (system/message), what Lewis called their *actuality*. To construct these, one needs the egonology module which supplies the actuality and so the *de se* and *nunc* . More specifically, one makes the assessment doxastically against the speaker's actualities' doxastic alternatives. To construct these one needs both the doxastology and counterpartology modules.

1.5.3.1.5.1 Current Work

With the modules in place, one has the beginnings of a framework for modelling what the speaker (at a time) believes in to be true (doxastically necessary), false (doxastically impossible) or possible. One can also model relative possibilities as well as belief revisions.

1.5.3.1.5.2 Future work

The broad framework is in place for modalology, but the details need a significant investment.

1.5.3.2 Future Work - evolving onomatology

1.5.3.2.1 Historic work

Onomatology deals with names. The basic original four-dimensional pattern was described in detail in [\(Partridge 1996 Business Objects\)](#) [\(see page 159\)](#). This pattern was inherited by the descendants of BORO, such as IDEAS. This soon evolved along with early versions of egonology [\(see Work-in-Progress - evolving egonology \(see page 27\)\)](#) into self-ascribed (or root) local naming spaces.

The pattern has more recently evolved as described in [\(Partridge 2019 Grounding for an Enterprise Computing Nomenclature Ontology\)](#) [\(see page 176\)](#). There are worked examples of the pattern in in code here: [bOSON 1.1 \(bCLEARer Ordnance Survey Open Names\) Surface Onomatology](#) [\(see page 215\)](#) and [bOSON 1.2 \(bCLEARer Ordnance Survey Open Names\) Deep Onomatology - Coordinates](#) [\(see page 216\)](#).

1.5.3.2.2 Future work

The latest pattern, from the recent papers, will be integrated into the model.

1.5.3.2.2.1 Integration of egonology and onomatology

The egonological aspects of the latest pattern, more specifically the self-ascribed (root) naming spaces, will also be integrated.

1.6 Additional Future Work

This section addresses the following topics.

- [Future Work - increasing expressiveness \(see page 32\)](#)
- [Future Work - implementing an Ontology Multi-Language Support Framework \(see page 34\)](#)
- [Future Work - ensuring the consistency of the ToLO \(see page 36\)](#)
- [Future Work - formalising testimony, including belief management \(see page 37\)](#)
- [Future Work - exploiting opportunities for formalising and automating a controlled doxastology \(see page 38\)](#)
- [Future Work - formalising relative possibilities \(see page 39\)](#)

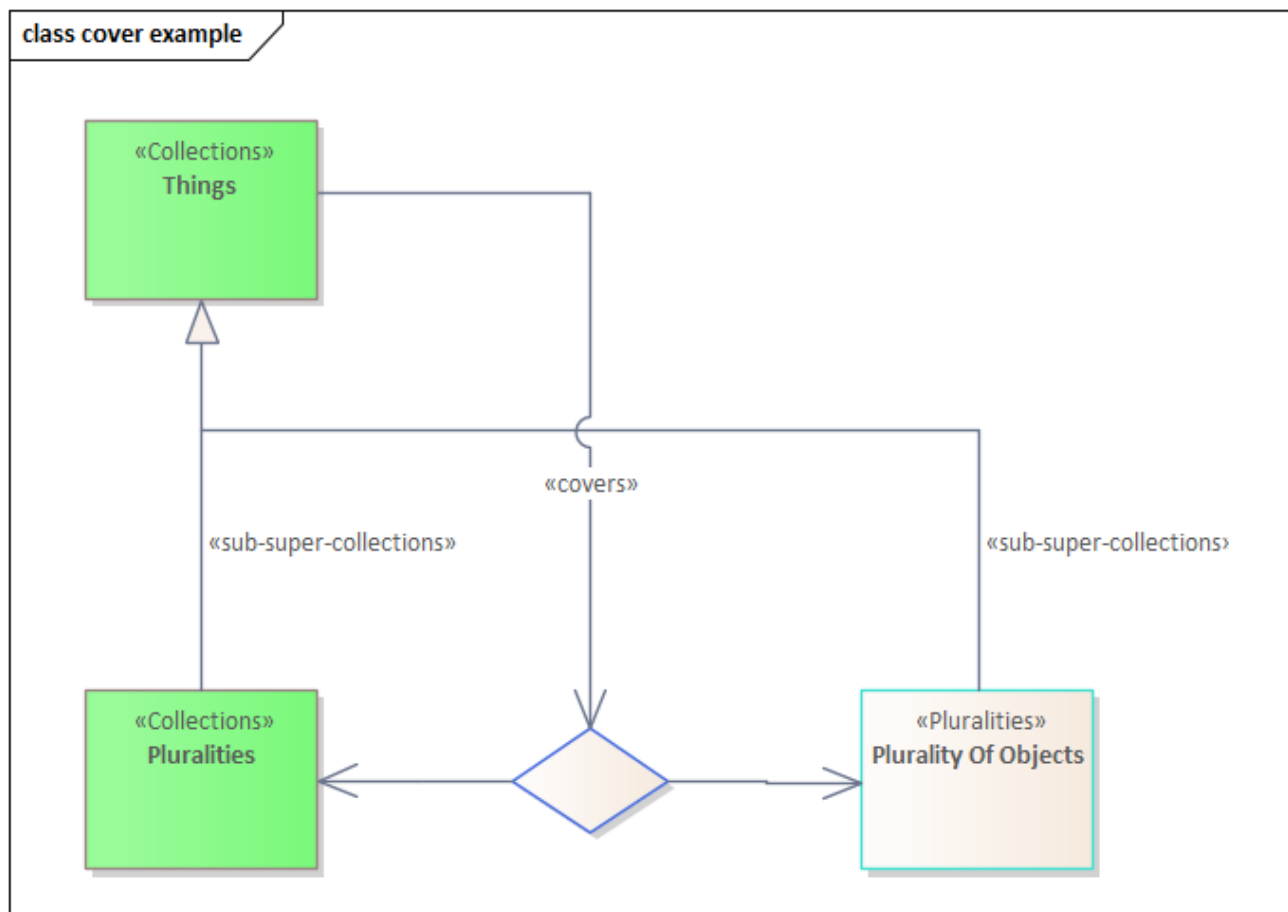
1.6.1 Future Work - increasing expressiveness

1.6.1.1 Historic Work

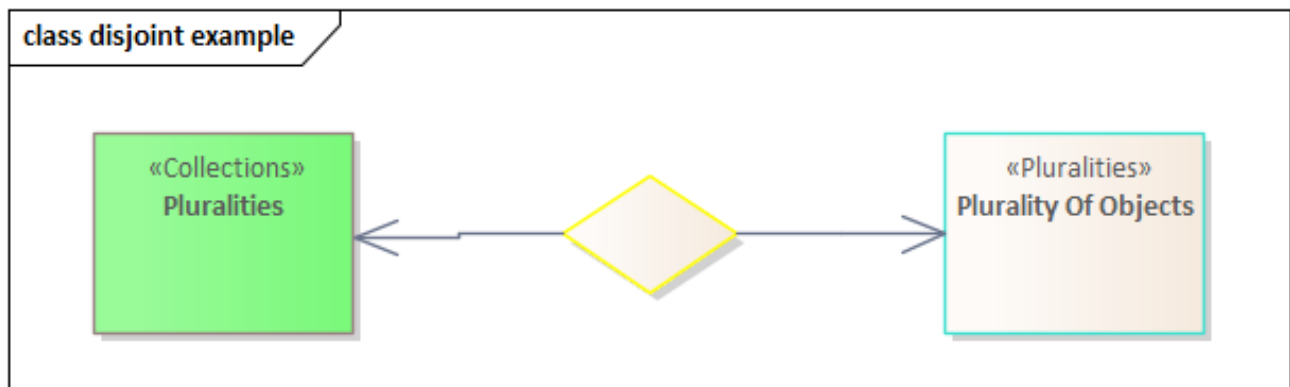
The BORO four-dimensional approach has used object patterns from the start to increase expressiveness. These are described in some detail in (Partridge 1996 Business Objects) (see page 159) - see for example, see here: (pp. 220, 227) object patterns - individual and class levels (see page 164). This includes patterns for partition and disjoint at both the individual and class level. More background details can be found here: (Partridge 2016 Formalization of the classification pattern) (see page 172). In the NDT CLIF work, the partition pattern was captured using a combination of cover and disjoint as this appeared to be more computationally tractable. These patterns were not yet integrated into IES4.

1.6.1.2 Current Work

Following the NDT CLIF work, patterns for cover and disjoint (at the individual and set level) are introduced into the the ToLO. As already noted, the combination of cover and disjoint is equivalent to partition. Examples of cover and disjoint from the ToLO model legend are given below.



1 cover



2 disjoint

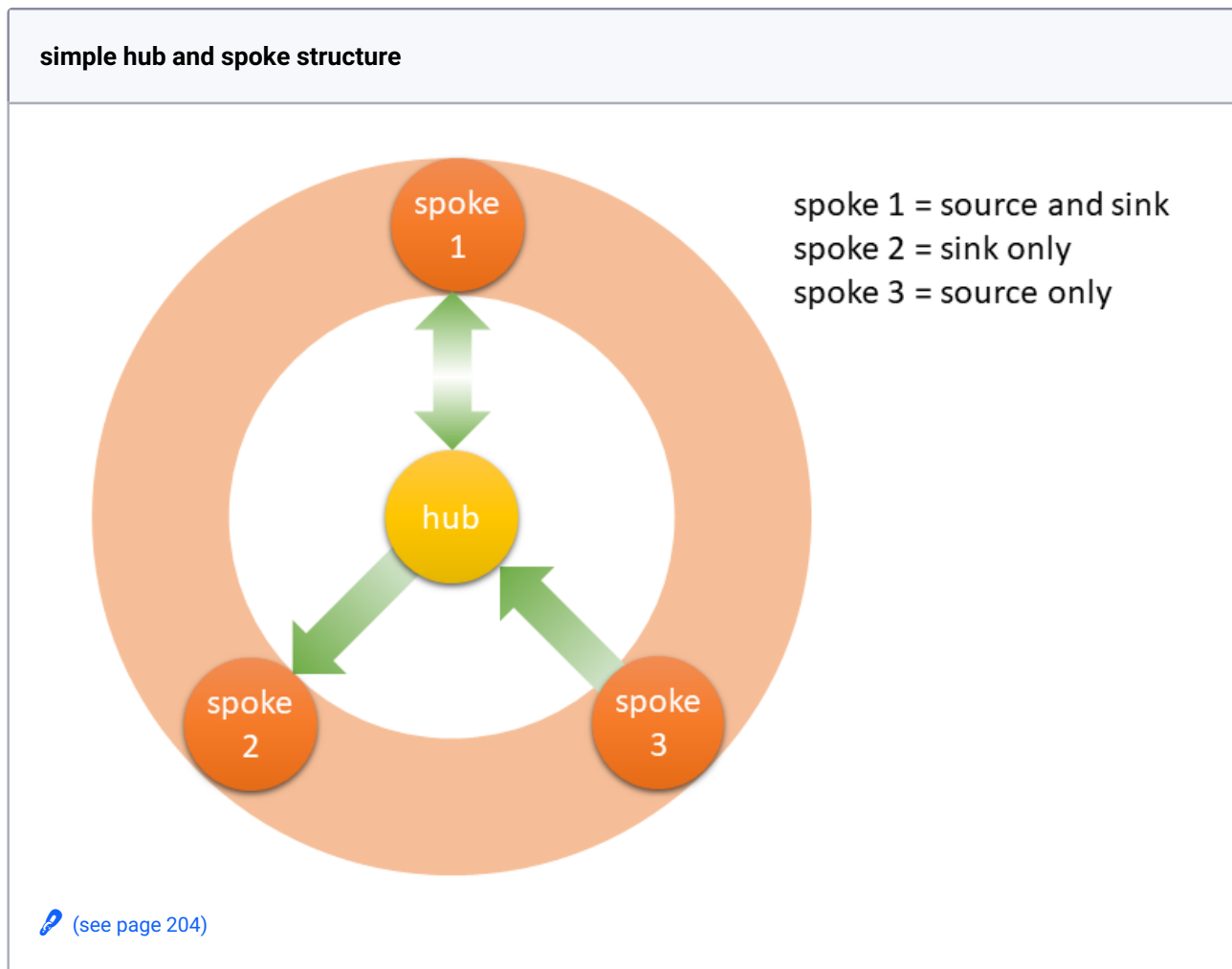
1.6.1.3 Future work

There are other patterns in the BORO four-dimensional approach that could usefully be added to ToLO to increase expressiveness. One example is the [patterns - derived](#) (see [page 168](#)).

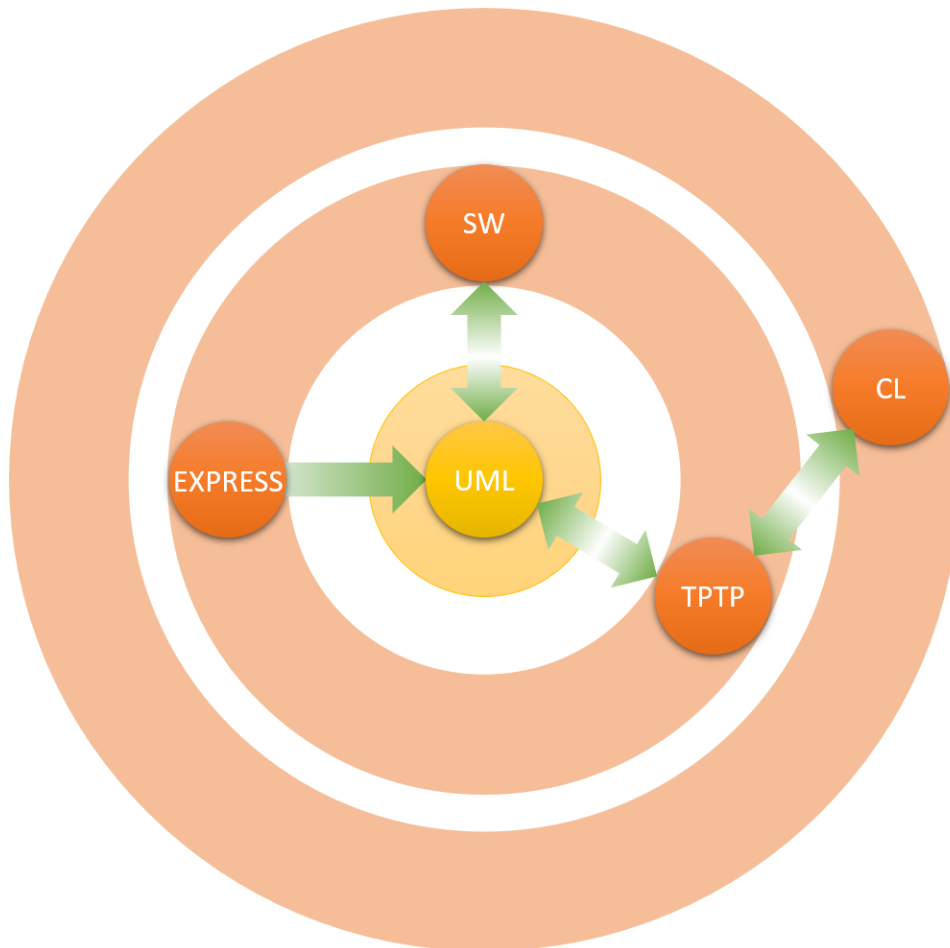
1.6.2 Future Work - implementing an Ontology Multi-Language Support Framework

The ontology has to be represented in a language. It is tempting to keep things simple by adopting a single preferred language. But this effectively locks one into the language, with all the attendant risks - such as when the language goes out-of-date. A more nimble strategy that embraces multiple languages can mitigate these risks. It also allows one to choose a language most suited to a specific functionality - as typically there is no one language that excels in all the capabilities one needs. For example, a language with good visualisation tools (for example, UML) may not have good automated theorem proving tools, whereas other languages do (for example, TPTP). So, it is useful to be able to deploy a top level ontology (and data using the ontology) in multiple languages. To use, for example, TPTP for checking formal consistency and UML for conceptual modelling.

As part of the NDT work, we started to develop a framework for ontology multi-language support, which is described in [Appendix - Ontology Multi-Language Support Framework \(see page 53\)](#). This framework was based upon a hub-and-spoke architecture (see a simple version visualised below)



The framework looked at various options - often with a two-level hub-and-spoke structure - as shown below.

Option - UML Hub (TPTP to CL)

 (see page 207)

There was a pilot implementation of parts of the framework.

1.6.2.1 Future Work

The pilot implementation needs to be resuscitated and further developed into a tool focusing on current requirements.

The current ToLO is implemented in a form of UML and likely to be reimplemented in RDF. In this case, if one wanted to develop the capability to formally check their formal consistency this would require a two-step process. The consistency checking capability is described here: [Future Work - ensuring the consistency of the ToLO \(see page 36\)](#). However, this requires CLIF as input. So one would also need to develop the capability to translate models to (and from) CLIF to interface with the consistency checking capability - in other words, the capability described in the ontology multi-language support framework.

1.6.3 Future Work - ensuring the consistency of the ToLO

The current ToLO has not been checked for consistency, as there are no tools for doing this. As part of the NDT work, we investigated ways to do this and arrived at the conclusion that A sensible approach would be to convert the UML (or RDF) models into CLIF and check the consistency of the CLIF formalisation TPTP-based theorem provers. This is described in [Appendix - CLIF Formalisation Ecosystem](#) (see page 52).

1.6.4 Future Work - formalising testimony, including belief management

The notion of an assertion is embedded in IES 4.0 and ubiquitous in many usages. One very interesting aspect of the initial counterpart analysis is how it has revealed the doxastic (belief-based) nature of these assertions - and how they need to be managed to allow for revision and nesting. To ensure a good model, it is essential to develop a better understanding of how to model this doxastic dimension. In particular, to understand the pragmatic issues that arise and how they can be formally resolved at scale.

We would plan to develop this understanding in the normal two ways (see here: [Example-based empirical development two pronged approach](#) (see page 11)):

1. empirical - through the use of specific examples
2. background literature review.

Obviously there is an interplay between these, where the review suggests examples and the examples suggest topics for investigation.

1.6.4.1 Empirical - examples

The examples test dataset will need to be extended to include a suitable range of assertions. An obvious initial example is an assertion of an event, believed by the asserter, but not (yet?) believed by the central coordinating system.

1.6.4.2 Background literature review

To give some idea of the background review, one can start with 'assertion'. Assertion as noted here, ([Pagin 2021 Assertion](#)) (see page 158), has been investigated in several disciplines, and generated an associated literature. Within intelligence, assertions are typically treated as testimony - evidence for a belief. Testimony, like assertion, has been investigated in a range of disciplines - and the investigations identify a range of issues. Philosophy's view on its epistemological issues are noted here ([Leonard 2023 Epistemological Problems of Testimony](#)) (see page 154), along with a rich bibliography. Some of these issues - such as epistemic authority - have practical implications in intelligence.

This then leads onto questions of belief revision (see, for example, ([Hansson 2022 Logic of Belief Revision](#)) (see page 151)) and aggregation (see, for example, ([Pigozzi 2021 Belief Merging and Judgment Aggregation](#)) (see page 179)) that result from being exposed to testimony.

1.6.5 Future Work - exploiting opportunities for formalising and automating a controlled doxastology

The most exciting aspect of Lewis's counterpart theory is the light it shines on the formal underpinnings for intelligence systems. It reveals the doxastic machinery that is at work in the various intelligence processes. With this understanding, one can see the intelligence system (the interaction processes) as being the institution's controlled doxastology. Where the control processes help to integrate the overall system.

The understanding of the doxastic nature of the processes, and the way they can be modelled, open up opportunities for formalising and automating the work. It also opens up the opportunity to devise better architectures, which should help the system to scale more efficiently and effectively.

1.6.6 Future Work - formalising relative possibilities

There is an obvious algebra that could be applied to the doxastic possibilities, which would give a formal relative ordering. For example, Anne being in Edinburgh at 10am, is more likely (possible) than Ann meeting Bindi in Edinburg at 10am. This could be developed.

In addition, it should be possible to assign relative likelihoods (in numerical terms) to the possibilities, and check this has been done consistently. There is no reason that for there not to be multiple views on the relative likelihoods. This could be developed.

In addition, we could try to apply a metric to independent possibilities. We would need to determine what the underlying structure this is trying to capture. The current research suggests it is a measure of the relative 'number' of possible worlds. Furthermore, it would need to account for different agents giving different assessments.

1.7 Appendices

The report contains the following appendices:

- [Appendix - Intelligence Examples \(see page 41\)](#)
- [Appendix - CLIF Formalisation Ecosystem \(see page 52\)](#)
- [Appendix - Ontology Multi-Language Support Framework \(see page 53\)](#)
- [Appendix - Work-in-Progress \(see page 144\)](#)
- [Appendix - Future Work \(see page 145\)](#)

1.7.1 Appendix - Intelligence Examples

We have been developing intelligence examples to help drive the evolution of the counterpart theory model. As discussed in [Example-based empirical development two pronged approach \(see page 11\)](#), these examples are central to the development of the model.

The approach we have used is to:

- Firstly, develop [Use Cases \(see page 42\)](#) that outline requirements that the examples are required to meet.
- Secondly, develop [Examples \(see page 44\)](#) that cover the use cases.

We plan to keep track of the mapping from use cases to examples in a matrix. We expect some of the general uses cases appear in multiple examples.

1.7.1.1 Use Cases

1.7.1.1.1 UC.1 - Actor provides an assertion based on their observation of the real world

e.g. Bindi saw Anne in Edinburgh at time X

1.7.1.1.2 UC.2 - Actor provides an assertion based on their own “possible” inference when studying two “independently sourced” assertions

1.7.1.1.3 UC.3 - Actor can asserts a pattern of life

e.g. person parks their car every Monday at lunch time at a given location

1.7.1.1.4 UC.4 - Actor can express an exception in a previously asserted pattern

e.g. the person didn't park there park at the usual spot at Monday lunch time

1.7.1.1.5 UC.5 - An assertion can have its certainty assessed at any time, by an actor, any number of times

e.g. Actor A things its 60% certain, actor B thinks its 40% certain, actor A thinks its 20% certain a week later

1.7.1.1.6 UC.6 - Two actors have conflicting views of where a person was located at a given time

e.g. Anne was in Edinburgh or Brighton at time X

1.7.1.1.7 UC.7 - Two actors have conflicting views of what class of event a person attended

e.g. Anne was at a Party or a Business Meeting

1.7.1.1.8 UC.8 - Two actors have conflicting views of an person's accent at a given time

e.g. Anne had a Brummie accent or an Australian accent at time X

1.7.1.1.9 UC.9 - Two actors have conflicting views of when an event happened on a given day

e.g. The Party was between 1600 and 1800 or between 2000 and 2200

1.7.1.1.10 UC.10 - Two actors have conflicting views of who committed a crime, was it person X or Y

e.g. Either Anne or Anita committed the murder

1.7.1.1.11 UC.11 - User can see all intelligence we are aware of about a person whether that is certain or uncertain

e.g. I can see all the intelligence we have on Anne

1.7.1.1.12 UC.12 - For a given threshold of tolerance to uncertainty, a user can see all intelligence we are aware of about a person

e.g. I can see all the intelligence we have on Anne which has been collectively assessed as being higher than 60% confidence

1.7.1.1.13 UC.13 - Given chains of inferences, a user can identify how parts of the graph is invalidated if a certain section is deemed false

e.g. Anne was not in Edinburgh at time X, hence all inferences based on this assertion are now invalid/less likely

1.7.1.2 Examples

The following examples have been developed to date.

- [Example 1 - Competing assertions \(see page 45\)](#)
- [Example 2 - Pattern of life \(see page 48\)](#)
- [Example 3 - Chains of inference \(see page 50\)](#)
- [Example 4 - Operation reviews \(see page 51\)](#)

1.7.1.2.1 Example 1 - Competing assertions

1.7.1.2.1.1 Use cases covered

UC.1 - Actor provides an assertion based on their observation of the real world (see page 0)

UC.5 - An assertion can have its certainty assessed at any time, by an actor, any number of times (see page 0)

UC.6 - Two actors have conflicting views of where a person was located at a given time (see page 0)

UC.11 - User can see all intelligence we are aware of about a person whether that is certain or uncertain (see page 0)

1.7.1.2.1.2 Narrative

Stage 1 - Bindi asserts a scenario that either happened or didn't

What the user is trying to do:

1. Bindi is an investigator working on Operation BADGER
2. Operation BADGER is investigating an Organised Crime Group called the RAIDERS
3. One of the members of the RAIDERS group is Anne
4. Bindi reviews intelligence that suggests that Anne was in Edinburgh at 10am on 1st January 2023.
5. Bindi understands that there is uncertainty about Anne's location at 10am on 1st January 2023.
6. Bindi wants to talk about the same Anne that has been talked about in other intelligence reports.
7. Bindi assess her intelligence as MEDIUM

Unconscious / tacit assumptions:

1. There is a collective moral certainty within her operation that Anne is actual
2. Bindi has a moral certainty that Anne is alive at 10am on 1st January 2023.
3. Someone could have already made the same assertion as Bindi

Implicit requirements:

1. Her organisation needs provenance to who made the above assertion (i.e. Bindi)
2. Her organisation needs provenance of the time at which the assertion was made
3. Her organisation needs provenance to the source that informed Bindi of this information

Stage 2 - Derrick asserts something independent to Bindi but which, unknown to each other, contradicts Bindi's assertion

1. Derrick is also an investigator working on Operation Badger
2. Derrick receives information from a different source to Bindi that states that an Anne was in Brighton at 10am on 1st January 2023.
3. Additionally she had blonde hair when she was observed.
4. Derrick understands that there is uncertainty about Anne's location at 10am on 1st January 2023 and the colour of her hair
5. Derrick wants to talk about the same Anne that has been talked about in other intelligence reports.
6. Derrick assesses his intelligence that Anne's location at 10am on 1st January 2023 and the colour of her hair is marked as MEDIUM

Unconscious / tacit assumptions:

1. Again, there is a collective moral certainty within her operation that Anne is actual
2. There is a collective moral certainty within her organisation that Brighton is certain
3. A is more likely A+B and vice versa
4. Other possibilities don't cross their minds

Implicit requirements:

1. His organisation needs provenance to who made the above assertion (i.e. Derrick)
2. His organisation needs provenance of the time at which the assertion was made
3. His organisation needs provenance to the source that informed Derrick of this information
4. His organisation needs to make sure that the Anne talked about by Derrick and the one talked about by Bindi somehow match up along with Anne spoke about in previous reports

Stage 3 - merged view of Bindi's and Derrick's assertions

What the user is trying to do:

1. Anne and Derrick should be able to see that a subsection of their assertion contradicts with the other (her location at 10am on 1st January 2023)
2. All users can see that Anne could have been in Brighton or Edinburgh.
3. All users can see that the only additional difference depending on where she was, was that in Brighton she was asserted to have blonde hair
4. All users can see who made each of the assertions

5. All user can see who made each assessment against the assertions

Unconscious / tacit assumptions:

1. Anne could have been in neither Brighton or Edinburgh
2. A posterior assessment is given for the assertions on if Anne was in Edinburgh or if Anne was in Brighton. These are based on their own respective assessments taken as priors.

Stage 4 - Charlie makes an inference which supports a subsection of Derrick's assertion

What the user is trying to do:

1. Charlie has been studying comms data related to Anne's mobile. This analysis points to Anne being in the South East of England for new years eve.
2. We wants to use this as intelligence that supports Derrick's assertion that Anne was in Brighton the next day
3. However if doesn't provide any support that she had blonde hair at the time.
4. Charlie asserts Anne's inferred location based on her mobile phone cell tower pings and want to link it to support part of Derrick's assertion - that Anne was in Brighton
5. But specifically not that she had blonde hair
6. Charlie wants to talk about the same Anne that has been talked about in other intelligence reports.
7. There is a collective assessment within her organisation that comms data can be treated as providing HIGH confidence

Unconscious / tacit assumptions:

1. Again, there is a collective moral certainty within her operation that Anne is actual
2. Again, there is a collective moral certainty within her organisation that Brighton is certain

Implicit requirements:

1. His organisation needs provenance to who made the above inference (i.e. Charlie inferring that cell pings indicate Anne's location)
2. His organisation needs provenance of the time at which the inference was made
3. His organisation needs provenance to the source that informed Charlie of this information (aka. the telecoms provider)
4. Her organisation needs to make sure that all statements about Anne are linked in their system

1.7.1.2.2 Example 2 - Pattern of life

1.7.1.2.2.1 Use cases covered

[UC.3 - Actor can asserts a pattern of life \(see page 0\)](#)

[UC.4 - Actor can express an exception in a previously asserted pattern \(see page 0\)](#)

1.7.1.2.2.2 Narrative

Stage 1 - Bindi asserts a pattern of life (scenario)

What the user is trying to do:

1. Bindi has noticed that a number of intelligence reports indicate that every Monday from roughly 0700 to 1900, Harry is away from his main base of operations (London), and is instead in Birmingham. If there were to be any covert action taken against his London apartment this would be the opportune period of time to do so.
2. Bindi records this pattern so that the Covert Asset Team (CAT) have information about opportunity available to them. A Covert Asset team is one that deploy technical assets e.g. against Buildings or Vehicles in order to support surveillance operations
3. WEIGHT THE PATTERN (ALMOST ALWAYS) - UNLESS PROVEN OTHERWISE

Unconscious / tacit assumptions:

1. We have moral certainty that this pattern of life is historic based
2. Window of the past (back to when the evidence first suggested the pattern) and a window into the future

Stage 2 - Bindi asserts additional information about an already asserted pattern of life (scenario)

3. Later that week Bindi receives more intelligence about Harry's Monday trips to Birmingham. It turns out he parks his car in the same car park every time her is up there, at New Street Station car park. Again this is valuable information for the CAT. With this information that could find opportunities to install a tracking device on Harry's car.

Stage 3 - Bindi asserts an exception to an already asserted pattern of life

4. Intelligence suggests that Harry won't be travelling to Birmingham in three Mondays time. She needs to update the intelligence on Harry's pattern of life on Mondays to mark out this exception. This is important for the covert action team so that (A) they don't attempt any action on that Monday at

Harry's apartment and/or (B) waste their time getting their team up to Birmingham for any attempted action against Harry's car

5. Intersecting plans for the covert action?? (Scheduling?)

Unconscious / tacit assumptions:

1. Highly likely that the pattern will be broken
2. Likelihood of Harry being in his apartment increases when he is not in Birmingham (relative Likelihoods)

Implicit requirement

1. Need to talk about specific instances of a breakage in a pattern (grouping pattern variations)

1.7.1.2.3 Example 3 - Chains of inference

1.7.1.2.3.1 Use cases covered

UC.2 - Actor provides an assertion based on their own “possible” inference when studying two “independently sourced” assertions (see page 0)

1.7.1.2.3.2 Narrative

Stage 1 - Bindi's receives an assertion and Harry's number has been in frequent contact with a single other

1. Bindi's organisation is investigating an member of an Organised Crime Group called Harry. Bindi lawfully obtains Communications Metadata records on a telephone number 07911112222 (morally certain) to be belonging to him (service provide A would have said in the past it was Harry's). This communication data comes from the service provider A of 07911112222 and asserts that this number has been in contract with another number 07988889999, 50 times over a space of a month.
2. Bindi identifies that 07988889999 is provided by comms service provider B

Assumption

Moral certain is the purpose of investigations, or in specific cases during an investigation

Calibrate, moral certainty between organisations?

Stage 2 - Bindi's receives an assertion about the subscriber of this other number

3. Bindi lawfully obtains the subscriber account details for 07988889999 from provider B. It comes back with the name of the account holder being a George SEARS
4. It is normal for Bindi, members of her organisation and UK Policing assume (collective moral certainty) what both service provider A's or B's asserts is certain. In fact in UK Policing data from a Communication Service Provider is (in most cases) considered evidential.

Stage 3 - Based on these two previously received assertions, Bindi makes an inference

5. It is normal for Bindi and members of her organisation to assume (collective level of confidence, let's say 80%) that the account holder is the primary user of a number.

Stage 4 - Based on this previous inference, Bindi makes another inference

6. Based on this line-of-thinking, Bindi now asserts that George could be an associate of Harry

1.7.1.2.4 Example 4 - Operation reviews

1.7.1.2.4.1 Use cases covered

UC.11 - User can see all intelligence we are aware of about a person whether that is certain or uncertain (see page 0)

UC.12 - For a given threshold of tolerance to uncertainty, a user can see all intelligence we are aware of about a person (see page 0)

1.7.1.2.4.2 Narrative

Stage 1 - Weekly Operation review begins with understanding the latest operation updates

1. Every two weeks the Senior Responsible Officer goes through the latest picture of what the operation team knows about the RAIDERS

Stage 2 - SRO wants to identify where the intelligence gaps are and where intelligence it's at its weakest

2. A Senior Responsible Officer would want to see both information that was below their threshold or above their threshold

Stage 3 - SRO wants to identify where we have confidence in our intelligence and hence what can be based onto evidential teams

3. Above so that if executive action needs to be taken place they being to disseminate this intelligence to evidential teams. Evidential teams would then have to find information from evidential

Stage 4 - SRO wants to identify to identify if the operation team has enough evidence to support executive action (arrests and seizures)

INTEL WORK WOULD BE MERGED FROM THE PREVIOUS EXAMPLES

1.7.2 Appendix - CLIF Formalisation Ecosystem

Even though it is important to be able to check the consistency of models, there are not good tools available for checking the consistency of formal models that use first-order or higher-order logic formalisation. This significantly hampers progress.

(ISO/IEC 21838-1:2021) (see page 152) recommends the use of CLIF - described in (ISO/IEC 24707:2018) (see page 153) - for formalising top level ontologies. While CLIF is a sensible way to formalise the ontologies, there are really no good tools for checking the consistency of CLIF theories the size of top ontologies.

A common interlingua for formal theories is TPTP (<https://www.tptp.org/> - Thousands of Problems for Theorem Provers) and useful provers have been written for this. These will check the consistency of a logical theory and if it is inconsistent give some (often obscure) reason for the this. Hence, a common approach to checking CLIF is to convert it to TPTP and use a TPTP theorem prover.

In this situation, the process of finding and resolving inconsistencies requires enormous manual effort - and does not scale well. A small change in one part of the theory can result in an inconsistency anywhere in the overall theory. A common approach is to slice the theory to find where the inconsistency is - but the number of slices to test grows exponentially. Good management of the slicing strategies can significantly reduce the effort needed to find and resolve inconsistencies. There are no good tools for managing slicing strategies at the moment.

So something richer is required for large CLIF theories, something closer to the role a compiler plays in programming languages. What is needed is a tool that clearly identifies an inconsistency and suggests how this might be resolved.

Some ground-breaking work done under NDT to producing such a tool, building on some earlier academic work. First a test architecture for the foundation was established in Python which was composed of:

- A CLIF parser - to read the theories (it was eventually established that Antlr (<https://www.antlr.org/>) would be a good basis for this.
- A CLIF (FOL) storage server
- A CLIF to TPTP exporter.

Then a checking strategy tool was deployed over the foundation.

The broad feasibility of this approach was established, but significant work remains to be done to produce a useable tool. (It should be noted that the academic community has little interest in this topic, as it is not regarded, perhaps rightly, as an academic issue.)

1.7.3 Appendix - Ontology Multi-Language Support Framework

1.7.3.1 Introduction

The earlier section in the main report, [Future Work - implementing an Ontology Multi-Language Support Framework \(see page 34\)](#), made the broad case for a multi-language strategy. This appendix briefly describes the work done to date on the framework for ontology multi-language support. There has been a pilot implementation, but this is not covered here. The material should be regarded as a work-in-progress, as it has not yet been completed. It is a report on the current state of knowledge.

The appendix has three sections, each dealing with a major concern:

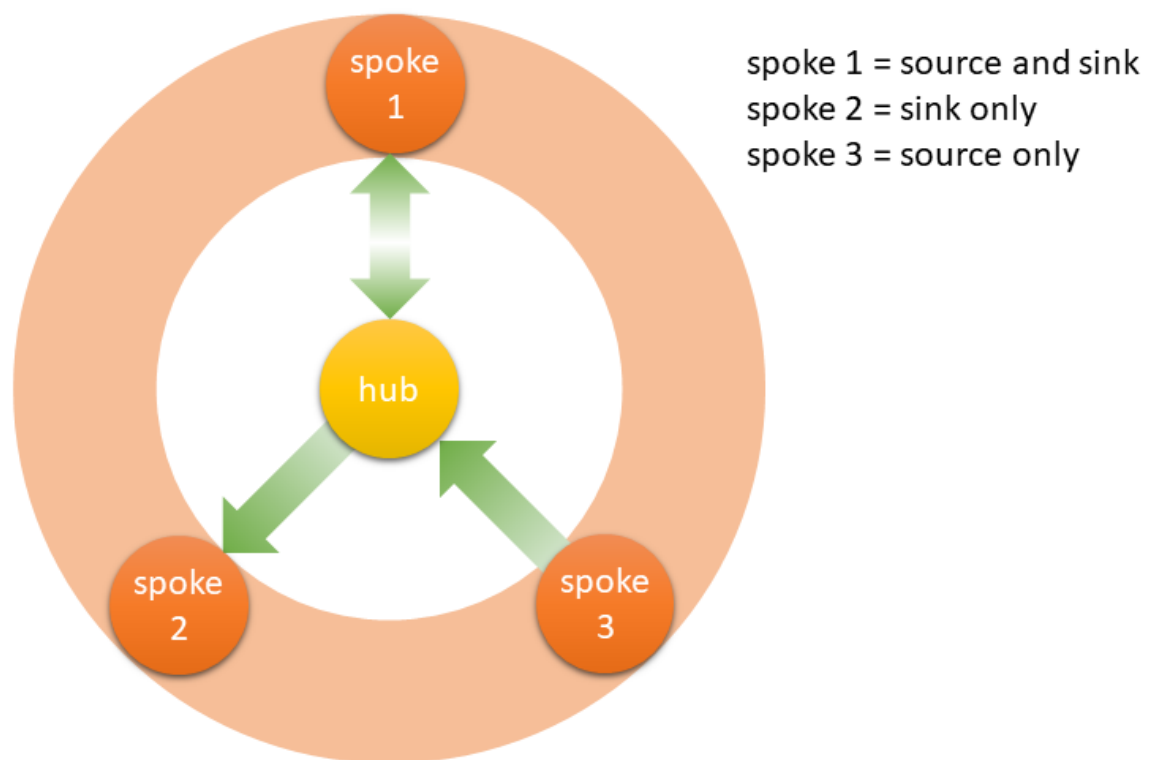
- the proposed architecture - [OMSF - Architecture \(Proposed\) \(see page 54\)](#) - as noted above we proposed a hub-and-spoke architecture.
- the candidate languages - [OMSF - Language/Storage Format \(see page 66\)](#) - this looks at relevant features such as expressivity and visualisation, and
- tools - [OMSF - Tools \(see page 101\)](#)

1.7.3.2 OMSF - Architecture (Proposed)

1.7.3.2.1 Proposed Conceptual Architecture

Propose a hub and spoke architecture - see below.

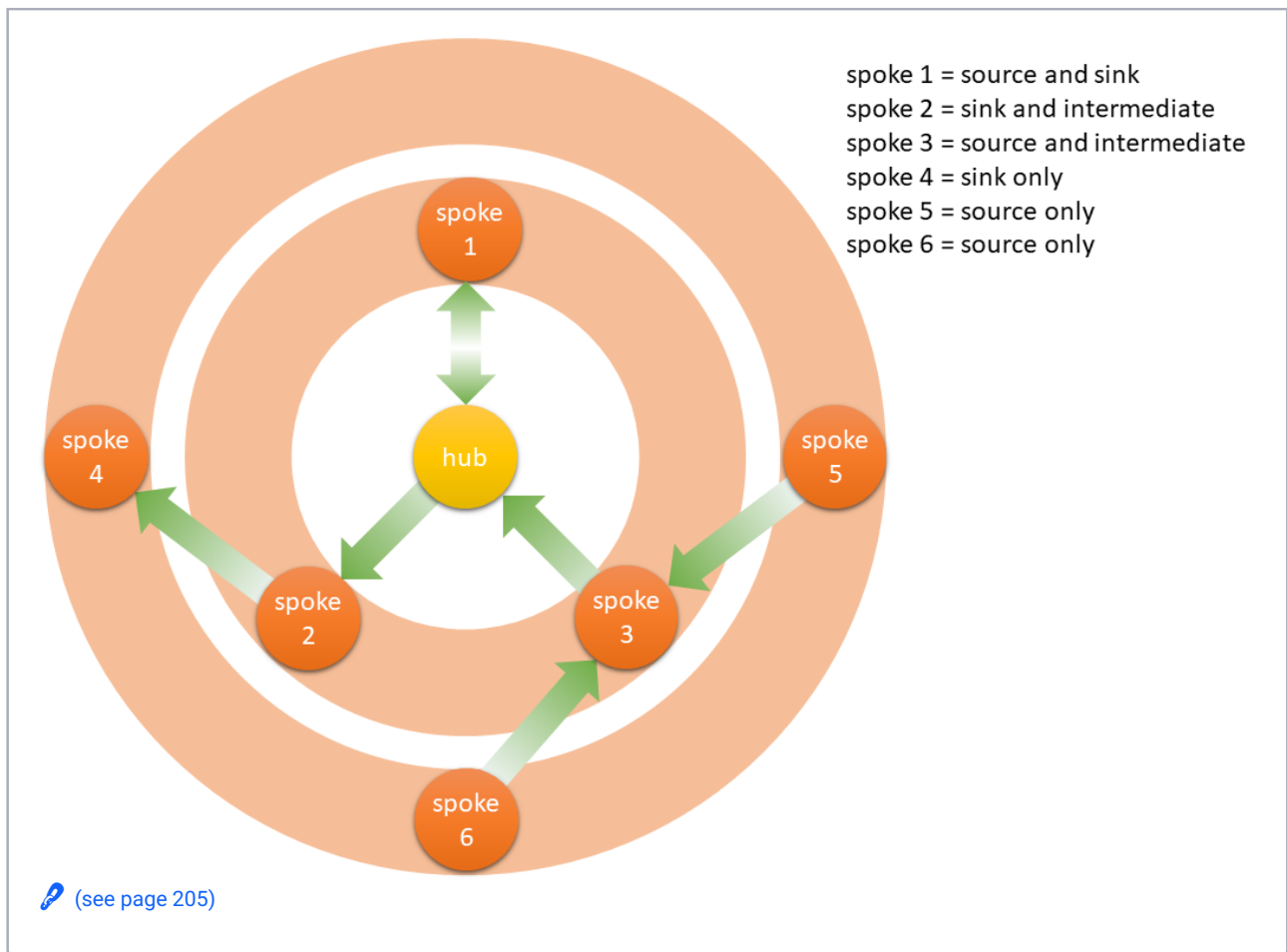
simple hub and spoke structure



[\(see page 204\)](#)

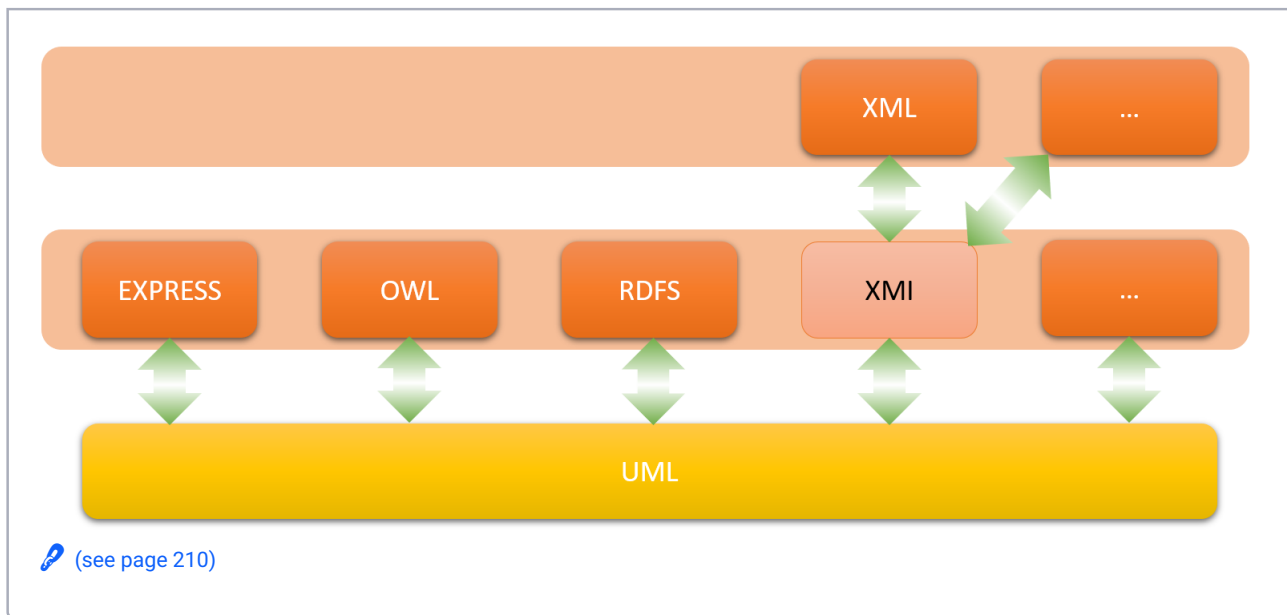
Translation has a cost. Where good tools exist, it may be worth having two layers of spokes - see below.

two-level hub and spoke structure



Currently, UML Looks an attractive hub. If it was selected, then the conceptual architecture might look something like that below.

UML as hub



[-] The issues that will drive the conceptual architecture (including the choice of hub and spokes) and the implementation are:

- the ability to translate losslessly from (and to) a language (i.e. its expressivity).
- the tools available for translating - and the quality of their translation and their cost
- the ease with which open source tools can be developed.

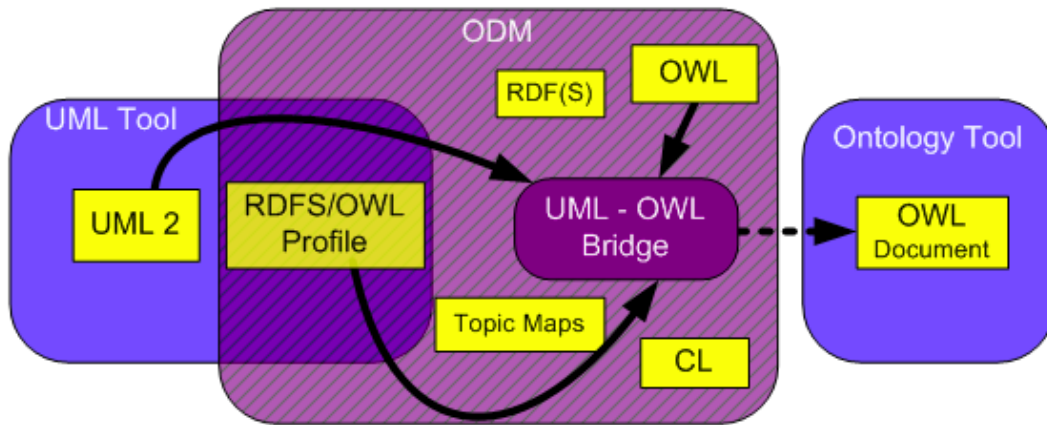
[-] example of hub and spoke - OWL is hub

<http://www.eclipse.org/atl/usecases/ODMImplementation/>

ATL Use Case - ODM Implementation (Bridging UML and OWL)

This work presents an implementation of the OMG ODM (Ontology Definition Metamodel) proposal using the ATL language.

ODM Overview



ODM Overview

[\(see page 211\)](#)

1.7.3.2.2 Architecture - Canonical Language(s)

1.7.3.2.2.1 Introduction

The hub-and-spoke architecture allows for an internal ecosystem of languages, with the more canonical at the hub, the less canonical in the spokes.

There are multiple industry standards for languages that could be included in the hub-and-spoke ecosystem. We recommend considering one of these, rather than developing one. In the hub, it may be a single standard language or a small group of standard languages. For example, one could choose to go with CL and UML (maybe with the ODM extension). If selecting a group, one needs to establish that lossless translation is feasible.

We have developed a candidate shortlist of industry standards to give some idea of what is available - see below.

We provide a matrix with possible translations between the candidate languages - together with the rationale.

1.7.3.2.2.2 Industry Standards Canonical Language(s) Shortlist

These have been selected on the basis of whether a language is an industry standard, its logical expressivity, its usage by TLOs and industry and extra features it provides.

See [Industry Standards Language Candidates](#) (see page 68) for details on each candidate.

candidate	standard	logical expressivity	TLO usage	Industry usage	features	comments
UML	YES (ISO and OMG)	OCL-level fragment of FOL	UFO, BORO, IES4	Widespread	visualisation	Note: ODM (UML Profile) extends UML's OCL to CL (CLIF) level. UML (see page 86)

candidate	standard	logical expressivity	TLO usage	Industry usage	features	comments
SW Family - OWL/RDFS/RDF	YES (W3C)	various (most often DL fragment of FOL)	BFO, gUFO, versions of DOLCE and GFO, OpenCyc	Widespread	visualisation, automatic reasoning, linked data	This is a family of languages with different syntax and semantics. Semantic Web Family (see page 90)
CL (CLIF)	YES (ISO)	between FOL and HOL	DOLCE, parts of BFO	Academic, not used much in industry	de facto standard for FOL	This is a family of languages with common semantics. CL (CLIF) (see page 69)
TPTP	de facto standard	FOL and HOL	-	Academic, not used much in industry	de facto standard for FOL and HOL, automatic reasoning	TPTP (see page 93)

candidate	standard	logical expressivity	TLO usage	Industry usage	features	comments
EXPRESS-STEP-File	YES (ISO)	FOL?	ISO15926, HQDM	Widespread		A marriage of two very different languages, one for the schema and another for the data. A common format for legacy system data. Probably not a viable format in the future. EXPRESS-STEP-File (see page 71)

1.7.3.2.2.3 Translation Topology - Architecture

We assume that a hub and spoke architecture will be adopted to minimise translation costs - where the hub is one or a group of abstract syntaxes.

We set out the possible architectures for this below based upon four scenarios:

1. that a single syntax:
 - a. UML - is chosen
 - with the flow going from TPTP to CL, where CL is required
 - with the flow going from CL to TPTP
 - b. CL - is chosen
2. that a group of syntaxes - UML and CL - is chosen

The kind of translation topology this would produce is shown in the table below - and visualised in the subsequent diagrams

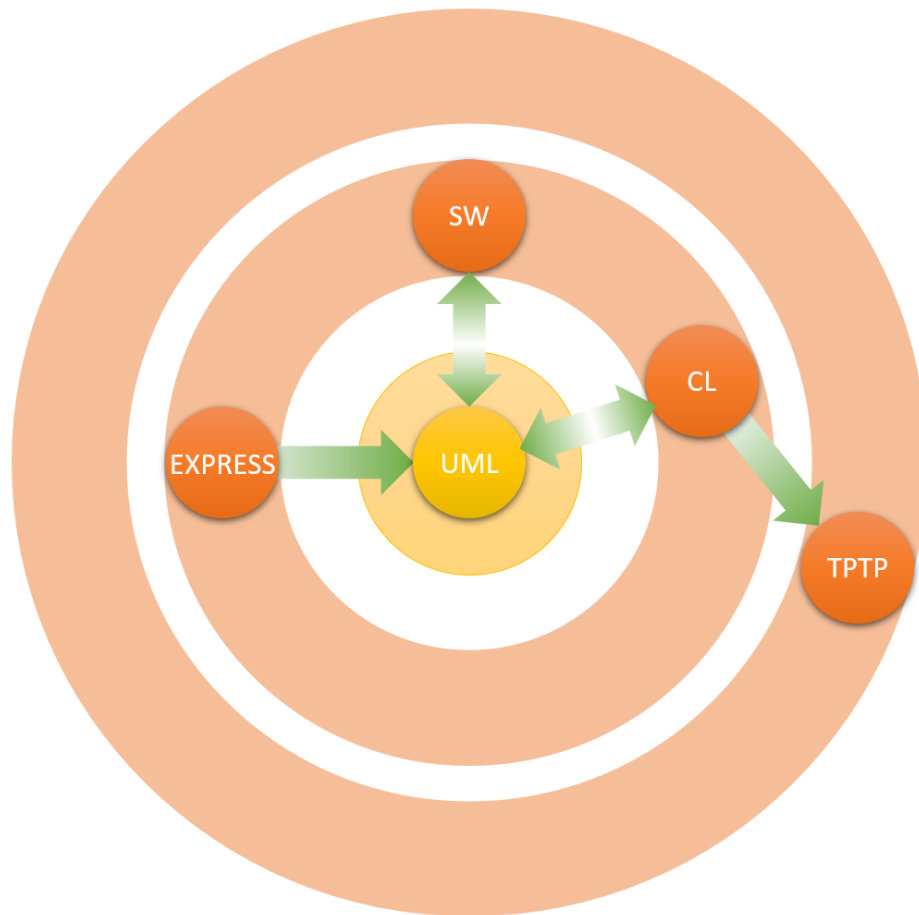
Tabulating the Topology

from\to	UML	CL	SW Family	EXPRESS-STEP-File	TPTP
UML		YES (or via TPTP Intermediate Spoke)	YES (or via CL Hub)	NO	YES (via CL)
CL	YES		YES (or via UML Hub)	NO	YES
SW Family	YES (or via CL Hub)	YES (or via UML Hub)		NO	NO
EXPRESS-STEP-File	YES (or via CL Hub)	YES (or via UML Hub)	YES (via UML and/or CL)		YES (via UML then CL, or via CL)
TPTP	YES (if TPTP is Intermediate Spoke)	YES (if TPTP is Intermediate Spoke)	NO	NO	

Visualising the Topology

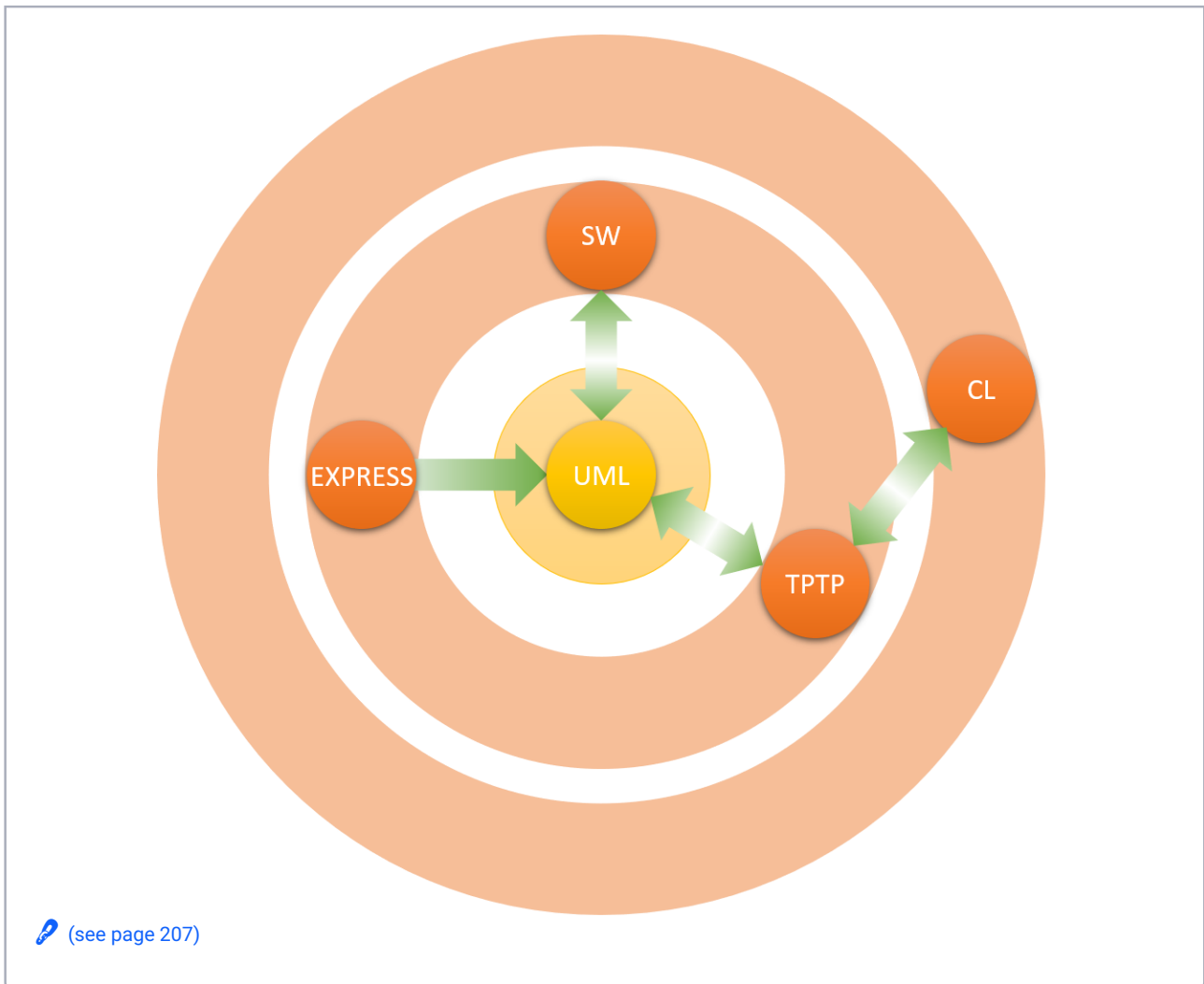
single syntax - UML

Option - UML Hub (CL to TPTP)



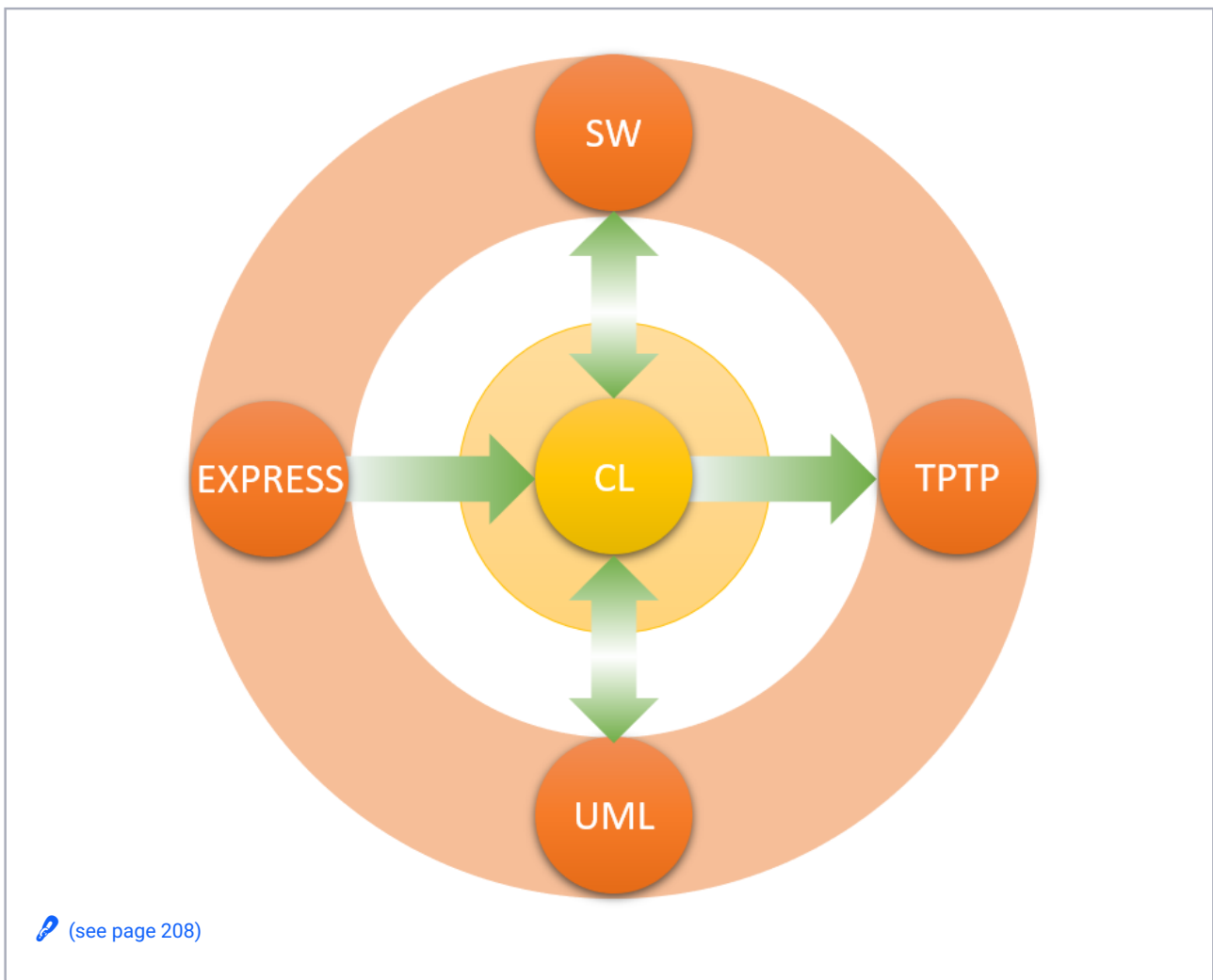
 (see page 206)

Option - UML Hub (TPTP to CL)



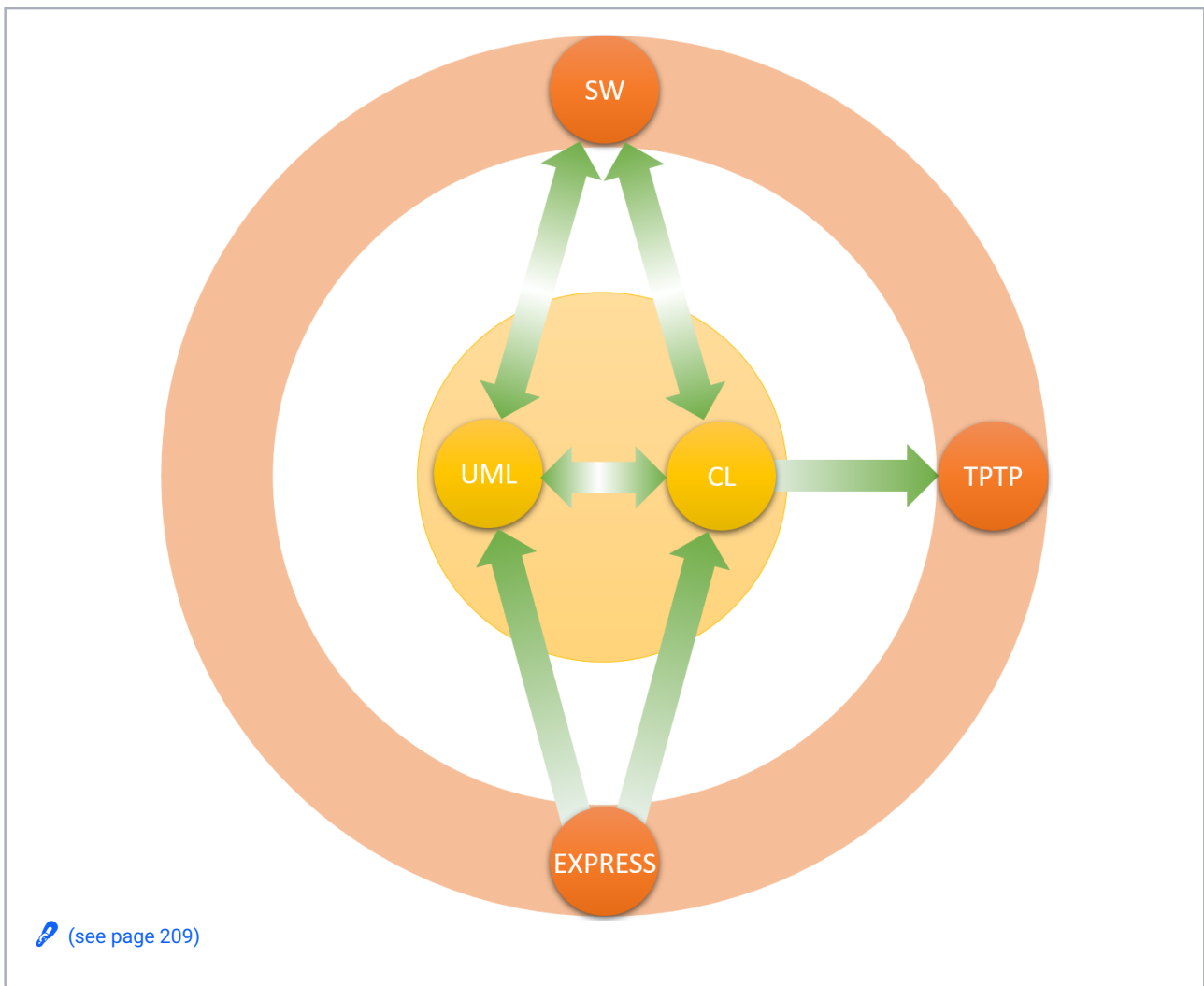
single syntax - CL

Option - CL Hub



group of syntaxes - UML and CL

Option - UML and CL Hub



Background Assumptions

We assume that CL and/or UML are prime candidates for the hub - not the SW Family.

The CL and TPTP are so closely semantically related (provided one sticks to first-order logic) that it makes sense to migrate between them directly.

EXPRESS-STEP-File is a legacy format - and so it will only be consumed.

There is a requirement to prove the consistency of FDM, and that this will be done by translation into the TPTP language, which is supported by various reasoners. Unless OWL is chosen as it has its own reasoners.

There is a requirement to publish the FDM and RDL based ontologies on the Web following the W3C recommendations - so the SW Family - OWL/RDFS/RDF.

There is a requirement to be able to visualise FDM and RDL data.

There is a requirement to be able to easily model FDM and RDL data.

There is a requirement to be able to export the FDM in CL - for standards purposes.

1.7.3.3 OMSF - Language/Storage Format

We undertook a brief survey and analysis of the languages and storage formats that we would consider as candidates for use in the proposed architecture. We have divided these into two categories: [Industry Standards Language Candidates](#) (see page 68) and [Others](#) (see page 94). These are summarised in the table below.

Languages List	
Candidate Industry Standards Languages	
Language	Comments
CL (CLIF) (see page 69)	Not at all used commercially. “Common Logic (CL) is a framework for a family of logic languages, based on first-order logic, intended to facilitate the exchange and transmission of knowledge in computer-based systems.”
EXPRESS-STEP-File (see page 71)	Showing its age? EXPRESS is a standard data modeling language for product data. EXPRESS is formalized in the ISO Standard for the Exchange of Product model STEP (ISO 10303), and standardized as ISO 10303-11. STEP-File (ISO 10303-21) defines the encoding mechanism for representing data conforming to a particular schema in the EXPRESS data modeling language. EXPRESS-STEP-File - STEP-File (see page 82)
OWL (see page 92)	Not used much commercially. “The Web Ontology Language (OWL) is a family of knowledge representation languages for authoring ontologies.”
RDF (RDFS) (see page 91)	Used commercially. “The Resource Description Framework (RDF) is a family of World Wide Web Consortium (W3C) specifications[1] originally designed as a metadata data model.”

Languages	Comments
UML (see page 86)	Used commercially. Unified Modeling Language “The Unified Modeling Language (UML) is a general-purpose, developmental, modeling language in the field of software engineering that is intended to provide a standard way to visualize the design of a system.” ODM (UML Profile) extends UML's OCL to CL (CLIF) level ODM (UML Profile) (see page 88)

Other Candidate Languages

Languages	Comments
JSON (see page 97)	“JavaScript Object Notation is an open standard file format, and data interchange format, that uses human-readable text to store and transmit data objects consisting of attribute–value pairs and array data types (or any other serializable value).”
RDBMS (see page 98)	“a relational database management system is a database management system (DBMS) based on the relational model.”
SQL (see page 99)	Used commercially. Structured Query Language.
XML (XSD) (see page 95)	Used commercially. eXtensible Markup Language. “Extensible Markup Language (XML) is a markup language that defines a set of rules for encoding documents in a format that is both human-readable and machine-readable.”

[P](#) (see page 212)

1.7.3.3.1 Industry Standards Language Candidates

Five candidate languages were considered:

- [CL \(CLIF\)](#) (see page 69)
- [EXPRESS-STEP-File](#) (see page 71)
- [UML](#) (see page 86)
- [Semantic Web Family](#) (see page 90)
- [TPTP](#) (see page 93)

1.7.3.3.1.1 CL (CLIF)

**Common Logic**

Common Logic (CL) is a framework for a family of [logic languages](#)¹, based on [first-order logic](#)², intended to facilitate the exchange and transmission of [knowledge](#)³ in [computer](#)⁴-based systems.

https://en.wikipedia.org/wiki/Common_Logic

 (see page 69)

CL is a family of languages with common semantics.

“Common Logic has [...] a syntax which is signature-free and permits 'higher-order' constructions such as quantification over classes or relations while preserving a first-order model theory, and a semantics which allows theories to describe intensional entities such as classes or properties.”

“Sequence markers take Common Logic beyond first-order expressivity.”

“This ability to represent infinite sets of sentences in a finite form means that Common Logic with sequence markers is not compact, and therefore not first-order;”

1 https://en.wikipedia.org/wiki/Logic_language

2 https://en.wikipedia.org/wiki/First-order_logic

3 <https://en.wikipedia.org/wiki/Knowledge>

4 <https://en.wikipedia.org/wiki/Computer>

CL (CLIF) - Tools

Source	Summary	Remarks
Mossakowski, T., Codescu, M., Kutz, O., Lange, C., & Grüninger, M. (2014). Proof Support for Common Logic. <i>ARQNL@IJCAR</i> . https://pdfs.semanticscholar.org/faad/5beafd4fe4f5ac8048e78062b3dcc4c3b9a9.pdf	1. No dedicated tools for CLIF 2. Use translation to TPTP⁵ 3. Use Hets⁶	Published in 2014
John F. Sowa, Diagrammatic Reasoning With EGs and EGIF http://jfsowa.com/pubs/diagram.pdf	1. Recommends Hets⁷	Published in 2016

⁵ <http://tptp.cs.miami.edu/>

⁶ <http://hets.eu/>

⁷ <http://hets.eu/>

1.7.3.3.1.2 EXPRESS-STEP-File



EXPRESS is a standard [data modeling](https://en.wikipedia.org/wiki/Data_modeling)⁸ [language](https://en.wikipedia.org/wiki/Modeling_language)⁹ for [product](https://en.wikipedia.org/wiki/Product_(business))¹⁰ data. EXPRESS is formalized in the ISO Standard for the Exchange of Product model [STEP \(ISO 10303\)](https://en.wikipedia.org/wiki/STEP_(ISO_10303))¹¹, and standardized as ISO 10303-11.

[https://en.wikipedia.org/wiki/EXPRESS_\(data_modeling_language\)](https://en.wikipedia.org/wiki/EXPRESS_(data_modeling_language))



(see page 71)

Sub-Pages

- [EXPRESS-STEP-File - Useful Links](#) (see page 72)
- [EXPRESS-STEP-File - IFC_Parser](#) (see page 76)
- [EXPRESS-STEP-File - STEP-File](#) (see page 82)
- [EXPRESS Reserved Words](#) (see page 84)

⁸ https://en.wikipedia.org/wiki/Data_modeling

⁹ https://en.wikipedia.org/wiki/Modeling_language

¹⁰ [https://en.wikipedia.org/wiki/Product_\(business\)](https://en.wikipedia.org/wiki/Product_(business))

¹¹ https://en.wikipedia.org/wiki/ISO_10303

EXPRESS-STEP-File - Useful Links

Code Projects

External Links	Name	Language	Comment 2	Internal Links
https://pypi.org/project/steputils/	STEPutils	Python	python parser for ISO 10303-21.	
https://github.com/donghoon/ifctoolkit	ifctoolkit	C#	Toolkit for IFC (Industry foundation class) buildingSmart data file, schema file, ISO 10303 STEP part 11 EXPRESS , ISO 10303 STEP part 21 Clear Text Encoding, bimSteel	
https://github.com/gsimon75/IFC_parser	IFC_parser	Python	Yet another parser for ISO 10303-11 / EXPRESS schemas and IFC files.	EXPRESS-STEP-File - IFC_Parser  IFC Parser Yet another parser for ISO 10303-11 / EXPRESS schemas and IFC files. https://github.com/gsimon75/IFC_parser Author: gsimon75 - Repositories¹²  (see page 76)

¹² <https://github.com/gsimon75?tab=repositories>

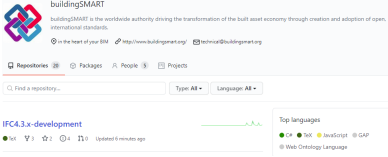
Background

External Links	Name	Type	Descriptions	Comment 2
http://deslab.mit.edu/DesignLab/dicpm/step.html		Blog	Overview of STEP and EXPRESS	At the NSF Invitational Workshop on Distributed Information, Computation and Process Management for Scientific and Engineering Environments (DICPM) I gave a presentation entitled STEP Models and Technology. At the end of the presentation I was asked to provide more pointers to STEP/EXPRESS related information sources. This note is in response to that request. I also append some personal suggestions resulting from the problems exposed and discussed during the workshop. peter.r.wilson@boeing.com ¹³ May 1998
STEP/EXPRESS and XML ¹⁴		Blog	Summary of work aligning STEP/EXPRESS specifications with XML and SGML	
GitHub - dustintownsend/EXPRESS-Modeling-Language-References ¹⁵		BNF Files	BNF specification for EXPRESS.	
http://osexpress.sourceforge.net/exparser.html		Parser Documentation	How to use ANTLR parser for EXPRESS.	

¹³ <mailto:peter.r.wilson@boeing.com>

¹⁴ <http://xml.coverpages.org/stepExpressXML.html>

¹⁵ <https://github.com/dustintownsend/EXPRESS-Modeling-Language-References>

External Links	Name	Type	Descriptions	Comment 2
https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=05bd9c782f929689e8f6943ce0f9246f2d3a3822	STEP, XML, and UML: Complementary Technologies	PDF		This paper discusses efforts underway to make STEP information models available via mechanisms familiar to more business application developers, specifically XML and the Unified Modeling Language (UMLÆ).
https://pdfs.semanticscholar.org/0c32/aea4d2ba2c26b693536634967d11923da623.pdf	Data Modelling Using EXPRESS-G for IFC Development	PDF		
https://steptools.github.io/	STEP Tools Software	Web Ring		
https://thinkmoult.com/using-ifcopenshell-parse-ifc-files-python.html	Using IfcOpenShell to parse IFC files with Python	Blog		
https://github.com/buildingSMART	buildingSMART	github Organisation	Contains 20 repositories 5 Members Top Languages: • C# • TeX • JavaScript • GAP • Web Ontology Language	

External Links	Name	Type	Descriptions	Comment 2
Specs ¹⁶	EXPRESS data modeling language, ISO 10303-11	Blog	Useful links at the bottom	
https://stackoverflow.com/questions/51216753/custom-parser-for-express-data-modeling-language	Custom Parser for EXPRESS data modeling language	Forum Query		

¹⁶ <https://www.loc.gov/preservation/digital/formats/fdd/fdd000449.shtml#specs>

EXPRESS-STEP-File - IFC_Parser**IFC Parser**

Yet another parser for ISO 10303-11 / EXPRESS schemas and IFC files.

https://github.com/gsimon75/IFC_parser

Author: [gsimon75 - Repositories](#)¹⁷

 (see page 76)

Summary

This project can be used to read in an EXPRESS file and it generates a tree object model of the schema. There are some changes needed to up date it from Python 2 to Python 3.

Comments on getting the code to run

Version - IFC_parser is written in Python 2. There are some changes that need to be made to get this to run in Python 3. So far:

- change xrange to range
- the imports had to be scoped with their namespace. e.g. a lot of them needed to have 'Ifc.' added to the beginning of them

Code Bugs**Parsing Types**

IFC parser does handle base types for EXPRESS types when they are described in the complex way, e.g.,

```
TYPE IfcArcIndex = LIST [3:3] OF IfcPositiveInteger;
END_TYPE;
```

Parsing such types results in their basetypes not being set.

¹⁷ <https://github.com/gsimon75?tab=repositories>


```

> ▮ 'IfcAngularVelocityMeasure' = {Type} <nf_express_source.ifc_parser.Ifctype.Type object at 0x000002594EE32388>
  01 basetype = {str} 'REAL'
  01 classname = {str} 'TYPE'
  01 defname = {str} 'IfcAngularVelocityMeasure'
  01 ttype = {str} 'SCALAR'
> ▮ 'IfcArcIndex' = {Type} <nf_express_source.ifc_parser.Ifctype.Type object at 0x000002594EE323C8>
  01 classname = {str} 'TYPE'
  01 defname = {str} 'IfcArcIndex'
  01 ttype = {str} 'LIST'

```

Results

Parsing IFC4X1

```

> ▮ schema.classes['TYPE']['IfcSolidAngleMeasure'] = {Type} <Ifc.Type.Type object at 0x00000294FB7DAC18>
> ▮ Context = {ContextMeta} <class 'jinja2.runtime.Context'>
> ▮ entity_template = {Template} <Template 'entity_parser.template.py'>
> ▮ env = {Environment} <jinja2.environment.Environment object at 0x00000294FAFCBC50>
> ▮ parser = {SchemaParser} <Ifc.SchemaParser.SchemaParser object at 0x00000294FB79AAC8>
> ▮ schema = {Schema} Schema(IFC4X1)
  ▮ classes = {dict: 4} {'TYPE': {'IfcStrippedOptional': <Ifc.Type.Type object at 0x00000294FB79A940>, 'IfcAbsorbedDoseM
    > ▮ 'TYPE' = {dict: 401} {'IfcStrippedOptional': <Ifc.Type.Type object at 0x00000294FB79A940>, 'IfcAbsorbedDoseM
    > ▮ 'ENTITY' = {dict: 801} {'IfcActionRequest': <Ifc.Entity.Entity object at 0x00000294FB7F3908>, 'IfcActor': <Ifc.Ent
    > ▮ 'FUNCTION' = {dict: 47} {'IfcAssociatedSurface': <Ifc.Function.Function object at 0x00000294FBA84B38>, 'IfcB
    > ▮ 'RULE' = {dict: 2} {'IfcRepresentationContextSameWCS': <Ifc.Rule.Rule object at 0x00000294FBA91630>, 'IfcSin
      01 __len__ = {int} 4
      01 classname = {str} 'SCHEMA'
      01 defname = {str} 'IFC4X1'
  > ▮ typedef_template = {Template} <Template 'type_parser.template.py'>
> ▮ Special Variables

```

Parsing IFC2X3

```

> schema.classes['TYPE']['IfcSolidAngleMeasure'] = {Type} <Ifc.Type.Type object at 0x000001B54951E588>
> Context = {ContextMeta} <class 'jinja2.runtime.Context'>
> entity_template = {Template} <Template 'entity_parser.template.py'>
> env = {Environment} <jinja2.environment.Environment object at 0x000001B54914DA58>
> parser = {SchemaParser} <Ifc.SchemaParser.SchemaParser object at 0x000001B5494DD1D0>
> schema = {Schema} Schema(IFC2X3)
  > classes = {dict: 4} {'TYPE': {'IfcAbsorbedDoseMeasure': <Ifc.Type.Type object at 0x000001B5494DD860>, 'IfcA
    > 'TYPE' = {dict: 327} {'IfcAbsorbedDoseMeasure': <Ifc.Type.Type object at 0x000001B5494DD860>, 'IfcA
    > 'ENTITY' = {dict: 653} {'Ifc2DCompositeCurve': <Ifc.Entity.Entity object at 0x000001B54952E4E0>, 'IfcAct
    > 'FUNCTION' = {dict: 38} {'IfcAddToBeginOfList': <Ifc.Function.Function object at 0x000001B549654FD0>
    > 'RULE' = {dict: 2} {'IfcRepresentationContextSameWCS': <Ifc.Rule.Rule object at 0x000001B549662780>,
      01 __len__ = {int} 4
      01 classname = {str} 'SCHEMA'
      01 defname = {str} 'IFC2X3'
  > typedef_template = {Template} <Template 'type_parser.template.py'>
> Special Variables

```

Parsing IFC2X3 collected from IFC - Stage - Collect - Schema¹⁸

```

> Context = {ContextMeta} <class 'jinja2.runtime.Context'>
> entity_template = {Template} <Template 'entity_parser.template.py'>
> env = {Environment} <jinja2.environment.Environment object at 0x000001EC9D6CEB70>
> parser = {SchemaParser} <Ifc.SchemaParser.SchemaParser object at 0x000001EC9D81C6D8>
> schema = {Schema} Schema(IFC2X3)
  > classes = {dict: 4} {'TYPE': {'IfcAbsorbedDoseMeasure': <Ifc.Type.Type object at 0x000001EC9D
    > 'TYPE' = {dict: 327} {'IfcAbsorbedDoseMeasure': <Ifc.Type.Type object at 0x000001EC9D8
    > 'ENTITY' = {dict: 653} {'Ifc2DCompositeCurve': <Ifc.Entity.Entity object at 0x000001EC9D8
    > 'FUNCTION' = {dict: 38} {'IfcAddToBeginOfList': <Ifc.Function.Function object at 0x00000
    > 'RULE' = {dict: 2} {'IfcRepresentationContextSameWCS': <Ifc.Rule.Rule object at 0x000001
      01 __len__ = {int} 4
      01 classname = {str} 'SCHEMA'
      01 defname = {str} 'IFC2X3'
  > typedef_template = {Template} <Template 'type_parser.template.py'>
> Special Variables

```

There is no difference in the counts between the IFC2X3 file in the IFC_parser project and the one downloaded from BuildingSmart.

Parsing lifecycle_integration_schema (ISO 15926) ISO 15926 - Stage - Collect - lifecycle_integration_schema¹⁹

¹⁸ <https://borocvi.atlassian.net/wiki/spaces/BCD/pages/2144503438>

¹⁹ <https://borocvi.atlassian.net/wiki/spaces/BCD/pages/2157871126/ISO+15926++Stage++Collect++lifecycle+integration+schema>

Error: SyntaxError: Cannot interpret defspec 'SUPERTYPE SUBTYPE OF (thing)' in Type abstract_object

In Code (class Entity):

```

while defspec:
    if defspec.startswith("ABSTRACT "):
        self.is_abstract = True
        defspec = defspec[9:].lstrip()
        continue

    if defspec.startswith("SUPERTYPE OF ("):
        (open_pos, close_pos) = find_matching_paren_pair(defspec)
        self.supertype_of = defspec[open_pos + 1:close_pos]
        defspec = defspec[close_pos + 1:].lstrip()
        continue

    if defspec.startswith("SUBTYPE OF ("):
        (open_pos, close_pos) = find_matching_paren_pair(defspec)
        self.subtype_of = defspec[open_pos + 1:close_pos]
        defspec = defspec[close_pos + 1:].lstrip()
        continue

    raise SyntaxError("Cannot interpret defspec '{val}' in Type {n}".format(v
al=defspec, n=self.defname))

```



ABSTRACT SUPERTYPE was not recognised and taken out of .exp

```

ENTITY abstract_object
  ABSTRACT SUPERTYPE
  SUBTYPE OF (thing);
END_ENTITY; -- abstract_object

```

```

ENTITY class_of_abstract_object
  ABSTRACT SUPERTYPE
  SUBTYPE OF (class);
END_ENTITY; -- class_of_abstract_object

```

Error: raise EOFError()

In code:

```

def read_statement(self, permit_eof=True, zap_whitespace=False):

```

```

"""
Read one statement from the input, remove the trailing semicolon
"""
while True:
    semicolon_pos = self.statement_buf.find(";", self.statement_until)
    if semicolon_pos != -1:
        result = self.statement_buf[:semicolon_pos].rstrip()
        self.statement_buf = self.statement_buf[semicolon_pos + 1:].lstrip()
        self.statement_until = 0
        if zap_whitespace:
            return " ".join(result.split())
        else:
            return result

    new_chunk = self.read_chunk()
    if new_chunk == None: # EOF
        if not self.statement_buf:
            # no leftover, return None to signal the end
            if permit_eof:
                return None
            raise EOFError()

```



Comments at the end of entities were not recognised and taken out of .exp

e.g. `-- abstract_object`

```

ENTITY abstract_object
  ABSTRACT SUPERTYPE
  SUBTYPE OF (thing);
END_ENTITY; -- abstract_object

```

```

> Context = {ContextMeta} <class 'jinja2.runtime.Context'>
> entity_template = {Template} <Template 'entity_parser.template.py'>
> env = {Environment} ... Loading Value
> parser = {SchemaParser} ... Loading Value
> schema = {Schema} ... Loading Value
  > classes = {dict: 1} {'ENTITY': {'abstract_object': <Ifc.Entity.Entity object at 0x000002E1
    > 'ENTITY' = {dict: 201} {'abstract_object': <Ifc.Entity.Entity object at 0x000002E1B2
      01 __len__ = {int} 1
      01 classname = {str} 'SCHEMA'
      01 defname = {str} 'lifecycle_integration_schema'
    > typedef_template = {Template} ... Loading Value
  > Special Variables

```

Parsing hqdm_framework <https://borocvi.atlassian.net/wiki/spaces/BCD/pages/2157936658/HQDM++Stage++Collect++hqdm+framework>

SyntaxError: Cannot interpret defs spec 'SUBTYPE OF(thing)' in Type spatio_temporal_extent

Code cannot handle lack of space between OF and (thing)

Find and replaced ' OF(' with ' OF (' in .exp file

```

> Context = {ContextMeta} <class 'jinja2.runtime.Context'>
> entity_template = {Template} <Template 'entity_parser.template.py'>
> env = {Environment} <jinja2.environment.Environment object at 0x000002C6107BFC88>
> parser = {SchemaParser} <Ifc.SchemaParser.SchemaParser object at 0x000002C610F63438>
> schema = {Schema} Schema(HQDM_FRAMEWORK)
  > classes = {dict: 2} {'TYPE': {'participant_in_activity_or_association': <Ifc.Type.Type object at (
    > 'TYPE' = {dict: 1} {'participant_in_activity_or_association': <Ifc.Type.Type object at 0x000
    > 'ENTITY' = {dict: 229} {'thing': <Ifc.Entity.Entity object at 0x000002C610F63BE0>, 'spatio_
      01 __len__ = {int} 2
      01 classname = {str} 'SCHEMA'
      01 defname = {str} 'HQDM_FRAMEWORK'
    > typedef_template = {Template} <Template 'type_parser.template.py'>
  > Special Variables

```

EXPRESS-STEP-File - STEP-File

-  **STEP-File** is a widely used ^{[1]²⁰} data exchange form of **STEP**²¹. ISO 10303 can represent 3D objects in **Computer-aided design**²² (CAD) and related information. Due to its **ASCII**²³ structure, a STEP-file is easy to read, with typically one instance per line. The format of a STEP-File is defined in ISO 10303-21 *Clear Text Encoding of the Exchange Structure*.^{[2]²⁴}

ISO 10303-21 defines the encoding mechanism for representing data conforming to a particular schema in the **EXPRESS**²⁵ data modeling language specified in **ISO 10303-11**²⁶. A STEP-File is also called *p21-File* and *STEP Physical File*. The file extensions *.stp* and *.step* indicate that the file contains data conforming to STEP Application Protocols while the extension *.p21* should be used for all other purposes.^{[3]²⁷}

https://en.wikipedia.org/wiki/ISO_10303-21

STEP FILE VIEWERS

The following table lists available STEP geometry viewing applications. The intent of this list is only informational, to provide potential users with a guide to STEP viewing applications on the market

https://www.cax-if.org/cax/step_viewers.php

[Note: some of these are free.]

stepcode²⁸

STEPcode (formerly NIST's STEP Class Library) is used with IFC, STEP, and other standards that utilize the technologies of ISO10303 (STEP). It generates C++ and Python from EXPRESS (10303-11) schemas. The code is capable of reading and writing STEP Part 21 exchange files. It also utilizes Parts 22 and 23 (SDAI and its C++ binding).

²⁰ https://en.wikipedia.org/wiki/ISO_10303-21#cite_note-1

²¹ https://en.wikipedia.org/wiki/ISO_10303

²² https://en.wikipedia.org/wiki/Computer-aided_design

²³ <https://en.wikipedia.org/wiki/ASCII>

²⁴ https://en.wikipedia.org/wiki/ISO_10303-21#cite_note-2

²⁵ https://en.wikipedia.org/wiki/ISO_10303-11

²⁶ https://en.wikipedia.org/wiki/ISO_10303-11

²⁷ https://en.wikipedia.org/wiki/ISO_10303-21#cite_note-3

²⁸ <https://github.com/stepcode/stepcode>

<https://github.com/stepcode>



Open Cascade Technology (OCCT), formerly called CAS.CADE, is an [open-source software](#)²⁹ development platform for 3D [CAD](#)³⁰, [CAM](#)³¹, [CAE](#)³², etc. that is developed and supported by [Open Cascade SAS](#)³³.

- **Data Exchange:** provides possibility to import/export various CAD formats. [STEP](#)³⁴, [IGES](#)³⁵, [glTF](#)³⁶, [OBJ](#)³⁷, [STL](#)³⁸ and [VRML](#)³⁹ are supported natively.^[18]⁴⁰ Other formats can be imported by using plug-ins.^[19]⁴¹ Extended Data Exchange (**XDE**) components rely on a unified **XCAF document** definition, which includes an assembly structure of CAD shapes, color/name/material/metadata/layer attributes as well as other supplementary information like [PMI](#)⁴².

https://en.wikipedia.org/wiki/Open_Cascade_Technology

29 https://en.wikipedia.org/wiki/Open-source_software

30 https://en.wikipedia.org/wiki/Computer-aided_design

31 https://en.wikipedia.org/wiki/Computer-aided_manufacturing

32 https://en.wikipedia.org/wiki/Computer-aided_engineering

33 https://en.wikipedia.org/wiki/Open_Cascade

34 https://en.wikipedia.org/wiki/ISO_10303

35 <https://en.wikipedia.org/wiki/IGES>

36 <https://en.wikipedia.org/wiki/glTF>

37 https://en.wikipedia.org/wiki/Wavefront_.obj_file

38 [https://en.wikipedia.org/wiki/STL_\(file_format\)](https://en.wikipedia.org/wiki/STL_(file_format))

39 <https://en.wikipedia.org/wiki/VRML>

40 https://en.wikipedia.org/wiki/Open_Cascade_Technology#cite_note-18

41 https://en.wikipedia.org/wiki/Open_Cascade_Technology#cite_note-19

42 https://en.wikipedia.org/wiki/Product_and_manufacturing_information

EXPRESS Reserved Words

ISO 10303-11:2004

p. 14

7.2 Reserved words

The reserved words of EXPRESS are the keywords and the names of built-in constants, functions and procedures. The reserved words shall not be used as identifiers. The reserved words of EXPRESS are described below.

7.2.1 Keywords

The EXPRESS keywords are shown in Table 1.

NOTE Keywords have an uppercase production which represents the literal. This is to enable easier reading of the syntax productions.

Table 1 – EXPRESS keywords

ABSTRACT	AGGREGATE	ALIAS	ARRAY
AS	BAG	BASED_ON	BEGIN
BINARY	BOOLEAN	BY	CASE
CONSTANT	DERIVE	ELSE	END
END_ALIAS	END_CASE	END_CONSTANT	END_ENTITY
END_FUNCTION	END_IF	END_LOCAL	END_PROCEDURE
END_REPEAT	END_RULE	END_SCHEMA	END_SUBTYPE_CONSTRAINT
END_TYPE	ENTITY	ENUMERATION	ESCAPE
EXTENSIBLE	FIXED	FOR	FROM
FUNCTION	GENERIC	GENERIC_ENTITY	IF
INTEGER	INVERSE	LIST	LOCAL
LOGICAL	NUMBER	OF	ONEOF
OPTIONAL	OTHERWISE	PROCEDURE	QUERY
REAL	RENAMED	REFERENCE	REPEAT
RETURN	RULE	SCHEMA	SELECT
SET	SKIP	STRING	SUBTYPE
SUBTYPE_CONSTRAINT	SUPERTYPE	THEN	TO
TOTAL_OVER	TYPE	UNIQUE	UNTIL
USE	VAR	WHERE	WHILE
WITH			

7.2.2 Reserved words which are operators

The operators defined by reserved words are shown in Table 2. See clause 12 for the definition of these operators.

Table 2 – EXPRESS reserved words which are operators

AND	ANDOR	DIV	IN
LIKE	MOD	NOT	OR
XOR			

1.7.3.3.1.3 UML



Unified Modeling Language

The **Unified Modeling Language (UML)** is a general-purpose, developmental, [modeling language](#)⁴³ in the field of [software engineering](#)⁴⁴ that is intended to provide a standard way to visualize the design of a system.

https://en.wikipedia.org/wiki/Unified_Modeling_Language

 (see page 86)

⁴³ https://en.wikipedia.org/wiki/Modeling_language

⁴⁴ https://en.wikipedia.org/wiki/Software_engineering

UML - OCL extension



Encyclopedia of Database Systems

Definition

The Unified Modeling Language (UML) includes a textual language called Object Constraint Language (OCL). OCL allows users to navigate class diagrams, to formulate queries, and to restrict class diagrams with integrity constraints. From a practical perspective, the OCL may be viewed as an object-oriented version of the Structured Query Language (SQL) originally developed for the relational data model. From a theoretical perspective, OCL may be viewed as a variant of first-order predicate logic with quantifiers on finite domains only. OCL has a well-defined syntax [1⁴⁵, 3⁴⁶] and semantics [2⁴⁷].

...

Recommended Reading

1. OMG (ed.). OMG Object Constraint Language Specification. OMG, 2007. www.omg.org⁴⁸.
2. Richters M. and Gogolla M. On Formalizing the UML Object Constraint Language OCL. In Proc. 17th Int. Conf. on Conceptual Modeling. 1998, pp. 449–464. [Google Scholar](#)⁴⁹
3. Warmer J. and Kleppe A. The Object Constraint Language: Getting Your Models Ready for MDA. Addison-Wesley, Reading, MA, 2003. [Google Scholar](#)⁵⁰

⁴⁵ https://link.springer.com/referenceworkentry/10.1007%2F978-0-387-39940-9_810#CR1_810

⁴⁶ https://link.springer.com/referenceworkentry/10.1007%2F978-0-387-39940-9_810#CR3_810

⁴⁷ https://link.springer.com/referenceworkentry/10.1007%2F978-0-387-39940-9_810#CR2_810

⁴⁸ <http://www.omg.org/>

⁴⁹ <https://scholar.google.com/scholar?q=Richters%20M.%20and%20Gogolla%20M.%20On%20Formalizing%20the%20UML%20Object%20Constraint%20Language%20OCL.%20In%20Proc.%2017th%20Int.%20Conf.%20on%20Conceptual%20Modeling.%201998%2C%20pp.%20449%E2%80%93464>

⁵⁰ http://scholar.google.com/scholar_lookup?title=The%20Object%20Constraint%20Language%3A%20Getting%20Your%20Models%20Ready%20for%20MDA&author=J..%20Warmer&author=A..%20Kleppe&publication_year=2003

ODM (UML Profile)



Ontology Definition MetaModel

The **Ontology Definition MetaModel** (ODM) is an [Object Management Group](https://en.wikipedia.org/wiki/Object_Management_Group)⁵¹ (OMG) specification to make the concepts of [Model-Driven Architecture](https://en.wikipedia.org/wiki/Model-Driven_Architecture)⁵² applicable to the engineering of [ontologies](https://en.wikipedia.org/wiki/Ontology_(information_science))⁵³. Hence, it links [Common Logic](https://en.wikipedia.org/wiki/Common_Logic)⁵⁴ (CL), the [Web Ontology Language](https://en.wikipedia.org/wiki/Web_Ontology_Language)⁵⁵ (OWL), and the [Resource Description Framework](https://en.wikipedia.org/wiki/Resource_Description_Framework)⁵⁶ (RDF).

https://en.wikipedia.org/wiki/Ontology_Definition_MetaModel



(see page 88)



OWL and RDF were initially defined to provide an XML-based machine to machine interchange of metadata and semantics. ODM now integrates these into visual modeling, giving a standard well-defined process for modeling the ontology, as well as, allowing for interoperability with other modeling based on languages like UML, SysML and UPDM.



ONTOLOGY DEFINITION METAMODEL™ (ODM™)

The specification defines a family of independent metamodels, related profiles, and mappings among the metamodels corresponding to several international standards for ontology and Topic Maps definition, as well as capabilities supporting conventional modeling paradigms for capturing conceptual knowledge, such as entity-relationship modeling.

The ODM is applicable to knowledge representation, conceptual modeling, formal taxonomy development and ontology definition, and enables the use of a variety of enterprise models as starting points for ontology development through mappings to UML and MOF. ODM-based ontologies can be used to support:

- interchange of knowledge among heterogeneous computer systems,
- representation of knowledge in ontologies and knowledge bases,
- specification of expressions that are the input to or output from inference engines.

[formal-14-09-02 - Ontology Definition Metamodel v1.1.pdf](#)

51 https://en.wikipedia.org/wiki/Object_Management_Group

52 https://en.wikipedia.org/wiki/Model-Driven_Architecture

53 [https://en.wikipedia.org/wiki/Ontology_\(information_science\)](https://en.wikipedia.org/wiki/Ontology_(information_science))

54 https://en.wikipedia.org/wiki/Common_Logic

55 https://en.wikipedia.org/wiki/Web_Ontology_Language

56 https://en.wikipedia.org/wiki/Resource_Description_Framework

<https://www.omg.org/odm/>



"ODM Specification

"The lack of reliable set semantics and model theory for UML prevents the use of automated reasoners on UML models. Such a capability is important to applying Model Drive Architecture to systems integration. A reasoner can automatically determine if two models are compatible, assuming they have a rigorous semantics and axioms are defined to relate concepts in the various systems.""

"8.5 Why Common Logic over OCL?Common Logic (CL) is qualitatively different from some of the other metamodels in that it represents a dialect of traditional first order logic, rather than a modeling language. UML already supports a constraint (rule) language, which includes similar logic features, OCL [OCL], so why not use it?

The short answer to that question is that the ODM does include OCL in the same way it includes UML. Unfortunately, just as UML lacks a formal model theoretic semantics, OCL also has neither a formal model theory nor a formal proof theory, and thus cannot be used for automated reasoning (today). Common Logic, on the other hand, has both, and therefore can be used either as an expression language for ontology definition or as an ontology development language in its own right."

1.7.3.3.1.4 Semantic Web Family

- [RDF \(RDFS\) \(see page 91\)](#)
- [OWL \(see page 92\)](#)

RDF (RDFS)**Resource Description Framework (Schema)**

The **Resource Description Framework (RDF)** is a family of [World Wide Web Consortium](#)⁵⁷ (W3C) [specifications](#)^{58[1]}⁵⁹ originally designed as a [metadata](#)⁶⁰ [data model](#)⁶¹. It has come to be used as a general method for conceptual description or modeling of information that is implemented in [web resources](#)⁶², using a variety of syntax notations and [data serialization](#)⁶³ formats. It is also used in [knowledge management](#)⁶⁴ applications.

https://en.wikipedia.org/wiki/Resource_Description_Framework

(see page 91)

57 https://en.wikipedia.org/wiki/World_Wide_Web_Consortium

58 <https://en.wikipedia.org/wiki/Specification>

59 https://en.wikipedia.org/wiki/Resource_Description_Framework#cite_note-1

60 <https://en.wikipedia.org/wiki/Metadata>

61 https://en.wikipedia.org/wiki/Data_model

62 https://en.wikipedia.org/wiki/Web_resource

63 https://en.wikipedia.org/wiki/Data_serialization

64 https://en.wikipedia.org/wiki/Knowledge_management

OWL



Web Ontology Language

The **Web Ontology Language (OWL)** is a family of [knowledge representation](#)⁶⁵ languages for authoring [ontologies](#)⁶⁶.

The OWL languages are characterized by [formal semantics](#)⁶⁷. They are built upon the [World Wide Web Consortium](#)⁶⁸'s (W3C) [XML](#)⁶⁹ standard for objects called the [Resource Description Framework](#)⁷⁰ (RDF).^[2]⁷¹ OWL and RDF have attracted significant academic, medical and commercial interest.

https://en.wikipedia.org/wiki/Web_Ontology_Language

(see page 92)

65 https://en.wikipedia.org/wiki/Knowledge_representation_and_reasoning

66 [https://en.wikipedia.org/wiki/Ontology_\(information_science\)](https://en.wikipedia.org/wiki/Ontology_(information_science))

67 [https://en.wikipedia.org/wiki/Semantics_\(computer_science\)](https://en.wikipedia.org/wiki/Semantics_(computer_science))

68 https://en.wikipedia.org/wiki/World_Wide_Web_Consortium

69 <https://en.wikipedia.org/wiki/XML>

70 https://en.wikipedia.org/wiki/Resource_Description_Framework

71 https://en.wikipedia.org/wiki/Web_Ontology_Language#cite_note-2

1.7.3.3.1.5 TPTP



The TPTP Problem Library for Automated Theorem Proving

The TPTP (Thousands of Problems for Theorem Provers) is a library of test problems for [automated theorem proving](http://www.tptp.org/)⁷² (ATP) systems.

<http://www.tptp.org/>

 (see page 93)

⁷² <http://www.tptp.org/OverviewOfATP.html>

1.7.3.3.2 Others

- [XML \(XSD\)](#) (see page 95)
- [JSON](#) (see page 97)
- [RDBMS](#) (see page 98)
- [LaTeX](#) (see page 100)

1.7.3.3.2.1 XML (XSD)



Extensible Markup Language

Extensible Markup Language (XML) is a [markup language](#)⁷³ that defines a set of rules for encoding [documents](#)⁷⁴ in a [format](#)⁷⁵ that is both [human-readable](#)⁷⁶ and [machine-readable](#)⁷⁷. The [World Wide Web Consortium](#)⁷⁸'s XML 1.0 Specification of 1998 and several other related specifications—all of them free [open standards](#)⁷⁹—define XML.

<https://en.wikipedia.org/wiki/XML>

(see page 95)

Sub-Pages

- [XML - xlink](#) (see page 96)

⁷³ https://en.wikipedia.org/wiki/Markup_language

⁷⁴ https://en.wikipedia.org/wiki/Electronic_document

⁷⁵ https://en.wikipedia.org/wiki/File_format

⁷⁶ https://en.wikipedia.org/wiki/Human-readable_medium

⁷⁷ https://en.wikipedia.org/wiki/Machine-readable_data

⁷⁸ https://en.wikipedia.org/wiki/World_Wide_Web_Consortium

⁷⁹ https://en.wikipedia.org/wiki/Open_standard

XML - xlink



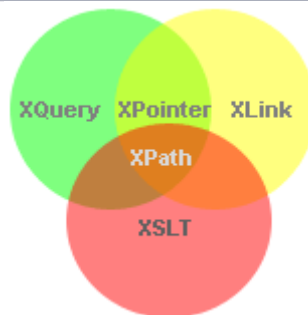
XLink

XLink is used to create hyperlinks in XML documents.

- XLink is used to create hyperlinks within XML documents
- Any element in an XML document can behave as a link
- With XLink, the links can be defined outside the linked files
- XLink is a W3C Recommendation

https://www.w3schools.com/xml/xml_xlink.asp

```
<fact xlink:type="simple" xlink:href="https://dog.com/dogbreeds.xml">
```



XPointer

- XPointer allows links to point to specific parts of an XML document
- XPointer uses XPath expressions to navigate in the XML document
- XPointer is a W3C Recommendation

```
<fact xlink:type="simple" xlink:href="https://dog.com/dogbreeds.xml#Rottweiler">
```

1.7.3.3.2.2 JSON



JavaScript Object Notation

JavaScript Object Notation is an [open standard](#)⁸⁰ [file format](#)⁸¹, and data interchange format, that uses [human-readable](#)⁸² text to store and transmit data objects consisting of [attribute–value pairs](#)⁸³ and [array data types](#)⁸⁴ (or any other [serializable](#)⁸⁵ value). It is a very common [data](#)⁸⁶ format, with a diverse range of applications, such as serving as a replacement for [XML](#)⁸⁷ in [AJAX](#)⁸⁸ systems.

<https://en.wikipedia.org/wiki/JSON>

(see page 97)

80 https://en.wikipedia.org/wiki/Open_standard

81 https://en.wikipedia.org/wiki/File_format

82 https://en.wikipedia.org/wiki/Human-readable_medium

83 https://en.wikipedia.org/wiki/Attribute%E2%80%93value_pair

84 https://en.wikipedia.org/wiki/Array_data_type

85 <https://en.wikipedia.org/wiki/Serialization>

86 <https://en.wikipedia.org/wiki/Data>

87 <https://en.wikipedia.org/wiki/XML>

88 [https://en.wikipedia.org/wiki/Ajax_\(programming\)](https://en.wikipedia.org/wiki/Ajax_(programming))

1.7.3.3.2.3 RDBMS



Relational Database Management System

Connolly and Begg define Database Management System (DBMS) as a "software system that enables users to define, create, maintain and control access to the database". RDBMS is an extension of that acronym that is sometimes used when the underlying database is relational.

https://en.wikipedia.org/wiki/Relational_database#RDBMS

 (see page 98)

SQL



Structured Query Language

Structured Query Language is a [domain-specific language](https://en.wikipedia.org/wiki/Domain-specific_language)⁸⁹ used in programming and designed for managing data held in a [relational database management system](https://en.wikipedia.org/wiki/Relational_database_management_system)⁹⁰ (RDBMS), or for stream processing in a [relational data stream management system](https://en.wikipedia.org/wiki/Relational_data_stream_management_system)⁹¹ (RDSMS). It is particularly useful in handling [structured data](https://en.wikipedia.org/wiki/Data_model)⁹², i.e. data incorporating relations among entities and variables.

<https://en.wikipedia.org/wiki/SQL>



(see page 99)

⁸⁹ https://en.wikipedia.org/wiki/Domain-specific_language

⁹⁰ https://en.wikipedia.org/wiki/Relational_database_management_system

⁹¹ https://en.wikipedia.org/wiki/Relational_data_stream_management_system

⁹² https://en.wikipedia.org/wiki/Data_model

1.7.3.3.2.4 LaTeX

<https://www.latex-project.org/>



LaTeX – A document preparation system

LaTeX is a high-quality typesetting system; it includes features designed for the production of technical and scientific documentation. LaTeX is the de facto standard for the communication and publication of scientific documents. LaTeX is available as [free software](#)⁹³.

⁹³ <https://www.latex-project.org/lppl/>

1.7.3.4 OMSF - Tools

We undertook a brief survey and analysis of the tools that we would consider for use with the multi-language ecosystem. An overall picture is given in the first section, [Tools Lists \(see page 102\)](#). The second section describes the [Candidate COTS Tools \(see page 106\)](#). The final section, [Functionality \(see page 119\)](#), expands the description for those tools deal with either translating from one language to another or with visualisation.

1.7.3.4.1 Tools Lists

This section gives an overall picture of the tools that we have considered for use with the multi-language ecosystem. There are two sub-sections, one for COTS tools ([COTS Tools Lists \(see page 104\)](#)) the other for BYO tools ([BYO Tools Lists \(see page 103\)](#)). Each sub-section indicates whether the listed tools are proposed as candidates.

1.7.3.4.1.1 BYO Tools Lists

Candidate Python BYO Tools

Tool	Focus	Link	Comments
untangle	XML	https://github.com/stchris/untangle	python module used to manipulate XML
minidom	XML	https://docs.python.org/3/library/xml.dom.minidom.html	python module used to manipulate XML

 (see page 103)

Non-Shortlisted Python BYO Tools

Tool	Focus	Link	Comments
lxml	XML	https://lxml.de/	python module used to manipulate XML
xmlschema	XML	https://xmlschema.readthedocs.io/en/latest/	python module used to manipulate XML

1.7.3.4.1.2 COTS Tools Lists

Candidate COTS Tools

Tool Vendor	Tool	Links	Comments
TopQuadrant	TopBraid Composer – Maestro Edition	TopBraid Composer – Maestro Edition (see page 114)	
Altova	XMLSpy	Altova - XmlSpy (see page 113)	
Altova	MapForce	Altova - MapForce (see page 112)	Mapping XML to RDB
Protégé	Protégé	Protégé (see page 108)	
Sparx Systems	Enterprise Architect	EA Sparx (UML) (see page 107)	UML Tool.
ShapeChange	ShapeChange	ShapeChange (see page 110)	ShapeChange is a Java tool that takes application schemas constructed according to ISO 19109 from a UML model and derives implementation representations. The most commonly used target representation is XML Schema (XSD). Others are: feature catalogues in DOCX and HTML, RDF schemas, and code list dictionaries in GML and SKOS
Safe Software	FME (aka. Feature Manipulation Engine)	Safe Software (see page 109)	FME (aka. Feature Manipulation Engine) is a data integration platform with the best support for spatial data worldwide. Easily address the question of “where” and convert your data precisely for your needs. Build your own custom workflows that improve access to data and solve compatibility issues, without needing to code anything.

 (see page 104)

Non-Shortlisted COTS Tools

Tool Vendor	Tool	Link	Comments
Régis Cosnier	XSD Diagram	http://regis.cosnier.free.fr/?page=XSDDiagram	XSD Diagram is a free Xml Schema Definition (XSD) diagram viewer for Windows written in C# Framework.NET 2.0
FiForms	xs3p	https://xml.fiforms.org/xs3p/	xs3p is a schema documentation generator.
Semyon A. Chertkov	XmlPad	https://lists.w3.org/Archives/Public/xmlschema-dev/2006Sep/0077.html	An XML docs editor

1.7.3.4.2 Candidate COTS Tools

This section provides more detail on the COTS tools that were proposed as candidates in the previous section: [COTS Tools Lists](#) (see page 104).

- [EA Sparx \(UML\)](#) (see page 107)
- [Protégé](#) (see page 108)
- [Safe Software](#) (see page 109)
- [ShapeChange](#) (see page 110)
- [Altova](#) (see page 111)
- [TopBraid Composer - Maestro Edition](#) (see page 114)
- [OSI - Tools - BYO](#) (see page 115)
- [Eurostep EXPRESS Parser](#) (see page 117)
- [reeper](#) (see page 118)

1.7.3.4.2.1 EA Sparx (UML)



Enterprise Architect

The perfect enterprise wide solution to visualise, analyse, model, test and maintain all of your systems, software, processes and architectures. Enterprise Architect is the ideal platform to help you to stay in control of your workspace, support your colleagues and team, enable collaboration and build confidence within your most complex projects.

<https://sparxsystems.com/>

 (see page 107)

Sub-Pages

1.7.3.4.2 Protégé



Protégé

A free, open-source ontology editor and framework for building intelligent systems

<https://protege.stanford.edu/>

 (see page 108)

1.7.3.4.2.3 Safe Software



Feature Manipulation Engine

FME is the data integration platform with the best support for spatial data worldwide.

<https://fme.safe.com/platform/>

 (see page 109)

1.7.3.4.2.4 ShapeChange



ShapeChange

ShapeChange is an open-source Java tool, mainly developed by interactive instruments, that takes application schemas constructed according to ISO 19109 from a UML model and derives implementation representations. UML models from [Enterprise Architect \(Sparx Systems\)](https://sparxsystems.com/products/ea/)⁹⁴ can be accessed directly from ShapeChange.

<https://shapechange.net/>

<https://www.interactive-instruments.de/en/shapechange/>



(see page 110)



The most commonly used target representation is **XML Schema** and several encoding rules specified in standards or by communities are supported.

In addition to the generation of XML Schema documents, the generation of **Schematron** documents from OCL constraints in the UML model is supported.

Other target representations include **JSON Schema**, **feature catalogues** in XML and HTML, **RDF Schema**, and **code list dictionaries** in GML and SKOS.

Comprehensive information about ShapeChange is available at <https://shapechange.net/>.



<https://shapechange.net/get-started/>

download files

See Also

[UML to XML - ShapeChange \(see page 138\)](#)

⁹⁴ <https://sparxsystems.com/products/ea/>

1.7.3.4.2.5 Altova

Sub-Pages

- [Altova - MapForce \(see page 112\)](#)
- [Altova - XmlSpy \(see page 113\)](#)

See Also

[XML to Tables - Altova \(see page 122\)](#)

Altova - MapForce



MapForce

Altova MapForce is an award-winning, graphical data mapping tool for any-to-any conversion and integration. Its powerful data mapping tools convert your data instantly and provide multiple options to automate recurrent transformations.

<https://www.altova.com/mapforce>

(see page 112)

See Also

[XML to Tables - Altova - MapForce \(see page 123\)](#)

Altova - XmlSpy



XmlSpy

XMLSpy JSON and XML Editor gives developers the tools they need to build the most sophisticated applications with its graphical schema designer, code generation, file converters, debuggers, and profilers for working with XSD, XSLT, XQuery, XBRL, SOAP, and more.

<https://www.altova.com/xmlspy-xml-editor>

 (see page 112)

See Also

[XML to Tables - Altova - XmlSpy](#) (see page 124)

1.7.3.4.2.6 TopBraid Composer - Maestro Edition



TopBraid Composer Maestro Edition

TopBraid Composer™ Maestro Edition (TBC-ME) combines world's leading semantic web modeling capabilities with the most comprehensive data conversion options and a powerful Integrated Development Environment (IDE) for implementing Knowledge Graph/Linked Data services.

<https://www.topquadrant.com/products/topbraid-composer/>

 (see page 114)

1.7.3.4.2.7 OSI - Tools - BYO

- [OSI - Tools - BYO - migrate_express_to_nf_ea_com](#) (see page 116)

OSI - Tools - BYO - migrate_express_to_nf_ea_com



Pilot code devolped and tested

1.7.3.4.2.8 Eurostep EXPRESS Parser

Eurostep EXPRESS Parser (Eep!) V1.4.41
(C) Eurostep Limited 1999-2020

[eep.zip](#)

Usage : eep [options] file

options one or more of:

- 2 EXPRESS edition 2 parsing
- t EXPRESS edition 1 TC 2 parsing
- o EXPRESS edition 1 TC 1 parsing
- i Information messages on
- w Warning messages on
- e Evaluated set evaluation on
- p POSC restrictions applied
- x XML export
- X XMI export
- f Export parsed express file
- c Comment parsing (only of use with -f, -x or -X)

1.7.3.4.2.9 reeper

<https://sourceforge.net/projects/reeper/>

A tool for reaping the contents of ISO EXPRESS data models by mapping them into other modeling and programming languages.

1.7.3.4.3 Functionality

This section expands the description for those tools dealing with either translating from one language to another or with visualisation.

- [Translation Functionalities \(see page 120\)](#)
- [Visualisation Functionalities \(see page 142\)](#)

1.7.3.4.3.1 Translation Functionalities

- [XML to Tables](#) (see page 121)
- [XSD-XML to Python](#) (see page 128)
- [UML to XML](#) (see page 137)
- [Translation Design Choices](#) (see page 139)

See Also

[OMS - Architecture \(Proposed\)](#) (see page 54)

XML to Tables

- [XML to Tables - Altova \(see page 122\)](#)
- [XML to Tables - Protégé \(see page 126\)](#)
- [XML to Tables - Topbraid \(see page 127\)](#)

XML to Tables - Altova**Altova**

https://www.altova.com/xmlconversiontool-a?gclid=EAIaIQobChMIy5eY152G6wIVw7HtCh2gOgS_EAAYASAAEgJ8avD_BwE

<https://www.altova.com/xmlspy-xml-editor>

<https://www.altova.com/xmlspy-xml-editor/download#>



https://www.altova.com/manual/XMLSpy/spyenterprise/xsmenuconvert_createdbstructurefromxsd.html

Create DB Structure from XML Schema

XMLSpy allows you to create an empty database (or skeleton database) based on an existing schema file. The method described below is generally the same for each type of database.

1. Open the schema file in Schema/WSDL View
2. Select the menu command Convert | Create DB Structure from XML Schema. This pops up the Connect to a Data Source dialog, which enables you to connect to a database (DB).
3. Use the steps described in the section Connecting to a Data Source to connect to the required database. For example, to connect to a Microsoft Access database, select the Microsoft Access radio button, and continue the process to select a database. You can use an existing database or create a new database in which the schema structure will be contained.
4. In the Create DB Structure from XML Schema dialog, tables are created from the schema and displayed in a tree format at the location where they will occur in the DB. For example, in the screenshot below, the Address table is created and selected for export. Tables that should not be exported should be deselected (by unchecking the check box or selecting the appropriate item from the context menu for that table)



<https://www.altova.com/forum/framehelper.aspx?g=posts&t=1100001271>

Forum Thread - XSD to SQL Tables

XML to Tables - Altova - XmlSpy

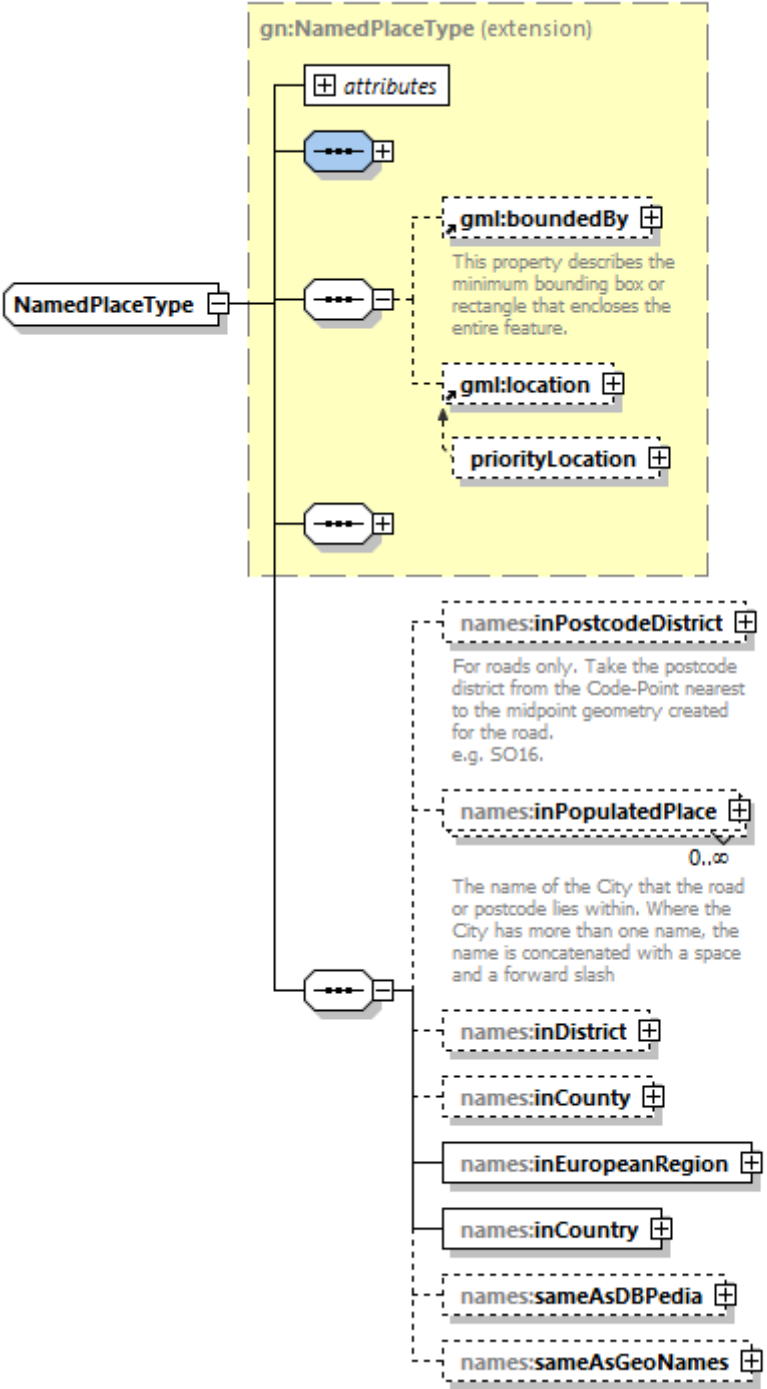
XML (gml) visualised and edited as tree(grid)

XML	
gml:FeatureCollection	
= xmlns:gml	http://www.opengis.net/gml/3.2
= xsi:schemaLocation	http://www.opengis.net/gml/3.2 http://schemas.opengis.net/gml/3.2.1/gml.xsd http://namespaces.os.uk/open/names/1.0/ http://www.ordnancesurvey.co.uk/xml/open/names/1.0/OSOpenNames.xsd
= gml:id	OS_Open_Names
= xmlns:xsi	http://www.w3.org/2001/XMLSchema-instance
= xmlns:xs	http://www.w3.org/2001/XMLSchema
= xmlns:xlink	http://www.w3.org/1999/xlink
= xmlns:gmd	http://www.isotc211.org/2005/gmd
= xmlns:gco	http://www.isotc211.org/2005/gco
= xmlns:gss	http://www.isotc211.org/2005/gss
= xmlns:gts	http://www.isotc211.org/2005/gts
= xmlns:gsr	http://www.isotc211.org/2005/gsr
= xmlns:names	http://namespaces.os.uk/open/names/1.0/
= xmlns:au	urn:x-inspire:specification:gmlas:AdministrativeUnits:3.0
= xmlns:stat	urn:x-inspire:specification:gmlas:StatisticalUnits:0.0
= xmlns:base	urn:x-inspire:specification:gmlas:BaseTypes:3.2
= xmlns:gn	urn:x-inspire:specification:gmlas:GeographicalNames:3.0
gml:featureMember (3)	
<> names:NamedPlace	
1 names:NamedPlace	gml:id= osgb4000000074602205
2 names:NamedPlace	gml:id= osgb4000000074748955
3 names:NamedPlace	gml:id= osgb4000000074749187

XSD visualised and edited as tree(grid)

XML	
schema	
= xmlns	http://www.w3.org/2001/XMLSchema
= xmlns:au	urn:x-inspire:specification:gmlas:AdministrativeUnits:3.0
= xmlns:gml	http://www.opengis.net/gml/3.2
= xmlns:gn	urn:x-inspire:specification:gmlas:GeographicalNames:3.0
= xmlns:names	http://namespaces.os.uk/open/names/1.0/
= targetNamespace	http://namespaces.os.uk/open/names/1.0/
= elementFormDefault	qualified
= version	1.0.0
import namespace=	http://www.opengis.net/gml/3.2 schemaLocation= http://schemas.opengis.net/gml/3.2.1/gml.xsd
import namespace=	urn:x-inspire:specification:gmlas:AdministrativeUnits:3.0 schemaLocation= http://inspire.ec.europa.eu/schemas/au/3.0/inspire-au.xsd
import namespace=	urn:x-inspire:specification:gmlas:GeographicalNames:3.0 schemaLocation= http://inspire.ec.europa.eu/schemas/gn/3.0/inspire-gn.xsd
<!-- Comment	XML Schema document created by ShapeChange - http://shapechange.net/
element	
= name	NamedPlace
= type	names:NamedPlaceType
= substitutionGroup	gn:NamedPlace
annotation	
complexType name=	NamedPlaceType
complexType name=	NamedPlacePropertyType

XSD visualised and edited as graph:



XML to Tables - Protégé

Protégé



https://protegewiki.stanford.edu/wiki/XML_Tab

The XML Tab enables users to import an XML document into Protege, creating a set of classes and instances in a knowledge base which correspond to the entries in the XML document



https://protegewiki.stanford.edu/wiki/PrF_UG_files_all

You can save the ontology-related information for a Protege-Frames project as a single table in a JDBC database. Note that this is a "flat" table that does not use any special database constructs to represent the information. Information specific to the Protege-Frames user interface is stored in a separate Protege project (.pprj) file.

XML to Tables - Topbraid

Topbraid

<https://www.topquadrant.com/release-notes/7-1>

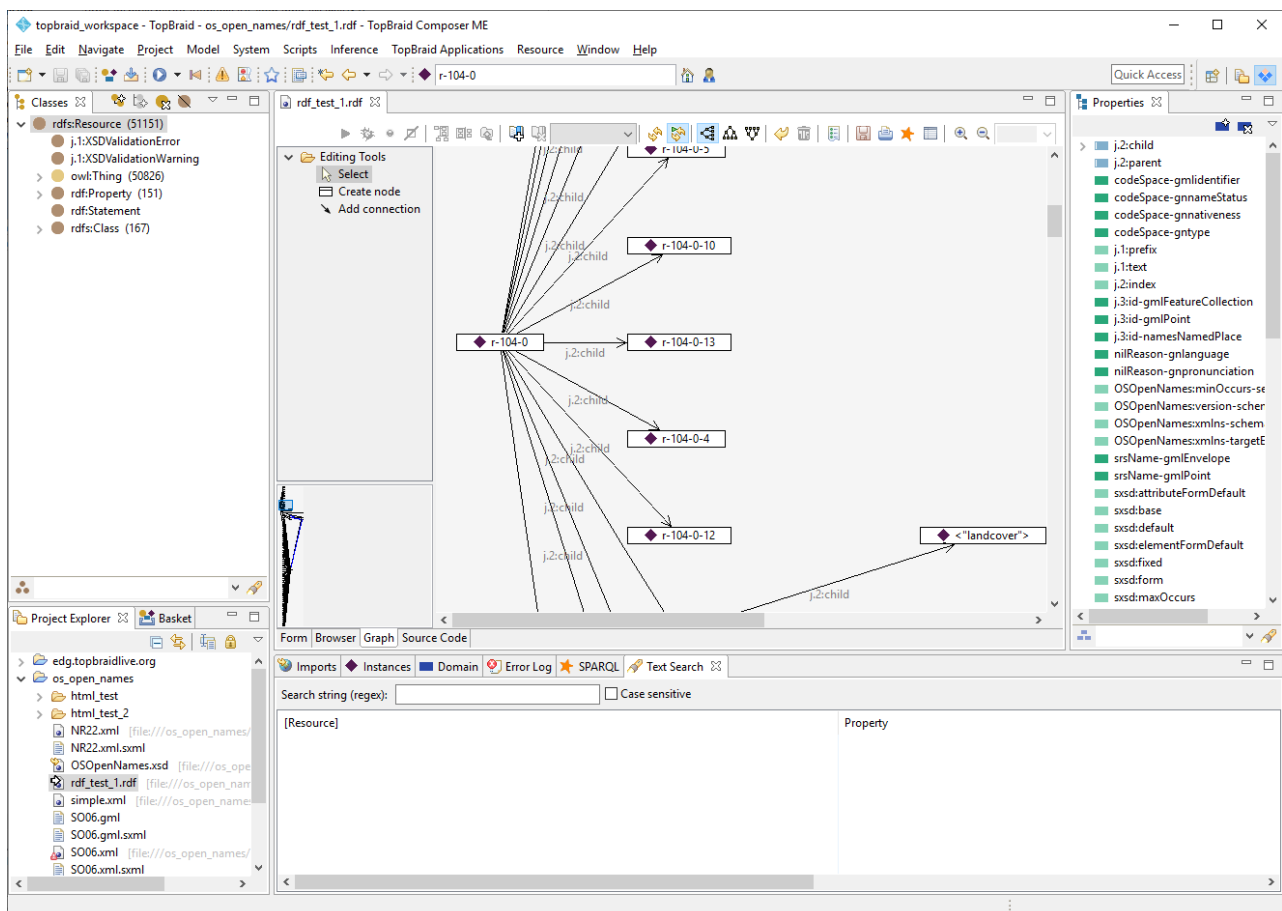
https://www.youtube.com/watch?v=_qHq3HcXHP4

Only topbraid composer maestro edition has XML import

Output databases might just be triple stores - need to check

XSD can be imported into TopBraid and then 'loaded' with XML. This can be exported as

Examples



XSD-XML to Python

- [XSD-XML to Python - lxml \(see page 129\)](#)
- [XSD-XML to Python - minidom \(see page 131\)](#)
- [XSD-XML to Python - untangle \(see page 134\)](#)
- [XSD-XML to Python - xmlschema \(see page 136\)](#)

XSD-XML to Python - lxml

<https://lxml.de/>

Notes

```
schema = etree.XMLSchema(file=name_schema_path)
parser = objectify.makeparser(schema = schema)
data_as_object_tree = objectify.fromstring(data, parser)
```

XML as tree of objects. Namespaces available but all other sets will need to be coded for.

```

▼ data_as_object_tree = {ObjectifiedElement}
  ▼ attrib = {_Attrib} {'http://www.w3.org/2001/XMLSchema-instance': 'http://www.op
    01 base = {NoneType} None
  ▼ featureMember = {ObjectifiedElement}
    ▼ attrib = {_Attrib} {}
      01 base = {NoneType} None
    > nsmap = {dict} <class 'dict'>: {'gml': 'http://www.opengis.net/gml/3.2', 'xsi': 'http://www.w3.org
      01 prefix = {str} 'gml'
      01 sourceline = {int} 2
      01 tag = {str} '{http://www.opengis.net/gml/3.2}featureMember'
      01 tail = {NoneType} None
      01 text = {NoneType} None
    ▼ nsmap = {dict} <class 'dict'>: {'gml': 'http://www.opengis.net/gml/3.2', 'xsi': 'http://www.w3.org/200
      01 'gml' (1856662406008) = {str} 'http://www.opengis.net/gml/3.2'
      01 'xsi' (1856662405896) = {str} 'http://www.w3.org/2001/XMLSchema-instance'
      01 'xs' (1856662406064) = {str} 'http://www.w3.org/2001/XMLSchema'
      01 'xlink' (1856662406120) = {str} 'http://www.w3.org/1999/xlink'
      01 'gmd' (1856662406176) = {str} 'http://www.isotc211.org/2005/gmd'
      01 'gco' (1856662406232) = {str} 'http://www.isotc211.org/2005/gco'
      01 'gss' (1856662406288) = {str} 'http://www.isotc211.org/2005/gss'
      01 'gts' (1856662406344) = {str} 'http://www.isotc211.org/2005/gts'
      01 'gsr' (1856662406400) = {str} 'http://www.isotc211.org/2005/gsr'
      01 'names' (1856662406456) = {str} 'http://namespaces.os.uk/open/names/1.0/'
      01 'au' (1856662406512) = {str} 'urn:x-inspire:specification:gmlas:AdministrativeUnits:3.0'
      01 'stat' (1856662406568) = {str} 'urn:x-inspire:specification:gmlas:StatisticalUnits:0.0'
      01 'base' (1856662406624) = {str} 'urn:x-inspire:specification:gmlas:BaseTypes:3.2'
      01 'gn' (1856662406680) = {str} 'urn:x-inspire:specification:gmlas:GeographicalNames:3.0'
      01 __len__ = {int} 14
    01 prefix = {str} 'gml'
    01 sourceline = {int} 1
    01 tag = {str} '{http://www.opengis.net/gml/3.2}FeatureCollection'
    01 tail = {NoneType} None
    01 text = {NoneType} None

```

XSD-XML to Python - minidom

<https://docs.python.org/3/library/xml.dom.minidom.html>

<https://stackabuse.com/reading-and-writing-xml-files-in-python/>

Notes

parse() creates XMLs and XSDs as DOM Document objects

```
data_as_dom = parse(name_data_path)
```

```

data_as_dom = {Document} <xml.dom.minidom.Document object at 0x00000245...
01 ATTRIBUTE_NODE = {int} 2
01 CDATA_SECTION_NODE = {int} 4
01 COMMENT_NODE = {int} 8
01 DOCUMENT_FRAGMENT_NODE = {int} 11
01 DOCUMENT_NODE = {int} 9
01 DOCUMENT_TYPE_NODE = {int} 10
01 ELEMENT_NODE = {int} 1
01 ENTITY_NODE = {int} 6
01 ENTITY_REFERENCE_NODE = {int} 5
01 NOTATION_NODE = {int} 12
01 PROCESSING_INSTRUCTION_NODE = {int} 7
01 TEXT_NODE = {int} 3
> _child_node_types = {tuple} <class 'tuple': (1, 7, 8, 10)
> _elem_info = {dict} <class 'dict': {}
> _id_cache = {dict} <class 'dict': {}
01 _id_search_stack = {NoneType} None
01 _magic_id_count = {int} 0
01 actualEncoding = {NoneType} None
01 async_ = {bool} False
01 attributes = {NoneType} None
> childNodes = {NodeList} [<DOM Element: gml:FeatureCollection at 0x2498f5...
01 doctype = {NoneType} None
> documentElement = {Element} <DOM Element: gml:FeatureCollection at 0x2...
01 documentURI = {NoneType} None
01 encoding = {str} 'UTF-8'
01 errorHandler = {NoneType} None
> firstChild = {Element} <DOM Element: gml:FeatureCollection at 0x2498f5dbb...
> implementation = {DOMImplementation} <xml.dom.minidom.DOMImpleme...
> lastChild = {Element} <DOM Element: gml:FeatureCollection at 0x2498f5dbb...
01 localName = {NoneType} None
01 namespaceURI = {NoneType} None
01 nextSibling = {NoneType} None

```

```

01 nodeName = {str} '#document'
01 nodeType = {int} 9
01 nodeValue = {NoneType} None
01 ownerDocument = {NoneType} None
01 parentNode = {NoneType} None
01 prefix = {NoneType} None
01 previousSibling = {NoneType} None
01 standalone = {NoneType} None
01 strictErrorChecking = {bool} False
01 version = {str} '1.0'

```

```
schema_as_dom = parse(name_schema_path)
```

```

v  schema_as_dom = {Document} <xml.dom.minidom.Document object at 0x000002498F4FF2E8>
01 ATTRIBUTE_NODE = {int} 2
01 CDATA_SECTION_NODE = {int} 4
01 COMMENT_NODE = {int} 8
01 DOCUMENT_FRAGMENT_NODE = {int} 11
01 DOCUMENT_NODE = {int} 9
01 DOCUMENT_TYPE_NODE = {int} 10
01 ELEMENT_NODE = {int} 1
01 ENTITY_NODE = {int} 6
01 ENTITY_REFERENCE_NODE = {int} 5
01 NOTATION_NODE = {int} 12
01 PROCESSING_INSTRUCTION_NODE = {int} 7
01 TEXT_NODE = {int} 3
> 1 2 _child_node_types = {tuple} <class 'tuple': (1, 7, 8, 10)
>  _elem_info = {dict} <class 'dict': {}
>  _id_cache = {dict} <class 'dict': {}
01 _id_search_stack = {NoneType} None
01 _magic_id_count = {int} 0
01 actualEncoding = {NoneType} None
01 async_ = {bool} False
01 attributes = {NoneType} None
v  childNodes = {NodeList} [<DOM Element: schema at 0x2498f5ff178>]
01 length = {int} 1
>  0 = {Element} <DOM Element: schema at 0x2498f5ff178>
01 __len__ = {int} 1
01 doctype = {NoneType} None
>  documentElement = {Element} <DOM Element: schema at 0x2498f5ff178>
01 documentURI = {NoneType} None
01 encoding = {str} 'UTF-8'
01 errorHandler = {NoneType} None

```



```

> errorHandler = {NoneType} None
> firstChild = {Element} <DOM Element: schema at 0x2498f5ff178>
> implementation = {DOMImplementation} <xml.dom.minidom.DOMImplementation object at
> lastChild = {Element} <DOM Element: schema at 0x2498f5ff178>
01 localName = {NoneType} None
01 namespaceURI = {NoneType} None
01 nextSibling = {NoneType} None
01 nodeName = {str} '#document'
01 nodeType = {int} 9
01 nodeValue = {NoneType} None
01 ownerDocument = {NoneType} None
01 parentNode = {NoneType} None
01 prefix = {NoneType} None
01 previousSibling = {NoneType} None
01 standalone = {NoneType} None
01 strictErrorChecking = {bool} False
01 version = {str} '1.0'

```

XSD-XML to Python - untangle

<https://docs.python-guide.org/scenarios/xml/>

Notes

```
data = untangle.parse(name_data_path)
```

Creates simple to read object tree of XML

```

▼ data = {Element} Element <None> with attributes None, children [Element(name = gml_FeatureCollection, attributes = {'xmlns:gml': 'http://www.open
  ▼ gml_FeatureCollection = {Element} Element <gml_FeatureCollection> with attributes {'xmlns:gml': 'http://www.opengis.net/gml/3.2', 'xsi:schemaL
    ▼ gml_featureMember = {list} <class 'list'>: [Element(name = gml_featureMember, attributes = {}, cdata = '\n\n'), Element(name = gml_feature
      ▼ 0 = {Element} Element <gml_featureMember> with attributes {}, children [Element(name = names_NamedPlace, attributes = {'gml:id': 'osgl
        ▼ names_NamedPlace = {Element} Element <names_NamedPlace> with attributes {'gml:id': 'osgb4000000074602205'}, children [Element(i
          > gml_boundedBy = {Element} Element <gml_boundedBy> with attributes {}, children [Element(name = gml_Envelope, attributes = {'
            ▼ gml_identifier = {Element} Element <gml_identifier> with attributes {'codeSpace': 'http://inspire.jrc.ec.europa.eu/ids'}, children [] an
            ▼ gn_beginLifespanVersion = {Element} Element <gn_beginLifespanVersion> with attributes {}, children [] and cdata 2020-07-06T14:53
            ▼ gn_geometry = {Element} Element <gn_geometry> with attributes {}, children [Element(name = gml_Point, attributes = {'srsName':
              ▼ gml_Point = {Element} Element <gml_Point> with attributes {'srsName': 'urn:ogc:def:crs:EPSG::27700', 'gml:id': 'LOCAL_ID_0'}, cl
                ▼ gml_pos = {Element} Element <gml_pos> with attributes {}, children [] and cdata 130769.0 639961.0
              > gn_inspireId = {Element} Element <gn_inspireId> with attributes {}, children [Element(name = base_Identifier, attributes = {}, cdata =
              > gn_leastDetailedViewingResolution = {Element} Element <gn_leastDetailedViewingResolution> with attributes {}, children [Element(i
              > gn_localType = {Element} Element <gn_localType> with attributes {}, children [Element(name = gmd_LocalisedCharacterString, attr
              > gn_mostDetailedViewingResolution = {Element} Element <gn_mostDetailedViewingResolution> with attributes {}, children [Element
              > gn_name = {Element} Element <gn_name> with attributes {}, children [Element(name = gn_GeographicalName, attributes = {}, cdat
            ▼ gn_type = {Element} Element <gn_type> with attributes {'codeSpace': 'http://inspire.ec.europa.eu/codelist/NamedPlaceTypeValue'}
            ▼ names_inCountry = {Element} Element <names_inCountry> with attributes {'xlink:title': 'Scotland', 'xlink:href': 'http://data.ordnanc
            ▼ names_inCounty = {Element} Element <names_inCounty> with attributes {'xlink:title': 'Argyll and Bute', 'xlink:role': 'http://data.ordn
            ▼ names_inEuropeanRegion = {Element} Element <names_inEuropeanRegion> with attributes {'xlink:title': 'Scotland', 'xlink:href': 'http
            ▼ names_inPostcodeDistrict = {Element} Element <names_inPostcodeDistrict> with attributes {'xlink:title': 'PA42', 'xlink:href': 'http://d
          > 1 = {Element} Element <gml_featureMember> with attributes {}, children [Element(name = names_NamedPlace, attributes = {'gml:id': 'osgl
          > 2 = {Element} Element <gml_featureMember> with attributes {}, children [Element(name = names_NamedPlace, attributes = {'gml:id': 'osgl
        01 _len_ = {int} 3

```

```
schema = untangle.parse(name_schema_path)
```

Creates simple to read object tree of XSD

- ▼  **schema** = {Element} Element <None> with attributes None, children [Element(name = schema, attributes = {'xmlns': 'http://www.w3.org/2001/XMLSchema', 'xmlns:gml': 'http://www.opengis.net/gml/3.2', 'xmlns:inspire': 'urn:x-...'})]
- ▼  **schema** = {Element} Element <schema> with attributes {'xmlns': 'http://www.w3.org/2001/XMLSchema', 'xmlns:gml': 'http://www.opengis.net/gml/3.2', 'xmlns:inspire': 'urn:x-...'}, children [Element(name = complexType, attributes = {'name': 'NamedPlaceType'})]
- ▼  **complexType** = {list} <class 'list'>: [Element(name = complexType, attributes = {'name': 'NamedPlaceType'})]
- ▼  **0** = {Element} Element <complexType> with attributes {'name': 'NamedPlaceType'}, children [Element(name = complexContent, attributes = {})]
- ▼  **complexContent** = {Element} Element <complexContent> with attributes {}, children [Element(name = extension, attributes = {'base': 'gn:NamedPlaceType'})]
- ▼  **extension** = {Element} Element <extension> with attributes {'base': 'gn:NamedPlaceType'}, children [Element(name = sequence, attributes = {})]
- ▼  **sequence** = {Element} Element <sequence> with attributes {}, children [Element(name = element, attributes = {'name': 'inPostcodeDistrict'})]
- ▼  **element** = {list} <class 'list'>: [Element(name = element, attributes = {'name': 'inPostcodeDistrict'})]
- ▶  **0** = {Element} Element <element> with attributes {'name': 'inPostcodeDistrict', 'type': 'gml:Text'}
 - ▶  **1** = {Element} Element <element> with attributes {'name': 'inPopulatedPlace', 'type': 'gml:Text'}
 - ▶  **2** = {Element} Element <element> with attributes {'name': 'inDistrict', 'type': 'gml:Text'}
 - ▶  **3** = {Element} Element <element> with attributes {'name': 'inCounty', 'type': 'gml:Text'}
 - ▶  **4** = {Element} Element <element> with attributes {'name': 'inEuropeanRegion', 'type': 'gml:Text'}
 - ▶  **5** = {Element} Element <element> with attributes {'name': 'inCountry', 'type': 'gml:Text'}
 - ▶  **6** = {Element} Element <element> with attributes {'name': 'sameAsDBPedia', 'type': 'gml:Text'}
 - ▶  **7** = {Element} Element <element> with attributes {'name': 'sameAsGeoNames', 'type': 'gml:Text'}
-  **__len__** = {int} 8
- ▶  **1** = {Element} Element <complexType> with attributes {'name': 'NamedPlacePropertyType'}, children [Element(name = element, attributes = {'name': 'NamedPlace'})]
- ▶  **__len__** = {int} 2
- ▼  **element** = {Element} Element <element> with attributes {'name': 'NamedPlace', 'type': 'names:NamedPlace'}
- ▼  **annotation** = {Element} Element <annotation> with attributes {}, children [Element(name = documentation, attributes = {})]
- ▼  **documentation** = {Element} Element <documentation> with attributes {}, children [] and cdata O...
- ▼  **import** = {list} <class 'list'>: [Element(name = import, attributes = {'namespace': 'http://www.opengis.net/gml/3.2'})]
- ▼  **0** = {Element} Element <import> with attributes {'namespace': 'http://www.opengis.net/gml/3.2', 'schemaLocation': 'http://www.opengis.net/gml/3.2'}]
- ▼  **1** = {Element} Element <import> with attributes {'namespace': 'urn:x-...': 'http://www.opengis.net/gml/3.2', 'schemaLocation': 'http://www.opengis.net/gml/3.2'}]
- ▼  **2** = {Element} Element <import> with attributes {'namespace': 'urn:x-...': 'http://www.opengis.net/gml/3.2', 'schemaLocation': 'http://www.opengis.net/gml/3.2'}]
-  **__len__** = {int} 3

XSD-XML to Python - xmlschema

<https://xmlschema.readthedocs.io/en/latest/>

xmlschema

This library includes the following features:

- Full XSD 1.0 and XSD 1.1 support
- Building of XML schema objects from XSD files
- Validation of XML instances against XSD schemas
- Decoding of XML data into Python data and to JSON
- Encoding of Python data and JSON to XML
- Data decoding and encoding ruled by converter classes
- An XPath based API for finding schema's elements and attributes
- Support of XSD validation modes *strict/lax/skip*
- Remote attacks protection by default using an XMLParser that forbids entities

UML to XML

- [UML to XML - ShapeChange \(see page 138\)](#)

UML to XML - ShapeChange

See [OSI - Tools - ShapeChange](#) (see page 110)

Translation Design Choices

ADD TEXT

- [Migrations - To UML \(see page 140\)](#)
- [Migrations - To OWL \(see page 141\)](#)

Migrations - To UML

UNICLASS Case

1. Super-sub-type relations are represented in EA output in two ways
 - a. stereotyped associations (using 'type-of' stereotype)
 - b. generalisations (when the super-subtype is not labelled)
2. At the moment, the transfer of there super-sub-types to OWL is being handled differently depending on if the connector is an association or a generalisation, and that makes the Protege versions look bad
 - a. To bypass it, currently we use two ideas:
 - UML generalisations are exported as owl:subClassOf (triples)
 - UML associations with an appropriate stereotype are exported as owl:subClassOf (triples)

Migrations - To OWL

UNICLASS Case

1. Transfer of super-sub-types to OWL is being handled differently depending on if the connector is an association or a generalisation:
2. To bypass it, currently we use two ideas:
 - a. UML generalisations are exported as owl:subClassOf (triples)
 - b. UML associations with an appropriate stereotype are exported as owl:subClassOf (triples)

1.7.3.4.3.2 Visualisation Functionalities

- [Visualisation Tools - XSD](#) (see page 143)

Visualisation Tools - XSD



https://medium.com/@guy.crets_85128/xsd-visualisation-part-1-8cd61398d2ff

Tools compared

- [XSD Diagram](#)⁹⁵
- [xs3p](#)⁹⁶
- <https://lists.w3.org/Archives/Public/xmlschema-dev/2006Sep/0077.html>

<https://windowsreport.com/view-read-xml-software/>

⁹⁵ <http://regis.cosnier.free.fr/?page=XSDDiagram>

⁹⁶ <https://xml.fiforms.org/xs3p/>

1.7.4 Appendix - Work-in-Progress

Work-in-progress that needs to be completed is described on the these linked pages:

-  [Work-in-Progress - evolving counterpartology \(see page 26\)](#)
-  [Work-in-Progress - evolving doxastology \(see page 28\)](#)
-  [Work-in-Progress - evolving egonology \(see page 27\)](#)
-  [Work-in-Progress - evolving modalology \(see page 29\)](#)
-  [Work-in-Progress - plurals - formalising collections \(see page 17\)](#)

1.7.5 Appendix - Future Work

Future work is described on the these linked pages:

-  [Future Work - building a dimensional worldframewology \(see page 23\)](#)
-  [Future Work - ensuring the consistency of the ToLO \(see page 36\)](#)
-  [Future Work - evolving onomatology \(see page 30\)](#)
-  [Future Work - exploiting opportunities for formalising and automating a controlled doxastology \(see page 38\)](#)
-  [Future Work - formalising testimony, including belief management \(see page 37\)](#)
-  [Future Work - implementing an Ontology Multi-Language Support Framework \(see page 34\)](#)
-  [Future Work - increasing expressiveness \(see page 32\)](#)
-  [Future Work - tupleology - radical adaption-evolution \(see page 18\)](#)

1.8 Bibliography

- [\(Florio 2021 The Many and the One\) \(see page 148\)](#) — Florio, S., & Linnebo, Ø. (2021). The many and the one: A philosophical study of plural logic (First edition). Oxford University Press. 
- [\(Florio 2022 Core Constructional Ontology\) \(see page 149\)](#) — Florio, S., & Linnebo, Ø. (2022). Core Constructional Ontology: The Foundation for the Top-Level Ontology of the Information Management Framework (DRAFT) (v. 1.91). NDT. 
- [\(Graham 2015 NORSOK Z-014: A 21st Century Update\) \(see page 150\)](#) — Graham, J., Khan, M., & Partridge, C. (2015). NORSOK Z-014: A 21st Century Update. 59th Annual Meeting of the AACE International 2015, 1,2. 
- [\(Hansson 2022 Logic of Belief Revision\) \(see page 151\)](#) — Hansson, S. O. (2022). Logic of Belief Revision. In E. N. Zalta (Ed.), The Stanford Encyclopedia of Philosophy (Spring 2022). Metaphysics Research Lab, Stanford University. <https://plato.stanford.edu/archives/spr2022/entries/logic-belief-revision/> <https://plato.stanford.edu/archives/spr2022/entries/logic-belief-revision/>. 
- [\(ISO/IEC 21838-1:2021\) \(see page 152\)](#) — International Organization for Standardization. (2021). ISO/IEC 21838-2:2021 Information technology—Top-level ontologies (TLO)—Part 1: Requirements. International Organization for Standardization. 
- [\(ISO/IEC 24707:2018\) \(see page 153\)](#) — International Organization for Standardization. (2018). ISO/IEC 24707:2018 Information technology—Common Logic (CL)—A framework for a family of logic-based languages. International Organization for Standardization. 
- [\(Leonard 2023 Epistemological Problems of Testimony\) \(see page 154\)](#) — Leonard, N. (2023). Epistemological Problems of Testimony. In E. N. Zalta & U. Nodelman (Eds.), The Stanford Encyclopedia of Philosophy (Spring 2023). Metaphysics Research Lab, Stanford University. <https://plato.stanford.edu/archives/spr2023/entries/testimony-episprob/> <https://plato.stanford.edu/archives/spr2023/entries/testimony-episprob/>. 
- [\(Lewis 1968 Counterpart Theory and Quantified Modal Logic\) \(see page 155\)](#) — Lewis, D. K. (1968). Counterpart Theory and Quantified Modal Logic. The Journal of Philosophy, 65(5), Article 5. <https://doi-org.uow.idm.oclc.org/10.2307/2024555> <https://doi-org.uow.idm.oclc.org/10.2307/2024555> 
- [\(Lewis 1986 On the plurality of worlds\) \(see page 156\)](#) — Lewis, D. K. (1986). On the plurality of worlds. Blackwell Publishers. 
- [\(Linnebo 2022 Plural Quantification\) \(see page 157\)](#) — Linnebo, Ø. (2022). Plural Quantification. In E. N. Zalta (Ed.), The Stanford Encyclopedia of Philosophy (Spring 2022). Metaphysics Research Lab, Stanford University. <https://plato.stanford.edu/archives/spr2022/entries/plural-quant/> <https://plato.stanford.edu/archives/spr2022/entries/plural-quant/>. 
- [\(Pagin 2021 Assertion\) \(see page 158\)](#) — Pagin, P., & Marsili, N. (2021). Assertion. In E. N. Zalta (Ed.), The Stanford Encyclopedia of Philosophy (Winter 2021). Metaphysics Research Lab, Stanford University. <https://plato.stanford.edu/archives/win2021/entries/assertion/> <https://plato.stanford.edu/archives/win2021/entries/assertion/>. 
- [\(Partridge 1996 Business Objects\) \(see page 159\)](#) — Partridge, C. (1996). Business Objects: Re-Engineering for Re-Use (1st Edition). Oxford: Butterworth-Heinemann. 
- [\(Partridge 2011 An Information Model for Geospatial and Temporal References\) \(see page 170\)](#) — Partridge, C. (2011). An Information Model for Geospatial and Temporal Reference. BORO. <https://www.academia.edu/35739957/>

- Ontology_then_agentology_A_finer_grained_framework_for_enterprise_modelling https://www.academia.edu/35739957/Ontology_then_agentology_A_finer_grained_framework_for_enterprise_modelling 
- [\(Partridge 2013 Geospatial and Temporal Reference\)](#) (see page 171) — Partridge, C. (2013). Geospatial and Temporal Reference – A Case Study Illustrating (Radical) Refactoring. ONTOBRAS-2013 6th Ontology Research Seminar in Brazil. https://www.academia.edu/27433806/Geospatial_and_temporal_reference_A_case_study_illustrating_radical_refactoring 
 - [\(Partridge 2016 Formalization of the classification pattern\)](#) (see page 172) — Partridge, C., Cesare, S. de, Mitchell, A., & Odell, J. (2016). Formalization of the Classification Pattern: Survey of Classification Modeling in Information Systems Engineering. *Software & Systems Modeling*, 1–37. <https://doi.org/10.1007/s10270-016-0521-5> 
 - [\(Partridge 2018 Ontology then Agentology\)](#) (see page 173) — Partridge, C., deCesare, S., Mitchell, A., Leon, A., Gailly, F., & Khan, M. (2018). Ontology then Agentology: A Finer Grained Framework for Enterprise Modelling. *Proceedings of the 6th International Conference on Model-Driven Engineering and Software Development (MODELSWARD 2018)*, 454–463. <https://www.academia.edu/35335427> 
 - [\(Partridge 2018 Thoroughly Modern Accounting\)](#) (see page 174) — Partridge, C., Khan, M., Cesare, S. de, Gailly, F., Verdonck, M., & Mitchell, A. (2018). Thoroughly Modern Accounting: Shifting to a De Re Conceptual Pattern for Debits and Credits. *International Conference on Conceptual Modeling (ER 2018)*, 11158, 134. 
 - [\(Partridge 2019 Coordinate Systems\)](#) (see page 175) — Partridge, C., Mitchell, A., Loneragan, M., Atkinson, H., de Cesare, S., & Khan, M. (2019). Coordinate Systems: Level Ascending Ontological Options. *2019 ACM/IEEE 22nd International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*, 78–87. <https://www.academia.edu/40354620> 
 - [\(Partridge 2019 Grounding for an Enterprise Computing Nomenclature Ontology\)](#) (see page 176) — Partridge, C., Mitchell, A., & de Cesare, S. (2019). Grounding for an Enterprise Computing Nomenclature Ontology. *International Conference on Conceptual Modeling*, 457–465. 
 - [\(Partridge 2020 Implicit Requirements for Ontological Multi-Level Types in the UNICLASS Classification\)](#) (see page 177) — Partridge, C., Mitchell, A., da Silva, M., Soto, O. X., West, M., Khan, M., & de Cesare, S. (2020). Implicit requirements for ontological multi-level types in the UNICLASS classification. *Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings*, 1–8. 
 - [\(Partridge 2022 Top-Level Categories\)](#) (see page 178) — Partridge, C. (2022). Top-Level Categories: Categories for the Top-Level Ontology of the Information Management Framework (DRAFT) (p. 25). NDT. 
 - [\(Pigozzi 2021 Belief Merging and Judgment Aggregation\)](#) (see page 179) — Pigozzi, G. (2021). Belief Merging and Judgment Aggregation. In E. N. Zalta (Ed.), *The Stanford Encyclopedia of Philosophy* (Spring 2021). Metaphysics Research Lab, Stanford University. <https://plato.stanford.edu/archives/spr2021/entries/belief-merging/> 
 - [\(Schaffer 2015 What Not to Multiply Without Necessity\)](#) (see page 180) — Schaffer, J. (2015). What Not to Multiply Without Necessity. *Australasian Journal of Philosophy*, 93(4), 644–664. <https://doi.org/10.1080/00048402.2014.992447> 

1.8.1 (Florio 2021 The Many and the One)

Florio, S., & Linnebo, Ø. (2021). *The many and the one: A philosophical study of plural logic* (First edition). Oxford University Press. [🔗 \(see page 148\)](#)

1.8.2 (Florio 2022 Core Constructional Ontology)

Florio, S., & Linnebo, Ø. (2022). *Core Constructional Ontology: The Foundation for the Top-Level Ontology of the Information Management Framework (DRAFT)* (v. 1.91). NDT. [🔗 \(see page 149\)Core_Constructional_Ontology_DRAFT_6_April_2022_v1.91.pdf](#)

1.8.3 (Graham 2015 NORSOK Z-014: A 21st Century Update)

Graham, J., Khan, M., & Partridge, C. (2015). NORSOK Z-014: A 21st Century Update. *59th Annual Meeting of the AACE International 2015*, 1,2. [🔗 \(see page 150\)](#)

1.8.4 (Hansson 2022 Logic of Belief Revision)

Hansson, S. O. (2022). Logic of Belief Revision. In E. N. Zalta (Ed.), *The Stanford Encyclopedia of Philosophy* (Spring 2022). Metaphysics Research Lab, Stanford University. <https://plato.stanford.edu/archives/spr2022/entries/logic-belief-revision/>.  (see page 157)

1.8.5 (ISO/IEC 21838-1:2021)

International Organization for Standardization. (2021). *ISO/IEC 21838-2:2021 Information technology—Top-level ontologies (TLO)—Part 1: Requirements*. International Organization for Standardization. [🔗 \(see page 152\)](#)

1.8.6 (ISO/IEC 24707:2018)

International Organization for Standardization. (2018). *ISO/IEC 24707:2018 Information technology—Common Logic (CL)—A framework for a family of logic-based languages*. International Organization for Standardization. [🔗 \(see page 153\)](#)

1.8.7 (Leonard 2023 Epistemological Problems of Testimony)

Leonard, N. (2023). Epistemological Problems of Testimony. In E. N. Zalta & U. Nodelman (Eds.), *The Stanford Encyclopedia of Philosophy* (Spring 2023). Metaphysics Research Lab, Stanford University. <https://plato.stanford.edu/archives/spr2023/entries/testimony-episprob/>  (see page 157)

1.8.8 (Lewis 1968 Counterpart Theory and Quantified Modal Logic)

Lewis, D. K. (1968). Counterpart Theory and Quantified Modal Logic. *The Journal of Philosophy*, 65(5), Article 5. <https://doi-org.uow.idm.oclc.org/10.2307/2024555>  (see page 155)

1.8.9 (Lewis 1986 On the plurality of worlds)

Lewis, D. K. (1986). *On the plurality of worlds*. Blackwell Publishers.  ([see page 156](#))

1.8.10 (Linnebo 2022 Plural Quantification)

Linnebo, Ø. (2022). Plural Quantification. In E. N. Zalta (Ed.), *The Stanford Encyclopedia of Philosophy* (Spring 2022). Metaphysics Research Lab, Stanford University. <https://plato.stanford.edu/archives/spr2022/entries/plural-quant/>  (see page 157)

1.8.11 (Pagin 2021 Assertion)

Pagin, P., & Marsili, N. (2021). Assertion. In E. N. Zalta (Ed.), *The Stanford Encyclopedia of Philosophy* (Winter 2021). Metaphysics Research Lab, Stanford University. <https://plato.stanford.edu/archives/win2021/entries/assertion/>  (see page 157)

1.8.12 (Partridge 1996 Business Objects)

Partridge, C. (1996). *Business Objects: Re-Engineering for Re-Use* (1st Edition). Oxford: Butterworth-Heinemann. [🔗 \(see page 159\)](#)

1.8.12.1 (pp. 214-215) here and now - de se and nunc

1.8.12.1.1 Chapter 9 Constructing Signs for Business Objects

1.8.12.1.1.1 6 Constructing signs for dynamic objects

pp. 214-215

So far we have been constructing signs for timeless objects. We now look at signs for the time-bound dynamic objects described in **Chapter 8** :

- the here event class,
- the now event class, and
- current couples.

Figure 9_36 - Sign for the -here- event class

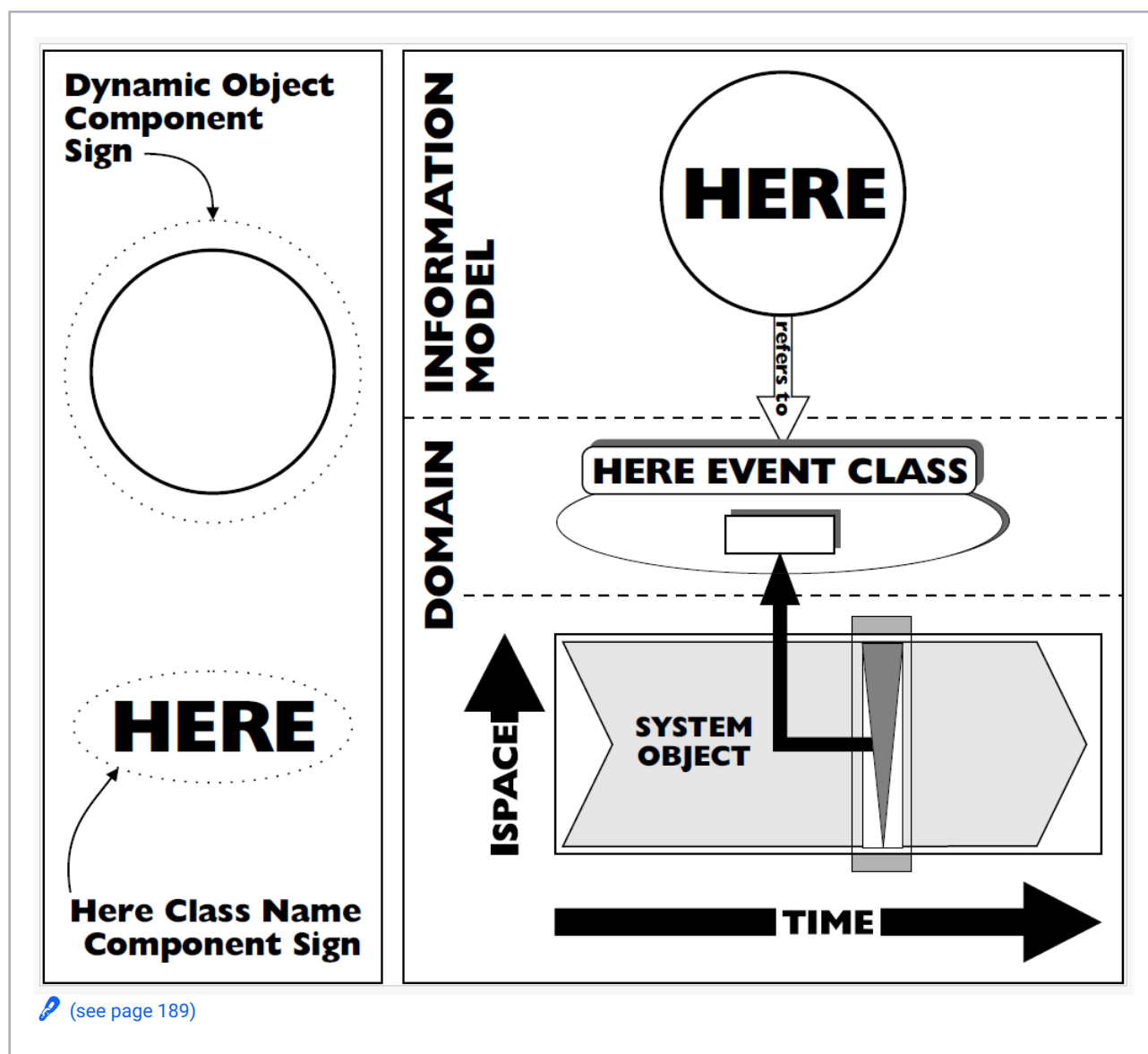
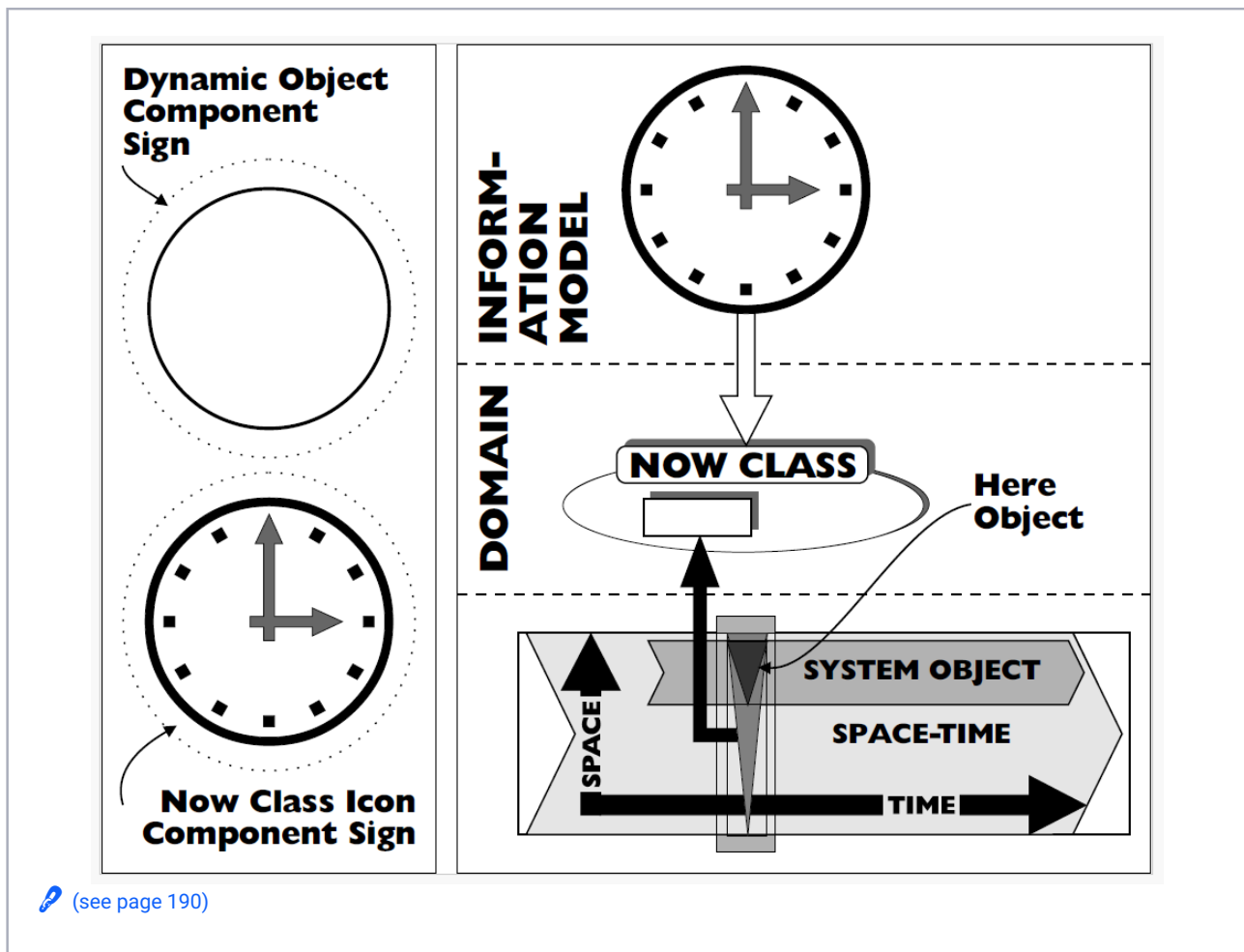


Figure 9_37 - Sign for the -now- event class



1.8.12.1.1.2 6.1 Constructing a sign for the 'here' event class

The here event class has as its single member an instantaneous time-slice of the system object—a physical body. This member moves, like all the dynamic events, along the time dimension with the system's 'consciousness'. The composite sign for the here event class is a circle, the component sign for a dynamic object, with the name 'HERE' in it (shown in **Figure 9.36**).

1.8.12.1.1.3 6.2 Constructing a sign for the 'now' event class

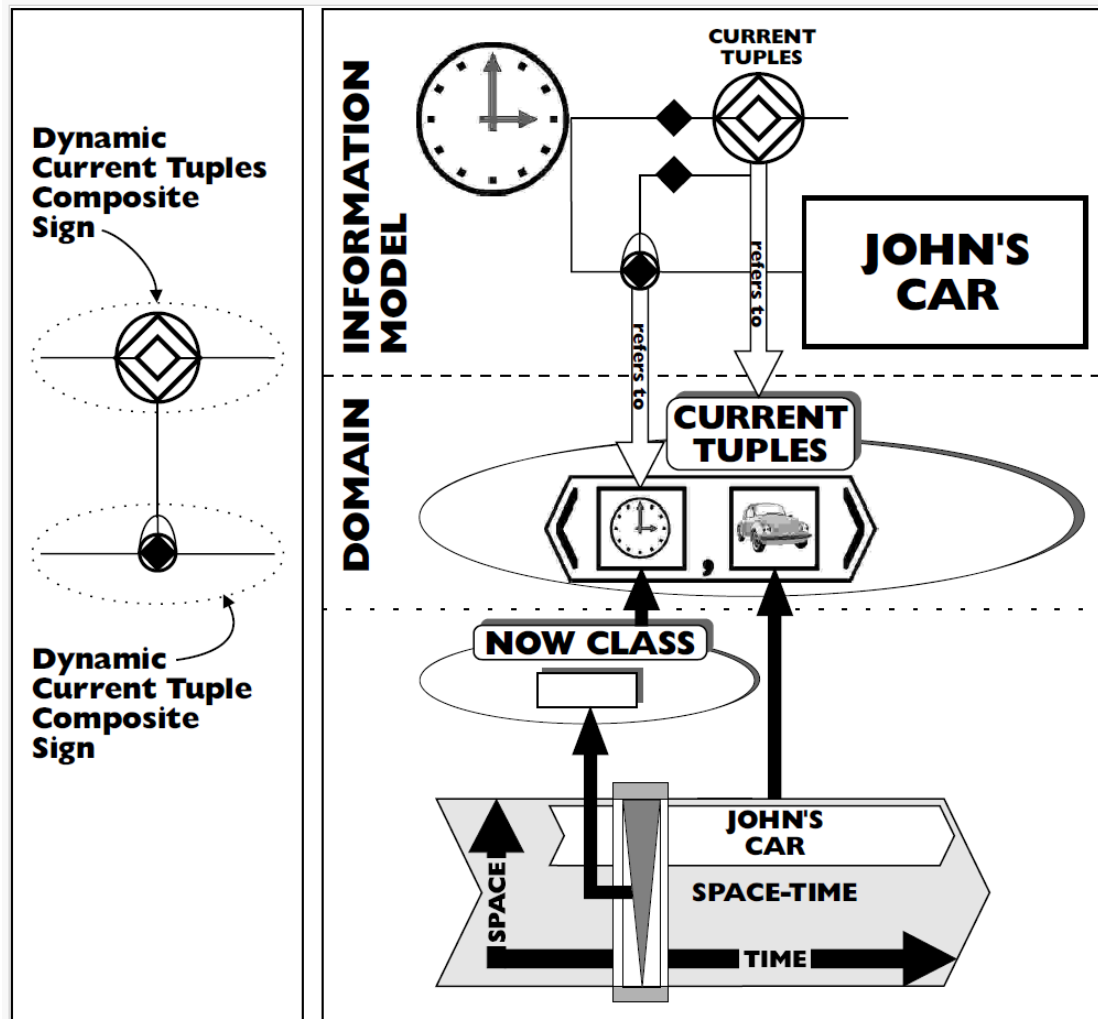
The 'now' event class has as its single member the instant that contains the 'here' event class's member. It is signed using a circle containing a clock face – as shown in **Figure 9.37**.

1.8.12.1.1.4 6.3 Constructing a sign for a current tuple

We now construct the signs for the current tuples class and its members, current tuples. However, the current tuples class sign is, to an extent, superfluous because any tuple signed as current automatically belongs to the current tuples class. This is done using a component dynamic circle sign (illustrated in **Figure 9.38**). As you can see, one of the sign's current tuple places is linked to the now event class, the other(s) to the object(s) currently classified as current.

You can also see the current tuples class signed in **Figure 9.38** as a tuples class with the dynamic circle component sign around it.

Figure 9_38 - Sign for a current tuple



[\(see page 191\)](#)

[\(see page 160\)](#)

(Partridge 1996 Business Objects)

Partridge, C. (1996). *Business Objects: Re-Engineering for Re-Use* (1st Edition). Oxford: Butterworth-Heinemann. [\(see page 159\)](#)

1.8.12.2 (pp. 220, 227) object patterns - individual and class levels

1.8.12.2.1 Chapter 10 Constructing Signs for Business Objects' Patterns

1.8.12.2.1.1 2.1 Individual object level patterns

p. 220

(p. 220) individual object level patterns

At the individual object level, any number of individual objects can have the distinct or overlapping pattern, but the pattern is at its simplest when only two objects are involved. So we start by looking at pairs of distinct and overlapping objects then move onto larger groups of objects. Finally, we examine the following associated patterns:

- Inheriting distinct and overlapping patterns,
- Known and unknown distinct and overlapping individual objects,
- Partition patterns for distinct individual objects,
- Intersection pattern for overlapping individual objects, and
- Fusion pattern for individual objects.

We also work out what objects the signs for distinct and overlapping individual patterns refer to. [🔗 \(see page 166\)](#)

1.8.12.2.1.2 2.2 Class object level patterns

p. 227

(p. 227) class object level patterns

The distinct and overlapping patterns between extensions, which we have just examined for individual objects, appear again at the class level. Although, at that level, the super-sub-class hierarchy plays the role of the whole-part hierarchy. We now analyse the class level patterns in the same way as we analysed the individual object level ones. Like before, we start with the simple patterns that hold between pairs of distinct and overlapping classes before moving onto larger groups of classes. We then examine a similar set of associated patterns:

- Inheriting distinct and overlapping class patterns,
- Known and unknown distinct and overlapping classes,
- Partitioning patterns for distinct classes,
- Intersection pattern for overlapping classes, and

- Fusion pattern for classes.

We also work out what objects the distinct and overlapping class signs refer to. [🔗 \(see page 167\)](#)

(Partridge 1996 Business Objects)

Partridge, C. (1996). *Business Objects: Re-Engineering for Re-Use* (1st Edition). Oxford: Butterworth-Heinemann. [🔗 \(see page 159\)](#)

1.8.12.2.2 (p. 220) individual object level patterns

1.8.12.2.2.1 Chapter 10 Constructing Signs for Business Objects' Patterns

2.1 Individual object level patterns

p. 220

At the individual object level, any number of individual objects can have the distinct or overlapping pattern, but the pattern is at its simplest when only two objects are involved. So we start by looking at pairs of distinct and overlapping objects then move onto larger groups of objects. Finally, we examine the following associated patterns:

- Inheriting distinct and overlapping patterns,
- Known and unknown distinct and overlapping individual objects,
- Partition patterns for distinct individual objects,
- Intersection pattern for overlapping individual objects, and
- Fusion pattern for individual objects.

We also work out what objects the signs for distinct and overlapping individual patterns refer to. [🔗 \(see page 166\)](#)

(Partridge 1996 Business Objects)

Partridge, C. (1996). *Business Objects: Re-Engineering for Re-Use* (1st Edition). Oxford: Butterworth-Heinemann. [🔗 \(see page 159\)](#)

1.8.12.2.3 (p. 227) class object level patterns

1.8.12.2.3.1 Chapter 10 Constructing Signs for Business Objects' Patterns

2.2 Class object level patterns

p. 227

The distinct and overlapping patterns between extensions, which we have just examined for individual objects, appear again at the class level. Although, at that level, the super–sub-class hierarchy plays the role of the whole–part hierarchy. We now analyse the class level patterns in the same way as we analysed the individual object level ones. Like before, we start with the simple patterns that hold between pairs of distinct and overlapping classes before moving onto larger groups of classes.

We then examine a similar set of associated patterns:

- Inheriting distinct and overlapping class patterns,
- Known and unknown distinct and overlapping classes,
- Partitioning patterns for distinct classes,
- Intersection pattern for overlapping classes, and
- Fusion pattern for classes.

We also work out what objects the distinct and overlapping class signs refer to. [🔗 \(see page 167\)](#)

(Partridge 1996 Business Objects)

Partridge, C. (1996). *Business Objects: Re-Engineering for Re-Use* (1st Edition). Oxford: Butterworth-Heinemann. [🔗 \(see page 159\)](#)

1.8.12.3 patterns - derived

1.8.12.3.1 6 A pattern for compacting classes

6 A pattern for compacting classes

So far, in this chapter, we have looked at how object syntax helps us model objects. Now, we turn our attention to a pattern that helps us generalise classes and so compact the model. This is the pattern of tuples classes defining their place classes.

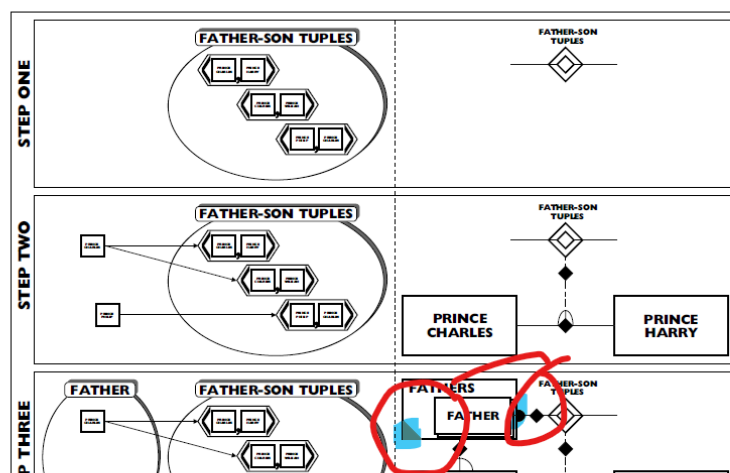
Once we identify the pattern, we generalise the place classes up the super-sub-class hierarchy. We can then eliminate the original, less general, place classes. This compacts the model without compromising its information content. This is a good illustration of one way in which compacting works and how we handle it within object syntax. We shall re-use this compacting pattern in the worked examples in Part Six.

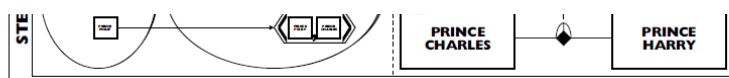
6.1 Constructing an example of the pattern

To illustrate the compacting, we need an example of the pattern. We get one by constructing a derived place class from a tuples class. Step one, shown in *Figure 10.45*, is taking the father-child tuples class. At this stage, it has no occupied class places. Step two is identifying in each of the member tuples, the object that occupies the father

place. Step three is collecting all these objects into a class. This gives us a fathers class that occupies one of the father-child tuples class places.

Figure 10.45:
Constructing the
logically depend-
ent place class
fathers

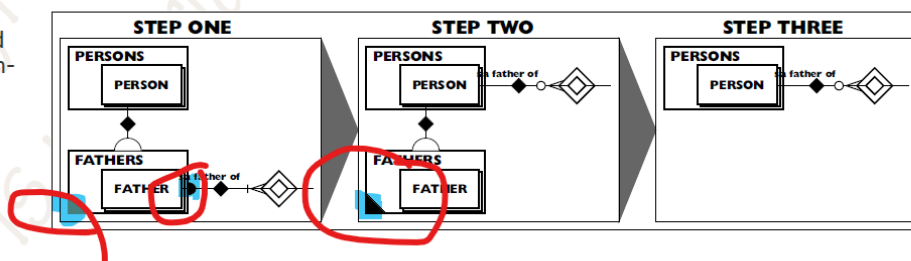




The class fathers is defined as those persons who have a father-child tuple linking them to a child—so it is logically dependent on the father-child tuples class. This makes it derived. This is modelled in the usual way; with logical dependency and derived signs (shown in *Figure 10.45*).

We will spot this pattern frequently if we keep asking whether there is a logical dependency between a tuples class and its place classes. Until now, we have tended to assume that they are logically independent. In this father-child tuples case, and many other cases, if we had asked ourselves the question, we would have realised that there is a logical dependency.

Figure 10.46:
Making a derived
place class redund-
ant



6.2 Using the pattern to compact the model

We now have an example of the pattern of tuples classes defining place classes. So we can illustrate the compacting. We do this in the three steps shown in *Figure 10.46*. In the first step, we generalise the fathers class (the occupied class place) up the super-

sub-class hierarchy to the persons class. In the second step, we generalise the 'is a father of' occupied class place from the father class to the persons class. At this stage, the fathers class no longer has a role to play; so, we classify it as redundant. The grey derived component sign in step one becomes a black redundant component sign. In the third and final step, we eliminate the now redundant fathers class from the model.

Often, when we are growing a business model, we construct classes that are logically dependent on tuples classes. These normally serve a purpose during the early stages. But, in most cases, they are redundant and so do not need to be implemented. As the model matures, we compact it by eliminating the redundant classes.

In this example of the compacting process, we eliminated the fathers class. However, it is sometimes useful to keep a record of redundant classes. Then, we do not eliminate the class but leave it in the model flagged as redundant. It then occupies a kind of limbo, kept in the model for reference purposes only.

1.8.13 (Partridge 2011 An Information Model for Geospatial and Temporal References)

Partridge, C. (2011). *An Information Model for Geospatial and Temporal Reference*. BORO. https://www.academia.edu/35739957/Ontology_then_agentology_A_finer_grained_framework_for_enterprise_modelling  (see page 170)

1.8.14 (Partridge 2013 Geospatial and Temporal Reference)

Partridge, C. (2013). Geospatial and Temporal Reference – A Case Study Illustrating (Radical) Refactoring. *ONTOBRAS-2013 6th Ontology Research Seminar in Brazil*. https://www.academia.edu/27433806/Geospatial_and_temporal_reference_A_case_study_illustrating_radical_refactoring  (see page 171)

1.8.15 (Partridge 2016 Formalization of the classification pattern)

Partridge, C., Cesare, S. de, Mitchell, A., & Odell, J. (2016). Formalization of the Classification Pattern: Survey of Classification Modeling in Information Systems Engineering. *Software & Systems Modeling*, 1–37. <https://doi.org/10.1007/s10270-016-0521-5>  (see page 172)

1.8.16 (Partridge 2018 Ontology then Agentology)

Partridge, C., deCesare, S., Mitchell, A., Leon, A., Gailly, F., & Khan, M. (2018). Ontology then Agentology: A Finer Grained Framework for Enterprise Modelling. *Proceedings of the 6th International Conference on Model-Driven Engineering and Software Development (MODELSWARD 2018)*, 454–463. <https://www.academia.edu/35335427>  (see page 173)

1.8.17 (Partridge 2018 Thoroughly Modern Accounting)

Partridge, C., Khan, M., Cesare, S. de, Gailly, F., Verdonck, M., & Mitchell, A. (2018). Thoroughly Modern Accounting: Shifting to a De Re Conceptual Pattern for Debits and Credits. *International Conference on Conceptual Modeling (ER 2018)*, 11158, 134. [🔗 \(see page 174\)](#)

1.8.18 (Partridge 2019 Coordinate Systems)

Partridge, C., Mitchell, A., Loneragan, M., Atkinson, H., de Cesare, S., & Khan, M. (2019). Coordinate Systems: Level Ascending Ontological Options. *2019 ACM/IEEE 22nd International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*, 78–87. <https://www.academia.edu/40354620>
 (see page 157)

1.8.19 (Partridge 2019 Grounding for an Enterprise Computing Nomenclature Ontology)

Partridge, C., Mitchell, A., & de Cesare, S. (2019). *Grounding for an Enterprise Computing Nomenclature Ontology*. International Conference on Conceptual Modeling, 457–465. [🔗 \(see page 157\)](#)

1.8.20 (Partridge 2020 Implicit Requirements for Ontological Multi-Level Types in the UNICLASS Classification)

Partridge, C., Mitchell, A., da Silva, M., Soto, O. X., West, M., Khan, M., & de Cesare, S. (2020). *Implicit requirements for ontological multi-level types in the UNICLASS classification*. Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings, 1–8. [🔑 \(see page 177\)](#)

1.8.21 (Partridge 2022 Top-Level Categories)

Partridge, C. (2022). *Top-Level Categories: Categories for the Top-Level Ontology of the Information Management Framework (DRAFT)* (p. 25). NDT. [\(see page 178\)Developing Thin Slices - final draft2.pdf](#)

1.8.22 (Pigozzi 2021 Belief Merging and Judgment Aggregation)

Pigozzi, G. (2021). Belief Merging and Judgment Aggregation. In E. N. Zalta (Ed.), *The Stanford Encyclopedia of Philosophy* (Spring 2021). Metaphysics Research Lab, Stanford University. <https://plato.stanford.edu/archives/spr2021/entries/belief-merging/>  (see page 157)

1.8.23 (Schaffer 2015 What Not to Multiply Without Necessity)

Schaffer, J. (2015). What Not to Multiply Without Necessity. *Australasian Journal of Philosophy*, 93(4), 644–664. <https://doi.org/10.1080/00048402.2014.992447>  (see page 170)

1.9 Acknowledgements

1.9.1 Lead Authors:



Andy Mitchell - BORO



Chris Partridge - BORO

1.9.3 Contributors:



Andreas Cola - Telicent

1.9.2 Authors:



Oscar Xiberta-Soto - BORO

2 Resources

- [ToLO Models](#) (see page 183)
- [Graphics](#) (see page 186)
- [Resources - Ontology Multi-Language Support Framework \(OMLSF\)](#) (see page 202)
- [bOSON 1.1 \(bCLEARer Ordnance Survey Open Names\) Surface Onomatology](#) (see page 215)
- [bOSON 1.2 \(bCLEARer Ordnance Survey Open Names\) Deep Onomatology - Coordinates](#) (see page 216)
- [IES 4.0](#) (see page 217)
- [Issued Document Identifier](#) (see page 218)

2.1 ToLO Models

2.1.1 ToLO EA (qea file) Current Model

(excluding WIP and FW)

[ies_top_level_ontology_wip_v0.27_20240326-current.qea.zip](#)

2.1.2 ToLO EA (qea file) Model including WIP and FW

Zipped (*.qea) file of this model: [ies_top_level_ontology_wip_v0.27_20240326-wip.qea.zip](#)

2.2 Graphics

- [Graphics - Documents \(see page 187\)](#)
- [Graphics - ToLO Architecture Framework \(see page 194\)](#)

2.2.1 Graphics - Documents

- [Graphics - \(Partridge 1996 Business Objects\)](#) (see page 188)

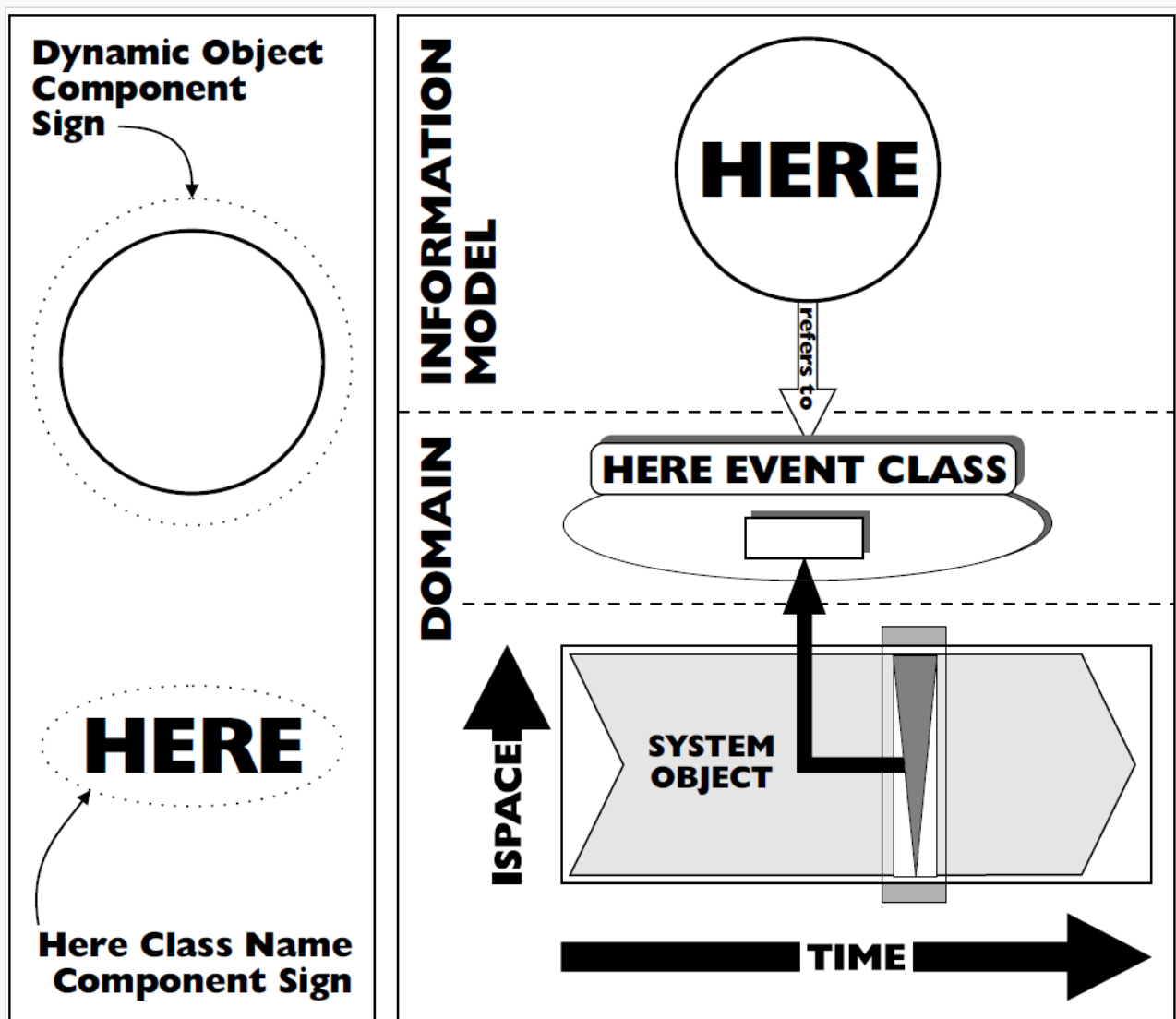
2.2.1.1 Graphics - (Partridge 1996 Business Objects)

(Partridge 1996 Business Objects)

Partridge, C. (1996). *Business Objects: Re-Engineering for Re-Use* (1st Edition). Oxford: Butterworth-Heinemann. [🔗 \(see page 159\)](#)

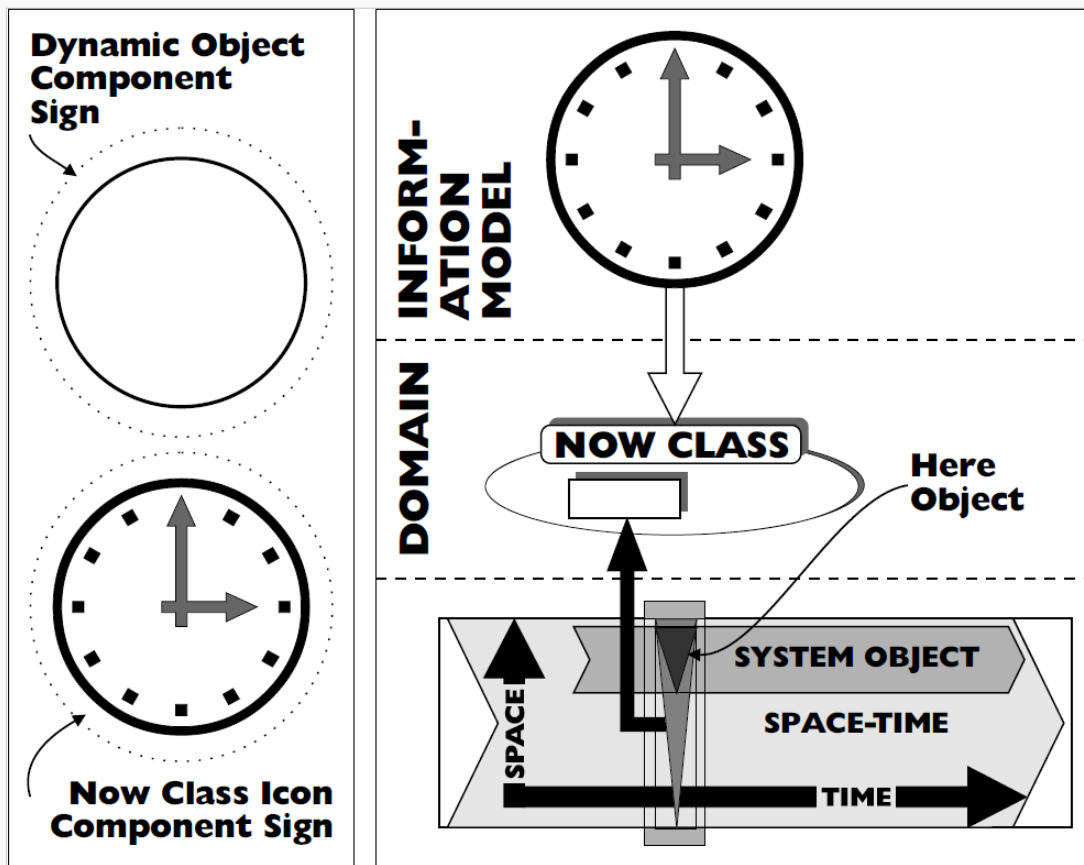
- [Figure 9_36 - Sign for the -here- event class \(see page 189\)](#)
- [Figure 9_37 - Sign for the -now- event class \(see page 190\)](#)
- [Figure 9_38 - Sign for a current tuple \(see page 191\)](#)
- [Figure 10_45 - Constructing the logically dependent place class fathers \(see page 192\)](#)
- [Figure 10_46 - Making a derived place class redundant \(see page 193\)](#)

2.2.1.1.1 Figure 9_36 - Sign for the -here- event class



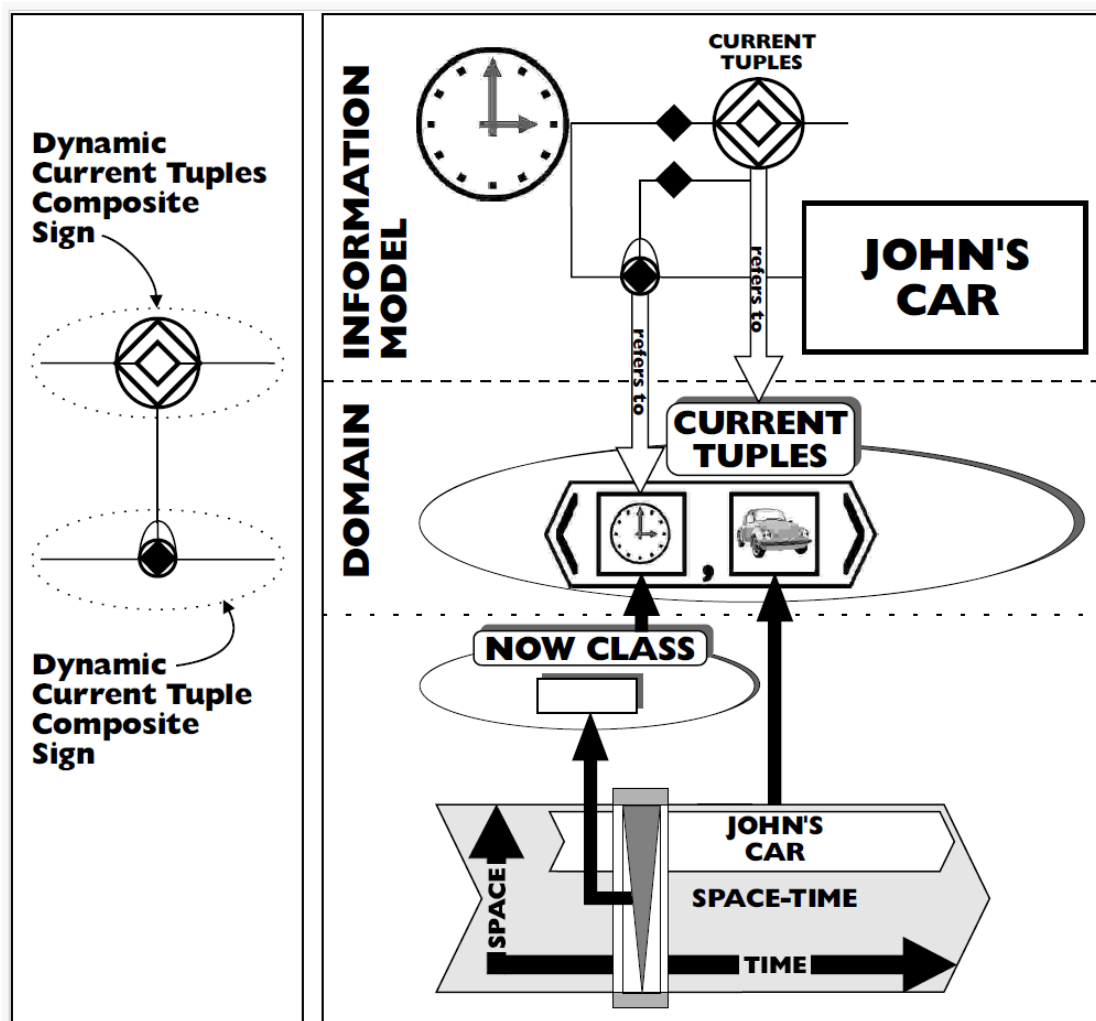
[\(see page 189\)](#)

2.2.1.1.2 Figure 9_37 - Sign for the -now- event class



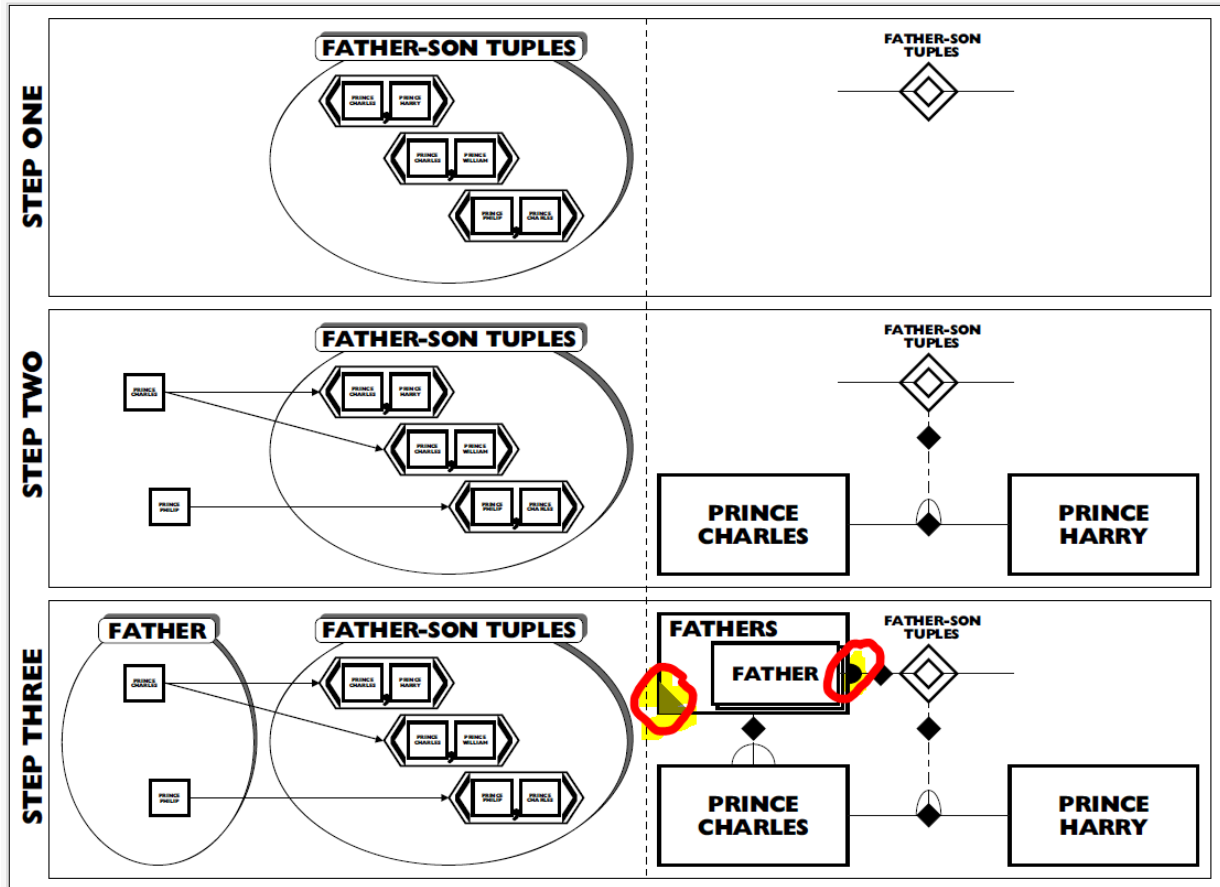
[\(see page 190\)](#)

2.2.1.1.3 Figure 9_38 - Sign for a current tuple



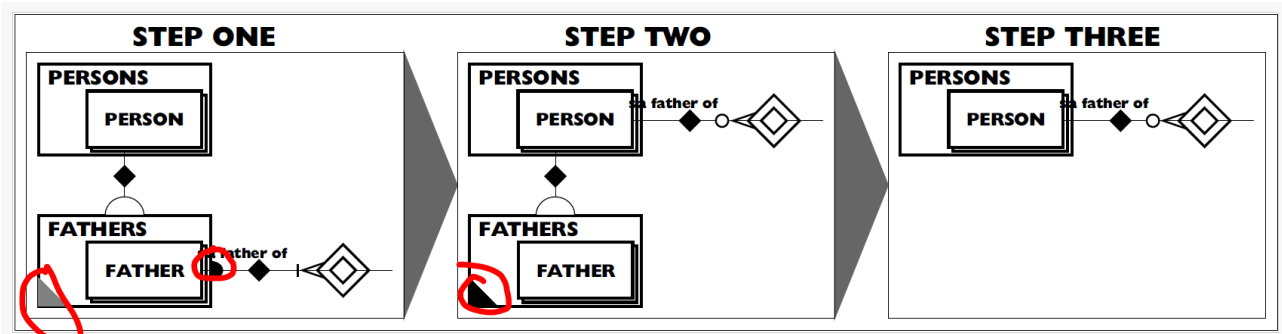
[\(see page 191\)](#)

2.2.1.1.4 Figure 10_45 - Constructing the logically dependent place class fathers



[\(see page 192\)](#)

2.2.1.1.5 Figure 10_46 - Making a derived place class redundant

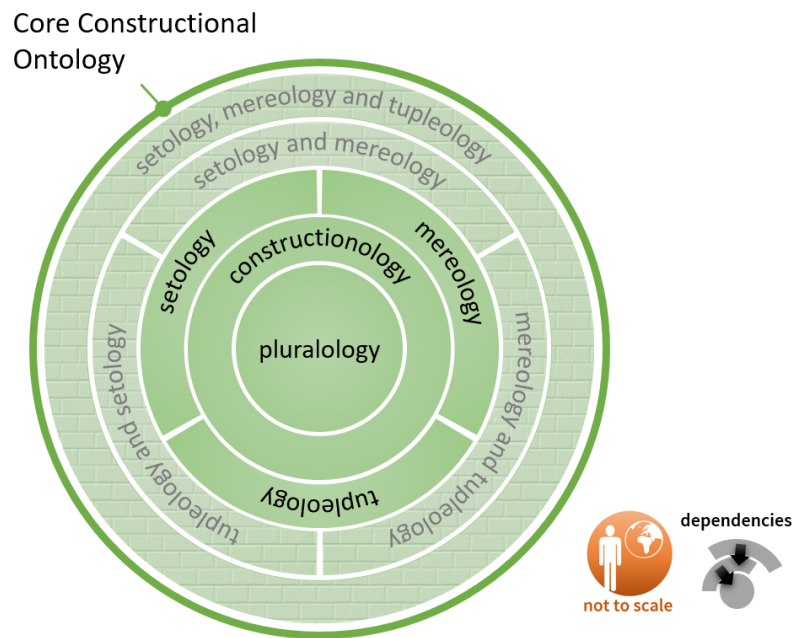


[\(see page 193\)](#)

2.2.2 Graphics - ToLO Architecture Framework

- [CoCO breakdown to components with dependencies \(circle\) \(see page 195\)](#)
- [CoCO breakdown to components with dependencies \(wall\) \(see page 196\)](#)
- [CoCO breakdown to modules with dependencies \(see page 197\)](#)
- [CoCO detailed breakdown to mortar with dependencies \(see page 198\)](#)
- [CoLO breakdown to modules with dependencies \(see page 199\)](#)
- [FoLO breakdown to components with dependencies \(see page 200\)](#)
- [ToLO breakdown to data sections with dependencies \(see page 201\)](#)

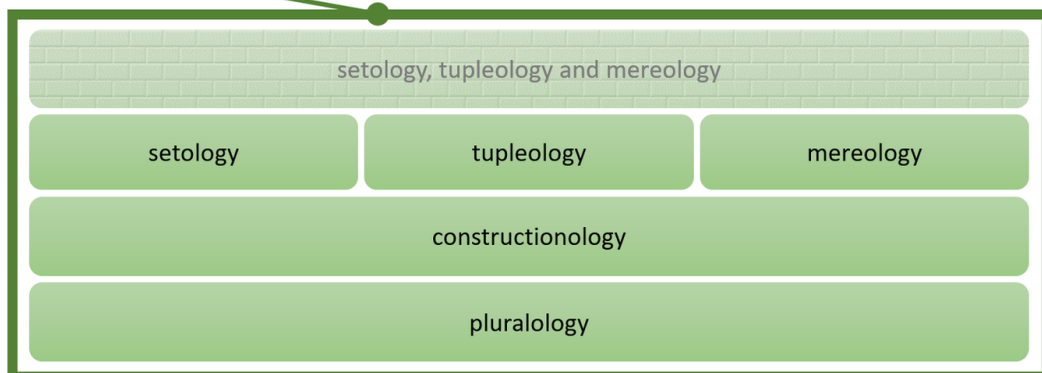
2.2.2.1 CoCO breakdown to components with dependencies (circle)



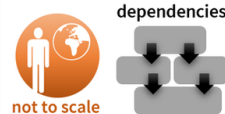
[\(see page 195\)](#)

2.2.2.2 CoCO breakdown to components with dependencies (wall)

Core Constructional Ontology

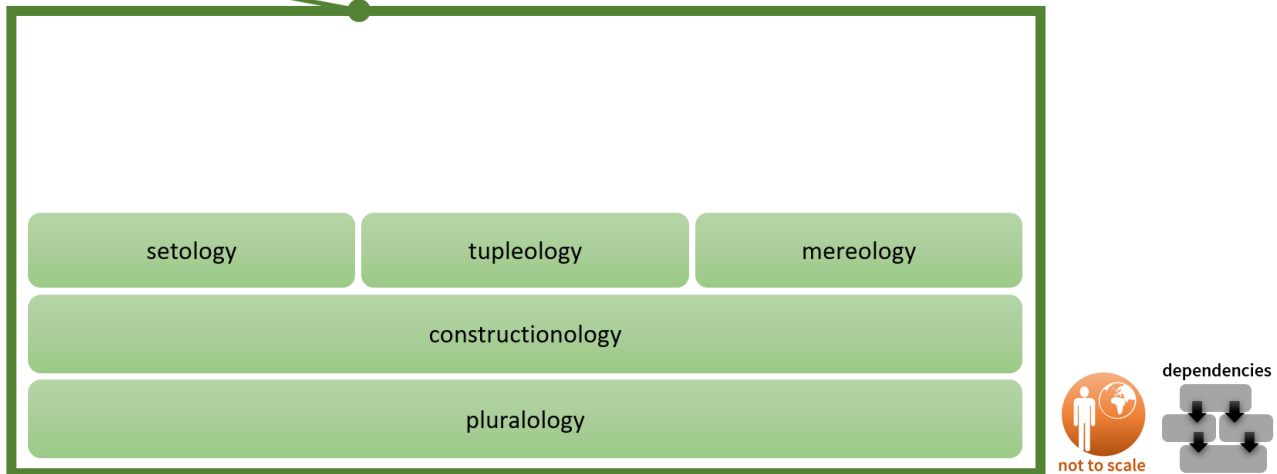


[🔗 \(see page 196\)](#)



2.2.2.3 CoCO breakdown to modules with dependencies

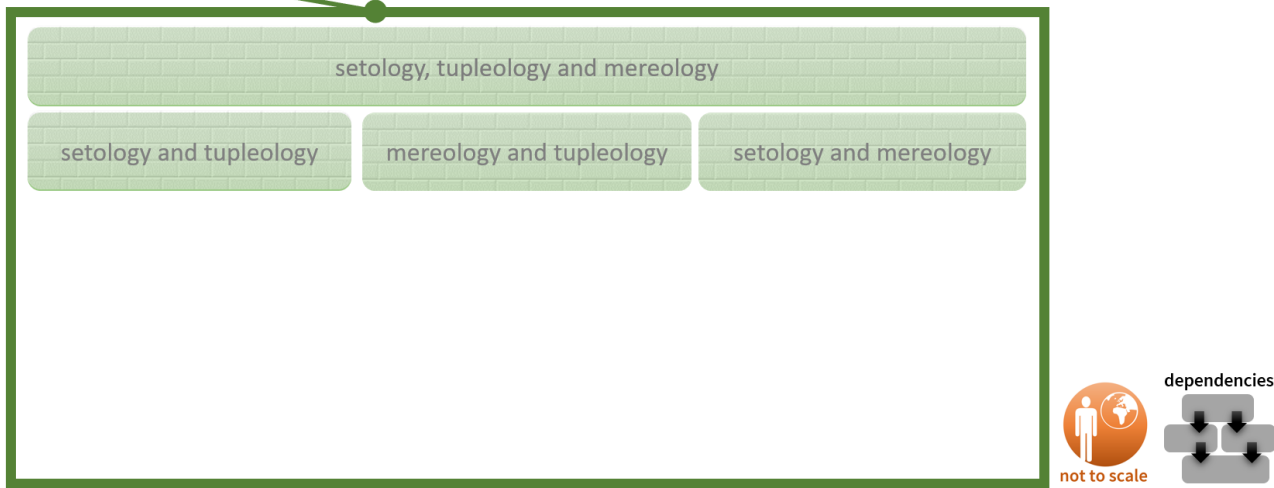
Core Constructional Ontology



[\(see page 197\)](#)

2.2.2.4 CoCO detailed breakdown to mortar with dependencies

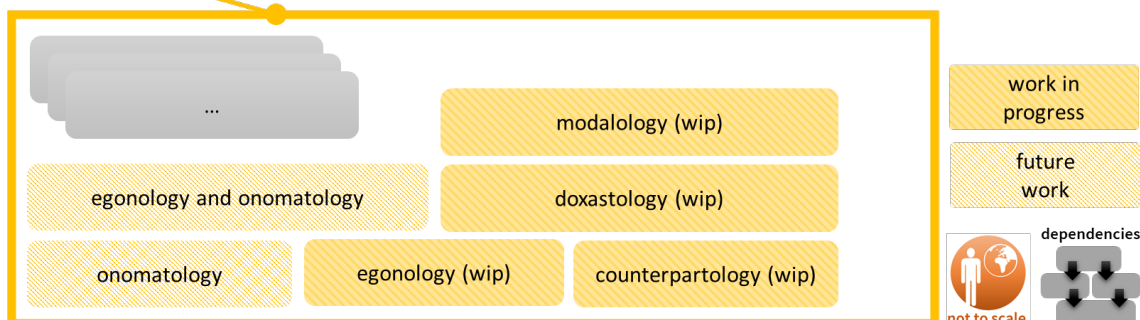
Core Constructional Ontology



 (see page 198)

2.2.2.5 CoLO breakdown to modules with dependencies

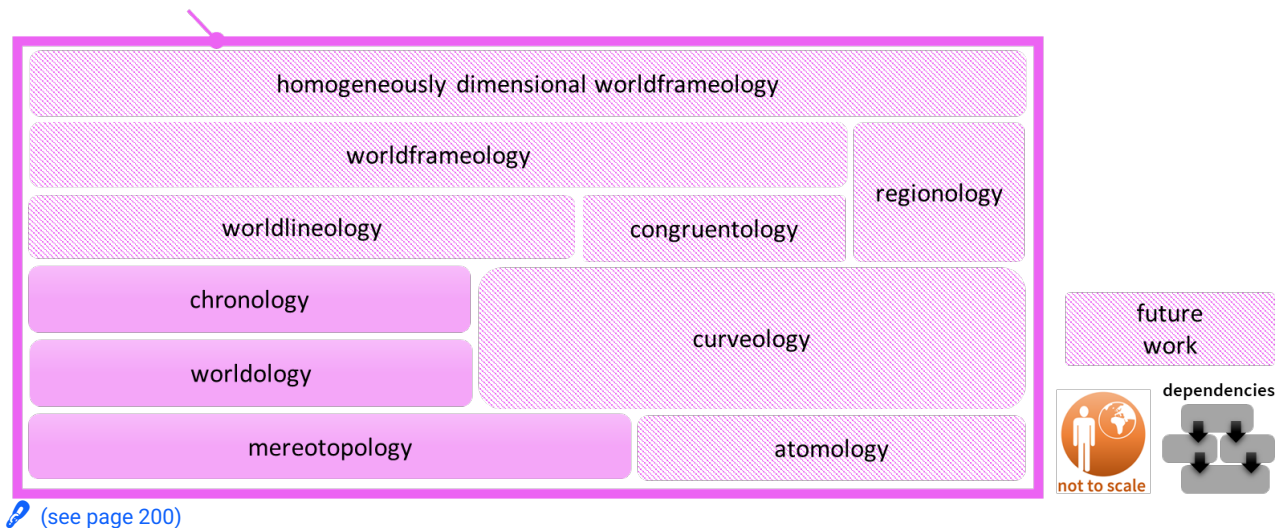
Common Level Ontology



[\(see page 199\)](#)

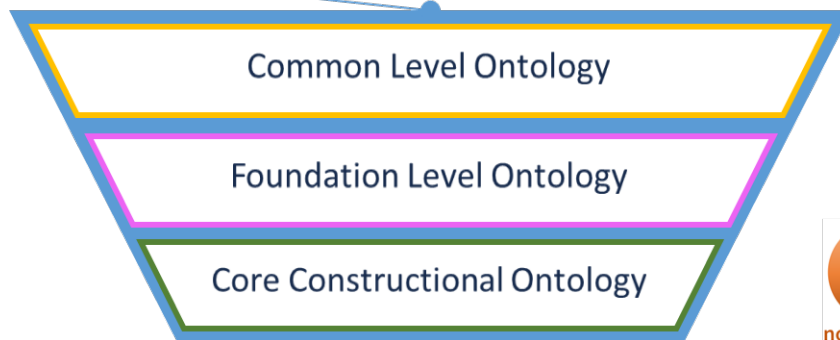
2.2.2.6 FoLO breakdown to components with dependencies

Foundation Level Ontology



2.2.2.7 ToLO breakdown to data sections with dependencies

Top Level Ontology



dependencies



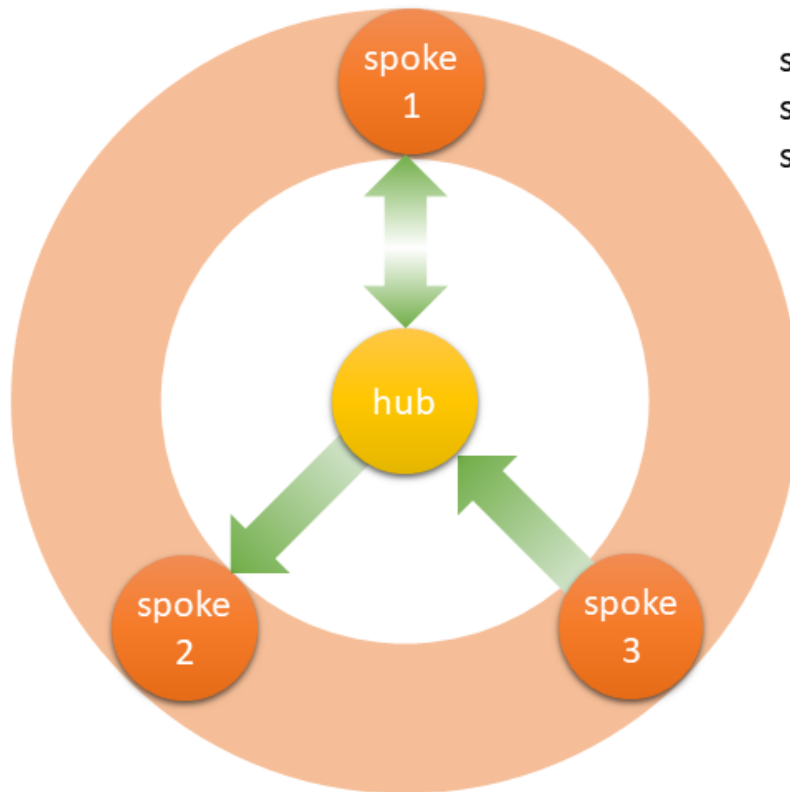
 (see page 201)

2.3 Resources - Ontology Multi-Language Support Framework (OMLSF)

- [Graphics - OMLSF - Architecture \(Proposed\) \(see page 203\)](#)
 - [simple hub and spoke structure \(see page 204\)](#)
 - [two-level hub and spoke structure \(see page 205\)](#)
 - [Option - UML Hub \(CL to TPTP\) \(see page 206\)](#)
 - [Option - UML Hub \(TPTP to CL\) \(see page 207\)](#)
 - [Option - CL Hub \(see page 208\)](#)
 - [Option - UML and CL Hub \(see page 209\)](#)
 - [UML as hub \(see page 210\)](#)
 - [ODM Overview \(see page 211\)](#)
- [Languages List \(see page 212\)](#)

2.3.1 Graphics - OMLSF - Architecture (Proposed)

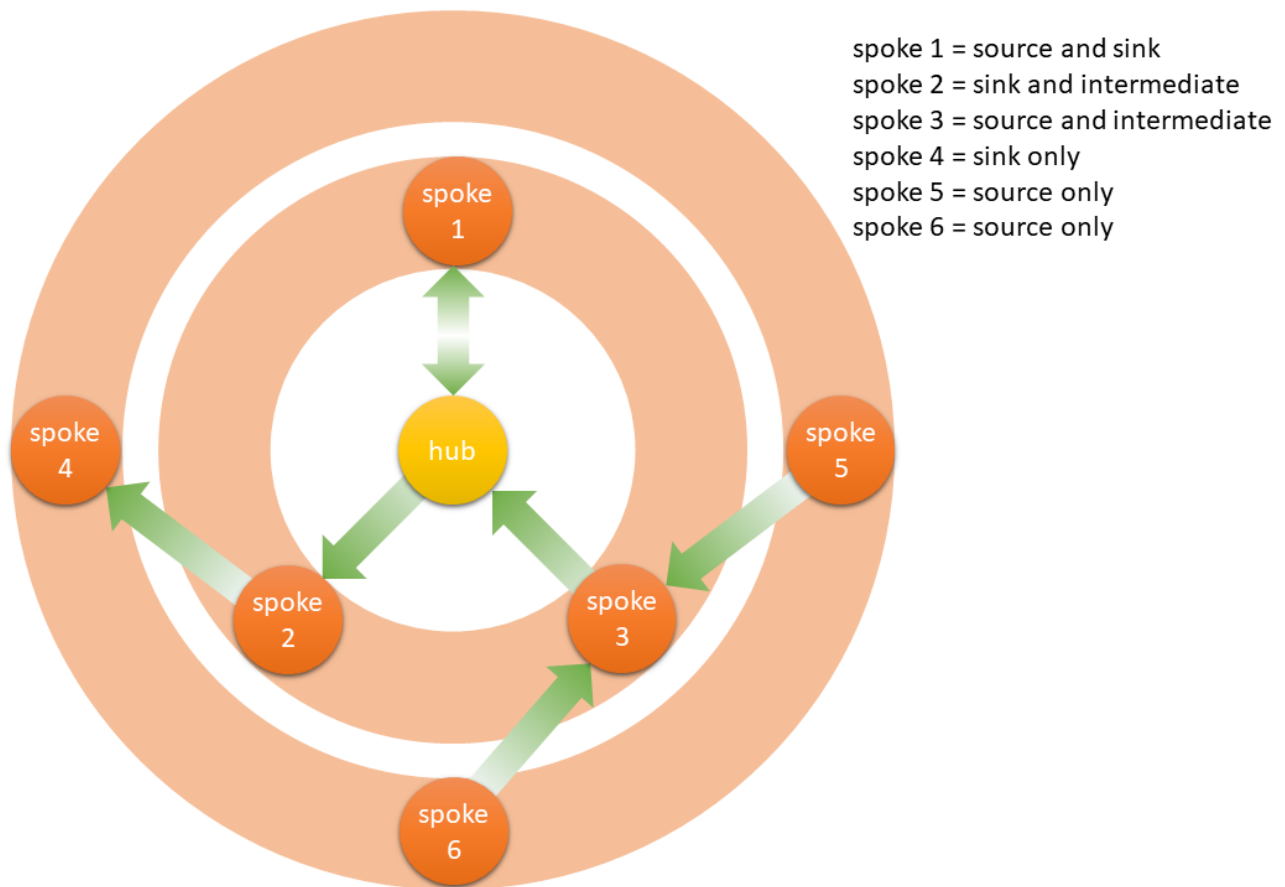
2.3.1.1 simple hub and spoke structure



spoke 1 = source and sink
spoke 2 = sink only
spoke 3 = source only

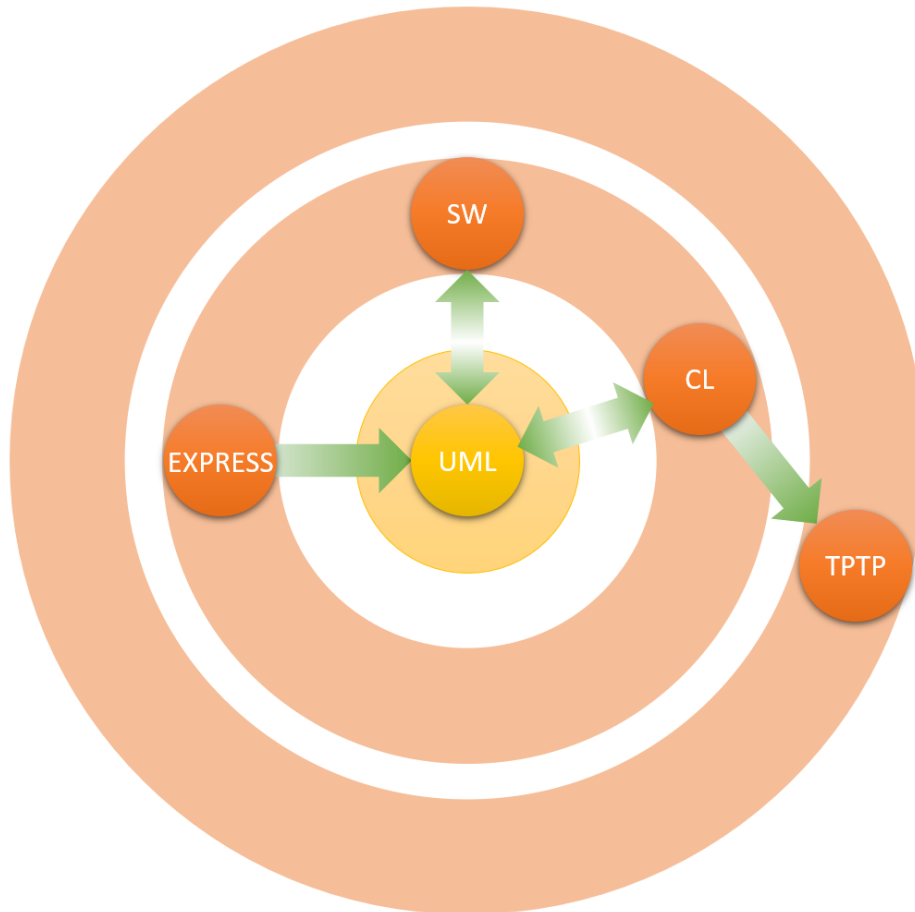
[🔗](#) (see page 204)

2.3.1.2 two-level hub and spoke structure



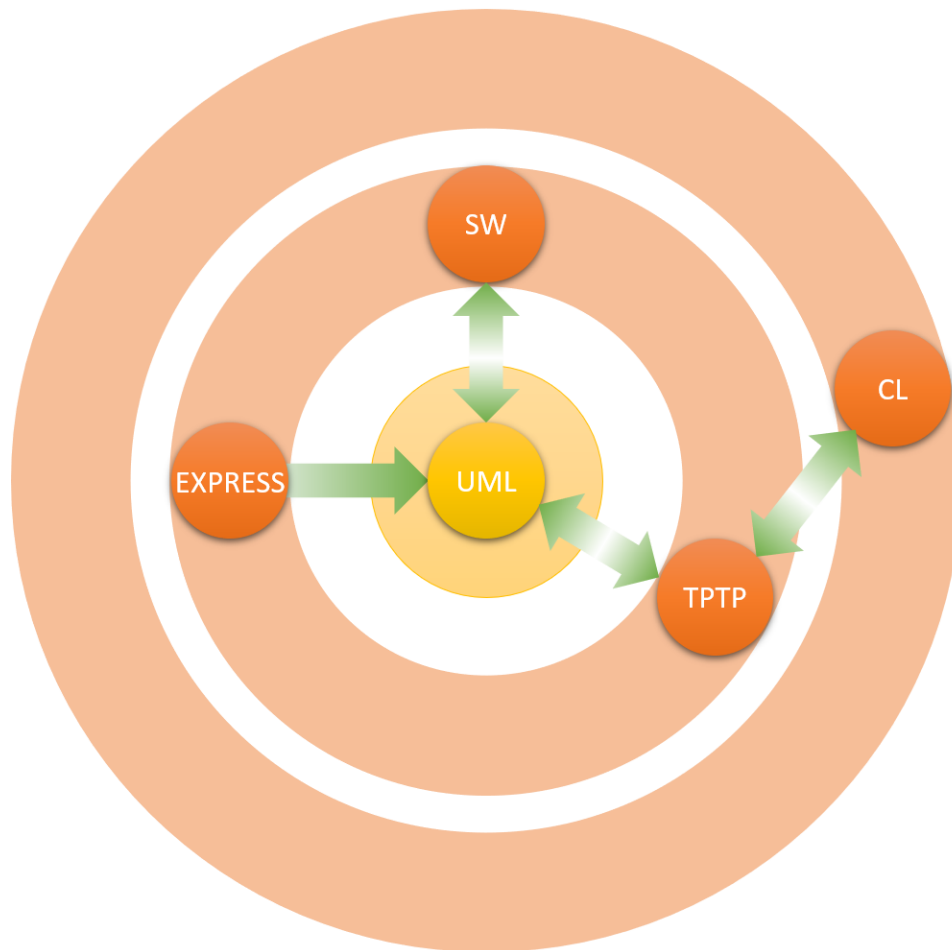
[🔗 \(see page 205\)](#)

2.3.1.3 Option - UML Hub (CL to TPTP)



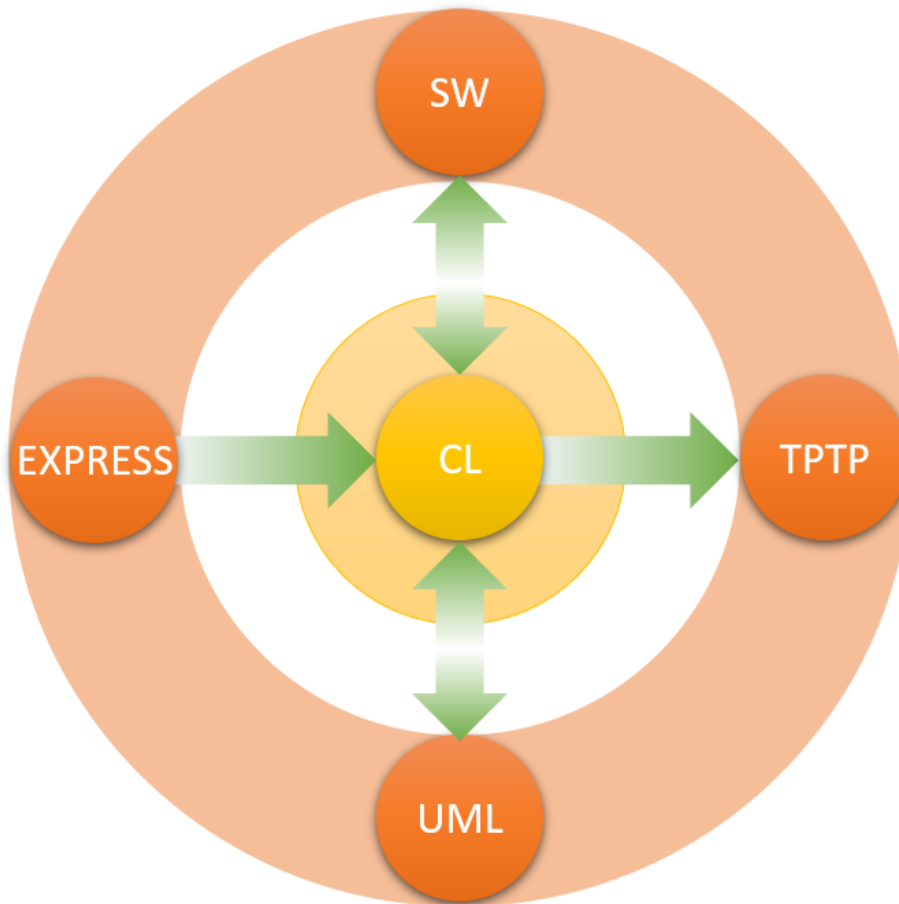
[🔗](#) (see page 206)

2.3.1.4 Option - UML Hub (TPTP to CL)



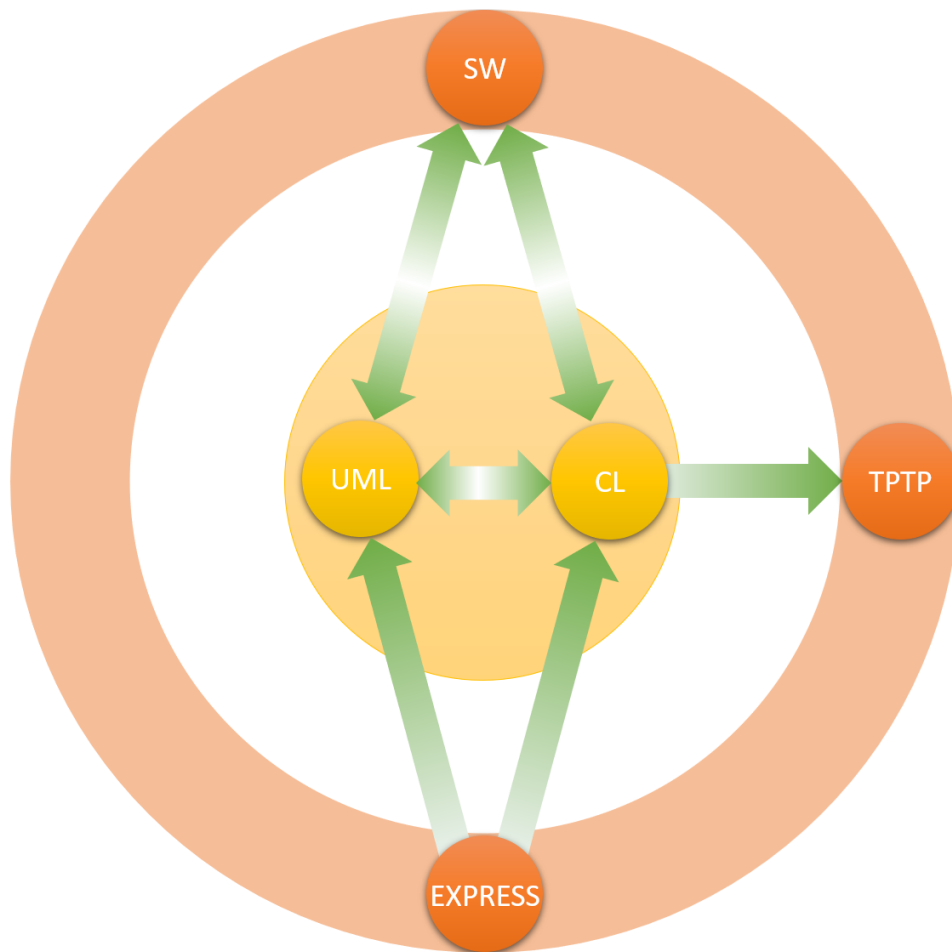
[\(see page 207\)](#)

2.3.1.5 Option - CL Hub



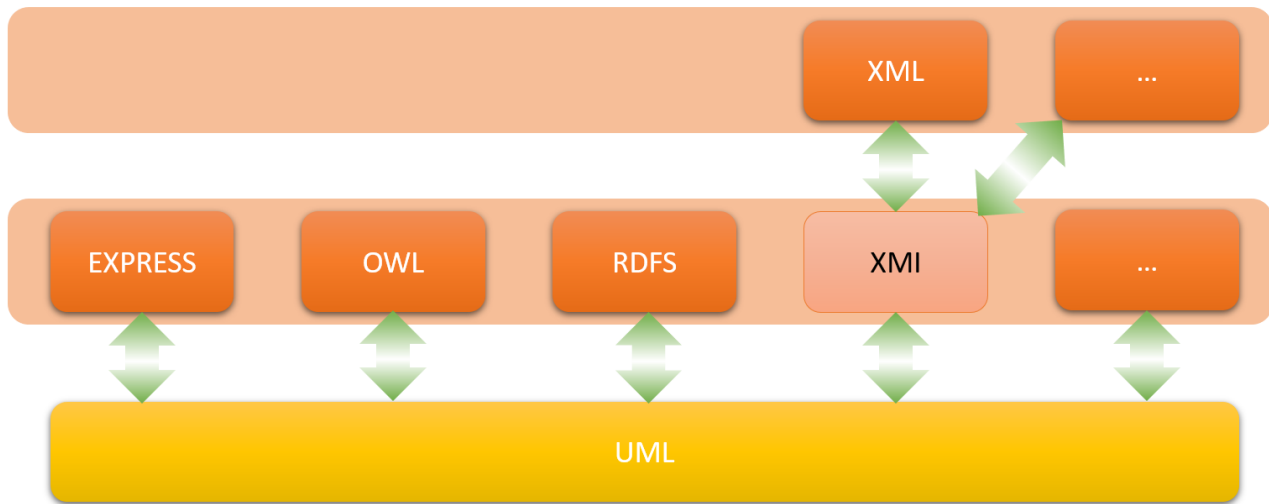
[🔗](#) (see page 208)

2.3.1.6 Option - UML and CL Hub



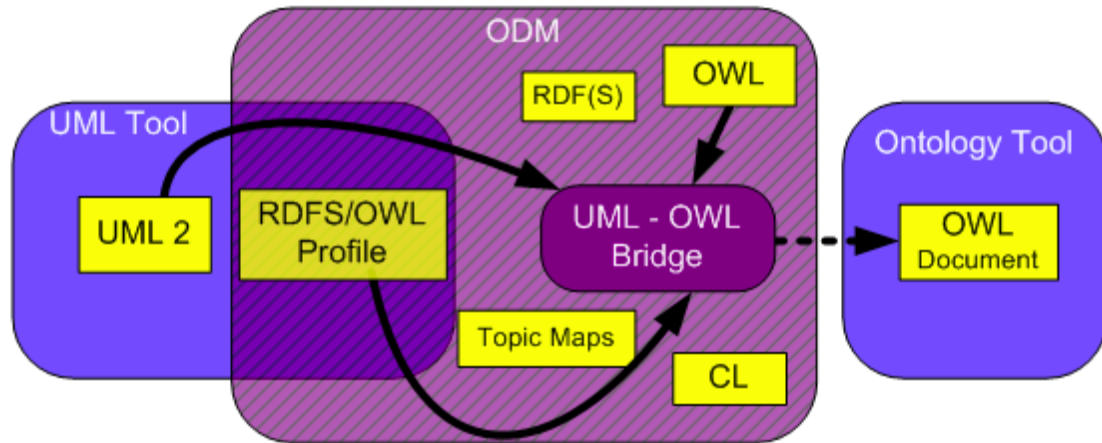
 (see page 209)

2.3.1.7 UML as hub



[🔗](#) (see page 210)

2.3.1.8 ODM Overview



ODM Overview

[🔗](#) (see page 211)

2.3.2 Languages List

2.3.2.1 Candidate Industry Standards Languages

Languages	Comments
CL (CLIF) (see page 69)	Not at all used commercially. “Common Logic (CL) is a framework for a family of logic languages, based on first-order logic, intended to facilitate the exchange and transmission of knowledge in computer-based systems.”
EXPRESS-STEP-File (see page 71)	Showing its age? EXPRESS is a standard data modeling language for product data. EXPRESS is formalized in the ISO Standard for the Exchange of Product model STEP (ISO 10303), and standardized as ISO 10303-11. STEP-File (ISO 10303-21) defines the encoding mechanism for representing data conforming to a particular schema in the EXPRESS data modeling language. EXPRESS-STEP-File - STEP-File (see page 82)
OWL (see page 92)	Not used much commercially. “The Web Ontology Language (OWL) is a family of knowledge representation languages for authoring ontologies.”
RDF (RDFS) (see page 91)	Used commercially. “The Resource Description Framework (RDF) is a family of World Wide Web Consortium (W3C) specifications[1] originally designed as a metadata data model.”
UML (see page 86)	Used commercially. Unified Modeling Language “The Unified Modeling Language (UML) is a general-purpose, developmental, modeling language in the field of software engineering that is intended to provide a standard way to visualize the design of a system.” ODM (UML Profile) extends UML's OCL to CL (CLIF) level ODM (UML Profile) (see page 88)

2.3.2.2 Other Candidate Languages

Lan gua ges	Comments
JSON (see page 97)	“JavaScript Object Notation is an open standard file format, and data interchange format, that uses human-readable text to store and transmit data objects consisting of attribute–value pairs and array data types (or any other serializable value).”
RDBMS (see page 98)	“a relational database management system is a database management system (DBMS) based on the relational model.”
SQL (see page 99)	Used commercially. Structured Query Language.
XML (XSD) (see page 95)	Used commercially. eXtensible Markup Language. “Extensible Markup Language (XML) is a markup language that defines a set of rules for encoding documents in a format that is both human-readable and machine-readable.”

[Pencil icon](#) (see page 212)

2.3.2.2.1 Non-Shortlisted Languages

Language	Comments
CycL	
DOGMA	Developing Ontology-Grounded Methods and Applications
F-Logic	Frame Logic

Language	Comments
KIF	Knowledge Interchange Format
Ontolingua	based on KIF
KL-ONE	
KM programm ing language	
LOOM (ontology)	
OCML	Operational Conceptual Modelling Language
OKBC	Open Knowledge Base Connectivity
PLIB	Parts LIbrary

See https://en.wikipedia.org/wiki/Ontology_language

2.4 bOSON 1.1 (bCLEARer Ordnance Survey Open Names) Surface Onomatology

The [bOSON 1.1 \(bCLEARer Ordnance Survey Open Names\) Surface Onomatology](#)⁹⁷ is a gitHub repository containing the python code that provides an example of the bCLEARer process applied to [Ordnance Survey Open Names](#)⁹⁸ dataset.

This code shows an example of how to automatically implement (at scale) a surface onomatology pattern over structured data.

The code transparently documents the transformations that arise from the data cleaning and ontological analysis that takes place in the different stages of the process, and it outputs the result in a self-contained folder system divided into sub-folders corresponding to each of the implemented bCLEARer stages and sub-stages (only the Collect, Load and early Evolve stages were implemented).

As is standard inspection practice, output is provided for each of the sub-stages. So the user can check the progress of the process by opening the csv files stored in the sub-stage output folders. The process also exports the final sub-stage as XML in the parent output folder.



This project is currently closed, but may be sporadically updated by the BORO Development Team in the future.

(see page 215)

⁹⁷ https://github.com/boro-alpha/bclearer_boson_1_1

⁹⁸ <https://www.ordnancesurvey.co.uk/business-government/products/open-map-names>

2.5 bOSON 1.2 (bCLEARer Ordnance Survey Open Names) Deep Onomatology - Coordinates

The [bOSON 1.2 \(bCLEARer Ordnance Survey Open Names\) Deep Onomatology](#)⁹⁹ is a gitHub repository containing the python code that provides an example of the bCLEARer process applied to the [Ordnance Survey Open Names](#)¹⁰⁰ dataset and the [INSPIRE](#)¹⁰¹ metamodel.

This code shows an example of how to automatically implement (at scale) a surface onomatology pattern over structured data.

The code transparently documents the transformations that arise from the data cleaning and ontological analysis that takes place in the different stages of the process, and it outputs the result in a self-contained folder system divided into sub-folders corresponding to each of the implemented bCLEARer stages and sub-stages.

As is standard inspection practice, output is provided for each of the sub-stages. So the user can check the progress of the process by opening the csv files stored in the sub-stage output folders. The process also exports the final sub-stage as XML in the parent output folder.

2.5.1 Repository

The repository for this code can be found [here](#)¹⁰².



This project is currently closed, but may be sporadically updated by the BORO Development Team in the future.

2.5.2 Resources

The OS Open Names GML source data can be downloaded from [here](#)¹⁰³.

The source INSPIRE metamodel can be downloaded from [here](#)¹⁰⁴.

(see page 216)

⁹⁹ https://github.com/boro-alpha/bclearer_boson_1_2

¹⁰⁰ <https://www.ordnancesurvey.co.uk/business-government/products/open-map-names>

¹⁰¹ <https://inspire.ec.europa.eu/>

¹⁰² https://github.com/boro-alpha/bclearer_boson_1_2

¹⁰³ <https://osdatahub.os.uk/downloads/open/OpenNames>

¹⁰⁴ <https://inspire.ec.europa.eu/portfolio/data-models>

2.6 IES 4.0

The Information Exchange Standard (IES) is a standard for information exchange developed within UK Government.

IES was developed by the UK Government with contributions from Dstl, Ministry of Defence, Metropolitan Police Service, Foreign & Commonwealth Office, Home Office and HMRC, with technical support from Aurora Consulting (to Home Office) and Telicent (to Dstl).

IES was developed by the UK Government with contributions from Dstl, Ministry of Defence, Metropolitan Police Service, Foreign & Commonwealth Office, Home Office, Department for Business and Trade (was BEIS), and HMRC, with technical support from Aurora Consulting (to Home Office) and Telicent (to Dstl).

There is a GitHub repo (<https://github.com/dstl/IES4>) which provides public access to the IES for collaboration purposes including the specification itself, sample data/code and supporting documentation.

2.7 Issued Document Identifier

TL/IN371/P04231771/02